第5章: Supabase セットアップとデータベース設計

> この章では、**Supabase**（スーパベース：BaaSと呼ばれる「バックエンドまるごと提供サービス」）というサービスを使って、アプリのデータを保存する**データベース**（Database：データを永続的に保管する仕組み。アプリを閉じてもデータが消えない箱のようなもの）を準備します。

この章で学ぶこと

- **データベースとは何か** — プログラムのデータを永続的に保存する「倉庫」
- **テーブル設計** — データをどのような「表」の形で保存するか考える作業
- **SQL の基本** — データベースに命令を出すための言語（SELECT, INSERT, UPDATE, DELETE）
- **RLS（Row Level Security：行レベルセキュリティ。「どの行に誰がアクセスできるか」を行ごとに制御する仕組み）** — 「誰がどのデータにアクセスできるか」を制御するセキュリティ機能
- **Supabase クライアント** — JavaScriptのコードからSupabaseに接続する方法

> **データベースはなぜ必要？** 変数や配列にデータを保持しても、ブラウザを閉じれば消えてしまいます。データベースはハードディスクにデータを保存するので、アプリを再起動してもデータが残ります。SNSの投稿、ECサイトの商品情報、そしてこのアプリの書籍情報も、すべてデータベースに保存されています。

> **補足: 用語の読み方**
>
> - **BaaS**（バース／ビーエーエーエス：Backend as a Service の略。サーバーやDBを自前で構築せずに使えるクラウドサービス）
> - **PostgreSQL**（ポストグレスキューエル：オープンソースのリレーショナルデータベース。Supabaseの中核）
> - **JWT**（ジョット／ジェイダブリュティー：JSON Web Token の略。ログイン状態を表すデジタル証明書のような文字列）
> - **Auth**（オース：Authentication = 認証。ログイン処理）
> - **Storage**（ストレージ：画像やファイルを保管する場所）
> - **リアルタイムサブスクリプション**（Realtime Subscription：DBの変更を即座に画面へ反映する仕組み。WebSocketで通知を受け取る）

この章のゴールと地図

この章を読み終えると、次の5つができるようになります。

1. テーブル・行・列・主キーの関係を、表計算ソフトのシートにたとえて説明できる
2. SQL の4つの命令（SELECT / INSERT / UPDATE / DELETE）で、表がどう変わるかを予想できる

3. Supabase にプロジェクトと `books` テーブルを作り、**GRANT（テーブルの公開）と RLS（行ごとの許可）** を正しく設定できる
4. 公開してよいキーと、絶対に隠すべきキーを見分けて `.env.local` に書ける
5. `supabase-js` で CRUD を実行し、戻ってきた `{ data, error }` を読んで「成功・失敗・0件」を判断できる

章全体は、下の地図の①〜⑦の順に進みます。迷子になったら、この図に戻って「今どこにいるか」を確かめてください。



図5-1: 第5章の地図。①〜⑦の順に進み、最後に次章で画面とつなぐ

> **ヒント:** この章は長いので、1日で全部読む必要はありません。おすすめの区切りは「①〜③（テーブル作成まで）」「④〜⑤（守りの設定とキー）」「⑥〜⑦（コードで操作）」の3回です。

目次

0. 前提知識: データベースとSQLの超基礎
1. Supabase とは
2. アカウント作成とプロジェクトセットアップ
3. データベース設計の基礎
4. 書籍管理アプリのデータベース設計
5. Row Level Security (RLS)
6. Supabase Client のセットアップ
7. Supabase の基本操作（CRUD）
8. テストデータの投入
9. トラブルシューティング
10. 発展: アプリでは使っていない重要なSupabase機能
11. 練習問題（解答・解説つき）

0. 前提知識: データベースとSQLの超基礎

Supabase の中身は **PostgreSQL（ポストグレスキューエル：30年以上の歴史を持つオープンソースのリレーショナルデータベース管理システム）** という有名なデータベースです。本格的に使い始める前に、データベース・テーブル・SQL のいろはを押さえておきましょう。

0.1 データベースとは何か

「データを永続的に・大量に・整理して保管しておく仕組み」がデータベース（DB）です。Excelで言えば「複数のシートを持つ巨大な台帳」のイメージです。本書で使う **PostgreSQL** は、最も広く使われている**リレーショナルデータベース（RDB：Relational Database。表の形でデータを管理し、表同士を関連付けられるDB）** の一つです。

0.2 テーブル・カラム・レコード

RDB では、データを **テーブル（表）** に保存します。

📊 **books テーブル**

id	title	author	status
1	リーダブルコード	Boswell	completed
2	プロを目指す人〜	鈴木僚太	reading
3	達人プログラマー	Thomas	want_to_read

> 横の見出し（ `id` ・ `title` ・ `author` ・ `status` ）が **カラム（列）**、データ1件ぶんの各行が **レコード（行）** です。

用語	意味	Excelで言うと
テーブル	データの表そのもの	シート1枚
カラム（列）	縦の項目（id, title, author...）	A列、B列、C列...
レコード（行）	横の1件分のデータ	1行目、2行目...
主キー（PK）	レコードを一意に区別する値（普通は `id` ）	通し番号

表計算ソフト（Excel や Google スプレッドシート）を開いた画面を思い浮かべると、テーブルの各部分の名前がすっきり整理できます。

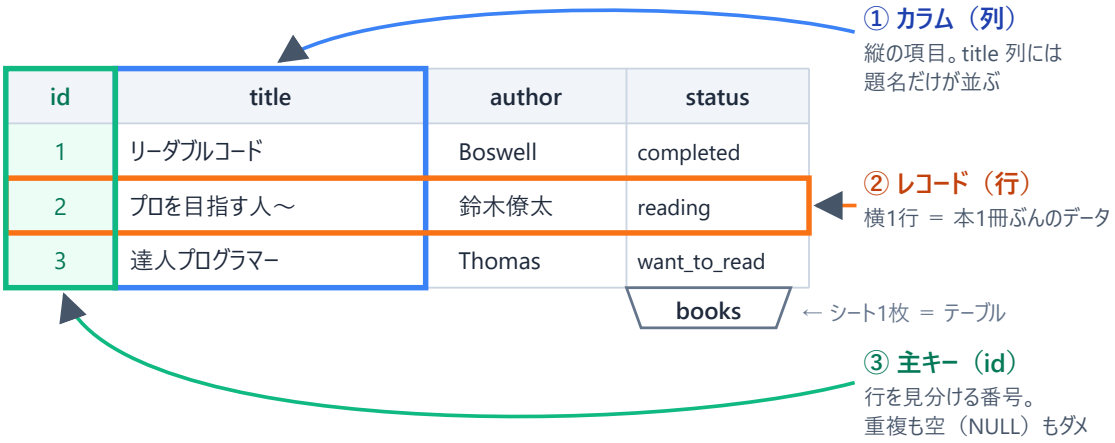


図5-2: テーブルは「表計算ソフトのシート1枚」。青い枠が列、橙の枠が行、緑の枠が主キー

たとえ話とのちがいも1つだけ押さえておきましょう。表計算ソフトでは同じ列に数字と文字を混ぜて書けますが、データベースでは**列ごとに入れてよい値の種類（データ型）が決まっていて**、違う種類の値は保存を断られます。この「断ってくれる」性質が、データを長く正しく保つための強みです（3.5 節と 4.3 節で実際に体験します）。

0.3 SQL の超ミニマム入門

データベースを操作する言語が **SQL（エスキューエル：Structured Query Language の略。データベース操作の世界標準言語）** です。よく使う4つだけ先に覚えましょう。

SELECT — データを取り出す

> **▼ このコードがやること（先に日本語で）**：テーブルに保存済みのデータを**取り出して表示する**命令です。SQL では「SELECT（取り出す列）FROM（どのテーブルから）」の順で書きます。Excel で言えば「この表のこの列だけ見せて」とお願いするイメージです。

```
-- SELECT : 「テーブルからデータを取り出す」SQL文の開始キーワード
-- title, author : 取り出したいカラム名をカンマ区切りで指定（必要な列だけ取れる）
-- FROM books    : どのテーブルから取るかを指定（ここでは books テーブル）
-- ;             : SQL文の終わりを表す記号（必ず必要）
SELECT title, author FROM books;
```

▼ 実行結果:

```
title             | author
------------------|----------
リーダブルコード    | Boswell
プロを目指す人〜    | 鈴木僚太
達人プログラマー    | Thomas
```

条件付きで取り出すこともできます。

> **▼ このコードがやること（先に日本語で）**：全部ではなく「**条件に合う行だけ**」を取り出します。`WHERE` のうしろに「どんな行が欲しいか」の条件を書くのがポイントです。ここでは「status が reading の本だけ見せて」とお願いしています。

```
-- SELECT *           : * は「全カラム」を意味するワイルドカード
-- FROM books         : booksテーブルから取り出す
-- WHERE status='reading' :  「statusカラムが'reading'と等しい行」だけに絞り込む条件
-- 文字列リテラルはシングルクォート ' で囲む（SQLのルール）
SELECT * FROM books WHERE status = 'reading';
```

▼ 実行結果（statusが'reading'の行だけ）：

```
id | title              | author       | status
---|--------------------|--------------|--------
 2 | プロを目指す人〜    | 鈴木僚太      | reading
```

INSERT — データを新しく追加する

> **▼ このコードがやること（先に日本語で）**：テーブルに**新しい行（レコード）を1件追加する**命令です。「どの列に」「どんな値を」入れるかを、`(列名...)` と `VALUES (値...)` で**同じ順番**にそろえて書くのがコツです。表に新しい1行を書き足すイメージです。

```
-- INSERT INTO books     : booksテーブルに新しい行を追加する宣言
-- (title, author, status): どのカラムに値を入れるか、カラム名を列挙
-- VALUES (..., ..., ...): 上で並べたカラムと同じ順番で値を指定
-- 'SQL入門'             : title カラムに入れる文字列リテラル
-- 'ミック'              : author カラムに入れる文字列
-- 'want_to_read'              : status カラムに入れる文字列
INSERT INTO books (title, author, status) VALUES ('SQL入門', 'ミック', 'want_to_read');
```

▼ 実行結果: `INSERT 0 1`（1件追加された）

実行後にテーブルを SELECT すると、新しい行が増えています。

UPDATE — データを書き換える

> **▼ このコードがやること（先に日本語で）**：すでにある行の値を**書き換える（更新する）**命令です。`SET` で「どの列をどの値に変えるか」を、`WHERE` で「どの行を対象にするか」を指定します。**`WHERE` を忘れると全行が書き換わってしまう**ので、必ず対象を絞り込むのが最重要ポイントです。

```sql
-- UPDATE books      : books テーブルの既存行を書き換える宣言
-- SET status='completed': どのカラムをどの値に変えるかを指定（=は代入）
-- WHERE id = 2      : 「id が 2 の行だけ」更新対象にする絞り込み条件
--                     WHERE を忘れると全行が書き換わるので超危険
UPDATE books SET status = 'completed' WHERE id = 2;
```

▼ 実行結果: `UPDATE 1`（1件更新された）。id=2 のレコードの status が `'reading'` から `'completed'` に変わります。

> **`WHERE` を必ず付ける!**: `WHERE` を忘れると**全レコードが書き換わってしまいます**。常に「どの行を対象にするか」を明示しましょう。

DELETE — データを消す

> **▼ このコードがやること（先に日本語で）**：テーブルから**行を削除する**命令です。`WHERE` で「どの行を消すか」を必ず指定します。**`WHERE` を忘れると全件削除（しかも復元不可）**になるため、削除対象の絞り込みが命綱だと覚えてください。

```sql
-- DELETE FROM books : books テーブルから行を削除する宣言
-- WHERE id = 3      : 「id が 3 の行だけ」削除する条件
--                     WHEREを忘れると全件削除で復元不可なので絶対に注意
DELETE FROM books WHERE id = 3;
```

▼ 実行結果: `DELETE 1`（1件削除された）。

> **これも `WHERE` 必須!**: 忘れると全件削除です。バックアップ無しで実行すると地獄を見ます。

通しで試す: 4つの命令で表がどう変わるか

ここまでの4つの命令を、練習用の小さな表で1回ずつ実行して、表がどう変わるかを確かめましょう。第2節でプロジェクトを作ったあと、4.3 節で開き方を説明する **SQL Editor** に貼り付けて「Run」を押せば、同じことを試せます（今は読むだけでも大丈夫です）。

▼ **このコードがやること（先に日本語で）**：練習用の表 `practice_books` を作って本を3冊入れ、「1冊追加」「1冊の状態を書き換え」「1冊削除」を順番に行ってから、最後に表の中身を全部表示します。本番の `books` テーブルとは別の名前なので、何度実行しても本番のデータには影響しません。

```sql
-- 同じ名前の練習用の表があれば、いったん消す（何度でもやり直せるようにするため）
DROP TABLE IF EXISTS practice_books;

-- 練習用の表を作る（id・題名・著者・状態の4列）
CREATE TABLE practice_books (
  id integer PRIMARY KEY,
  title text NOT NULL,
  author text NOT NULL,
  status text NOT NULL
);

-- 最初の3冊をまとめて入れる（( ) をカンマで区切ると複数行になる）
INSERT INTO practice_books (id, title, author, status) VALUES
  (1, 'リーダブルコード', 'Boswell', 'completed'),
  (2, 'プロを目指す人〜', '鈴木僚太', 'reading'),
  (3, '達人プログラマー', 'Thomas', 'want_to_read');

-- C（作成）：4冊目を追加する
INSERT INTO practice_books (id, title, author, status) VALUES (4, 'SQL入門', 'ミック',
'want_to_read');

-- U（更新）：id が 2 の本だけを「読み終えた」にする
UPDATE practice_books SET status = 'completed' WHERE id = 2;

-- D（削除）：id が 3 の本だけを消す
DELETE FROM practice_books WHERE id = 3;

-- R（読み取り）：最後に全部を id の小さい順に表示する
SELECT * FROM practice_books ORDER BY id;
```

▼ **実行結果**（PostgreSQL 17 の psql で1文ずつ実行した出力。SQL Editor では最後の `SELECT` の表だけが結果欄に表示されます）

```
DROP TABLE
CREATE TABLE
INSERT 0 3
INSERT 0 1
UPDATE 1
DELETE 1
 id |      title       |  author  |    status
----+------------------+----------+--------------
  1 | リーダブルコード | Boswell  | completed
  2 | プロを目指す人〜 | 鈴木僚太 | completed
  4 | SQL入門          | ミック   | want_to_read
(3 rows)
```



図5-3: 同じ3行の表に4つの命令を実行したときの変化（色の付いた行が変わった所）

▼ コードを1つずつ分解して解説

解説1: DROP TABLE IF EXISTS で「やり直せる」ようにする

```
DROP TABLE IF EXISTS practice_books;
```

- `DROP TABLE` は表を丸ごと消す命令です。`IF EXISTS` を付けると「あれば消す、なければ何もしない」になり、初回でもエラーになりません。
- 練習用のスクリプトの先頭にこれを書いておくと、何度実行しても同じ状態から始められます。**本番の `books` には絶対に使わない**でください（中身ごと消えます）。

> **用語: DROP** = 表などの「入れ物」そのものを消す命令。DELETE は「中身の行」を消す命令で、表は残ります。

解説2: 結果のメッセージは「何行に効いたか」

```
INSERT 0 3
UPDATE 1
DELETE 1
```

- `INSERT 0 3` の最後の数字は「追加した行数」です（真ん中の `0` は歴史的な理由で残っている値なので、気にしなくて大丈夫です）。
- `UPDATE 1`・`DELETE 1` は「1行を書き換えた」「1行を消した」という意味です。**ここが予想と違う数なら、WHERE の条件を疑う**のが鉄則です（次のつまずきポイントを参照）。

解説3: ORDER BY で並び順を決める

```
SELECT * FROM practice_books ORDER BY id;
```

- データベースは、`ORDER BY` を書かない限り**行の並び順を約束しません**。追加した順に出てくるとは限らないので、順番が大事なときは必ず `ORDER BY 列名` を付けます。
- 小さい順（昇順）が基本で、大きい順にしたいときは `ORDER BY id DESC` と書きます。

> **ヒント:** 試し終わったら `DROP TABLE practice_books;` で練習用の表を片付けておきましょう。

> **つまずきポイント: UPDATE / DELETE で WHERE を忘れて、全行が変わってしまう**
>
> `WHERE` は「どの行に効かせるか」の指定です。書き忘れても**エラーにはならず**、表のすべての行が対象になります。上の練習の直後の表（id が 1・2・4 の3行）で、id 4 の本だけを「読書中（reading）」にしたい場面を比べると、次のようになります。

```
-- ❌ NG: WHERE がないので、全部の行の status が 'reading' になる
UPDATE practice_books SET status = 'reading';
```

```
UPDATE 3
```

```
-- ✅ OK: まず同じ条件で SELECT して、対象が狙った1行だけか確かめる
SELECT id, title, status FROM practice_books WHERE id = 4;

-- 確かめた条件をそのまま付けて UPDATE する
UPDATE practice_books SET status = 'reading' WHERE id = 4;
```

```
 id |  title  |    status
----+---------+--------------
  4 | SQL入門 | want_to_read
(1 row)

UPDATE 1
```

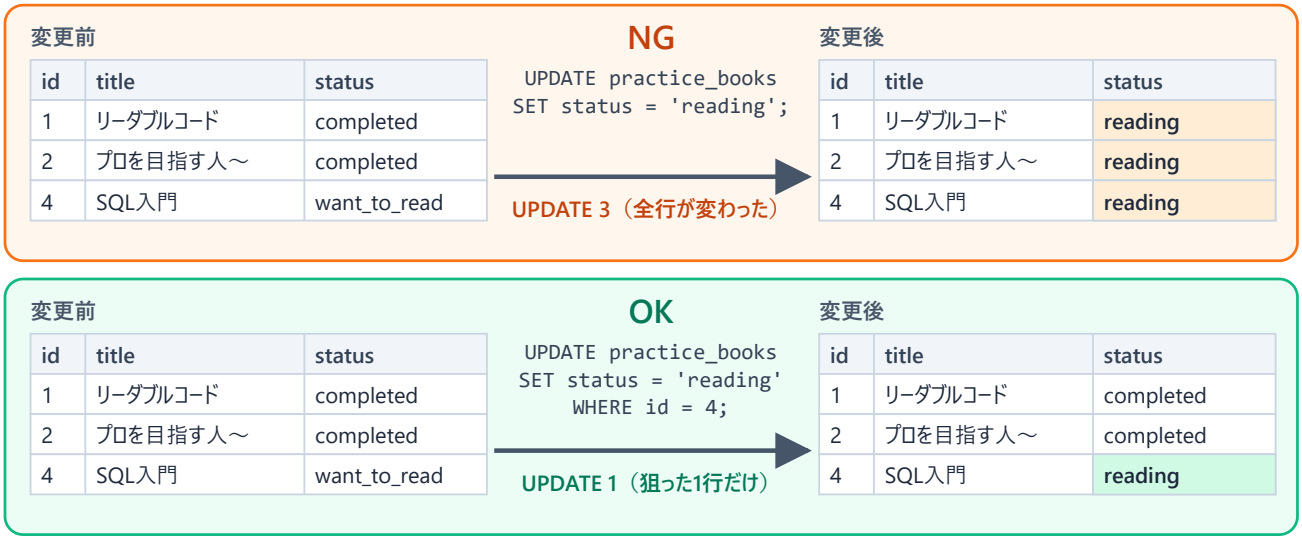


図5-4: WHERE を忘れた UPDATE（上）と、正しく絞り込んだ UPDATE（下）

ヒント:「SELECT で確かめてから、`SELECT ... FROM` の部分だけを `UPDATE ... SET` や `DELETE FROM` に書き換える」習慣をつけると、WHERE の書き忘れや条件ミスを実行前に見つけられます。結果のメッセージ（`UPDATE 3` など）の数字も必ず見ましょう。

つまずきポイント: 文字列をダブルクォート `"` で囲んでしまう

SQL では、**文字列はシングルクォート `'...'`** で囲みます。ダブルクォート `"..."` は「列名や表名」を表す記号なので、`"Boswell"` と書くと「Boswell という名前の列」を探しにいってエラーになります。JavaScript ではどちらのクォートでも文字列になるので、特に間違えやすいポイントです。

```
-- ❌ NG: "Boswell" が列名だと解釈される
SELECT * FROM practice_books WHERE author = "Boswell";
```

```
ERROR:  column "Boswell" does not exist
LINE 1: SELECT * FROM practice_books WHERE author = "Boswell";
                                                    ^
```

```
-- ✅ OK: 文字列はシングルクォートで囲む
SELECT * FROM practice_books WHERE author = 'Boswell';
```

```
 id |      title       | author  |  status
----+------------------+---------+-----------
  1 | リーダブルコード | Boswell | completed
(1 row)
```

- エラー文の `^` は「ここがおかしい」という位置の目印です。英語のエラーでも、`column "..." does not exist`（**列が存在しない**）と、`"` で囲んだ文字がそのまま列名として出てくる点に注目すれば原因に気づけます。

0.4 CRUDという言葉

上の4つの操作を頭文字でまとめて **CRUD（クラッド：Create / Read / Update / Delete の頭文字）** と呼びます。

| C | Create | INSERT | データを作る | | R | Read | SELECT | データを読む | | U | Update | UPDATE | データを更新する | | D | Delete | DELETE | データを削除する |

ほぼすべてのアプリは、CRUD のどれか（または組み合わせ）で動いています。本書の書籍管理アプリも、CRUD の練習が目的です。

0.5 本書での書き方: SQL 直接書かない

本書では、これらの SQL を**直接書く機会は少なめ**です。代わりに **Supabase のJavaScriptクライアント**を使い、TypeScriptのコードで操作します。

> **▼ このコードがやること（先に日本語で）**: さっきの SQL（ `SELECT * FROM books` ）を、**TypeScript のコードで同じことをする**書き方に置き換えた例です。`supabase.from("books").select("*")` が「booksテーブルから全部取り出す」にあたります。SQL を直接書かなくても、メソッドをつなげるだけでDB操作できる、という感覚をつかむのが目的です。

```
// SQLの「SELECT * FROM books」と同じ意味のコード
// await       : このAPI呼び出しは非同期（時間がかかる）ので結果が返るまで待つ
// supabase    : 別ファイルで作成した Supabase クライアントオブジェクト
// .from("books"): 操作対象のテーブル名を指定（戻り値はクエリビルダ）
// .select("*")  : 取得するカラムを指定。"*" は全カラム
// 分割代入 { data } : 戻り値オブジェクトから data プロパティだけを取り出す
const { data } = await supabase.from("books").select("*");
// console.log : ブラウザ開発者ツールのConsoleタブに値を出力する関数
console.log(data);
// ▼ data の中身（実行結果のイメージ）
// [
//   { id: 1, title: "リーダブルコード", author: "Boswell", status: "completed" },
//   { id: 2, title: "プロを目指す人〜", author: "鈴木僚太", status: "reading" },
//   ...
// ]
```

JavaScript のコードと SQL の対応関係は次のとおり。

操作	Supabase クライアント	SQL
読む	`.from("books").select("*")`	`SELECT * FROM books`
追加	`.from("books").insert({ title: "..." })`	`INSERT INTO books (...)`
更新	`.from("books").update({ status: "completed" }).eq("id", 2)`	`UPDATE books SET status='completed' WHERE id=2`
削除	`.from("books").delete().eq("id", 3)`	`DELETE FROM books WHERE id=3`

> **要点:** 「SQLは知らなくても書ける」けど「SQLを知っていると見通しがよくなる」。Supabaseには**SQLエディタ画面**があるので、迷ったらそこで生のSQLを書いて確認できます。

裏側で何が起きているか（通信の流れ）

`supabase.from("books").select(...)` の1行は、手元で SQL に変わるわけではありません。たとえるなら、**レストランの注文**です。あなた（客）は日本語で注文票を書き、店員（supabase-js）がそれを厨房向けの伝票（HTTP リクエスト）にしてインターネット越しに届け、受付（API Gateway）が会員証（キー）を確認し、伝票係（PostgREST）が料理人向けの指示（SQL）に書き直し、料理人（PostgreSQL）が作った料理（データ）が逆の順番で戻ってきます。

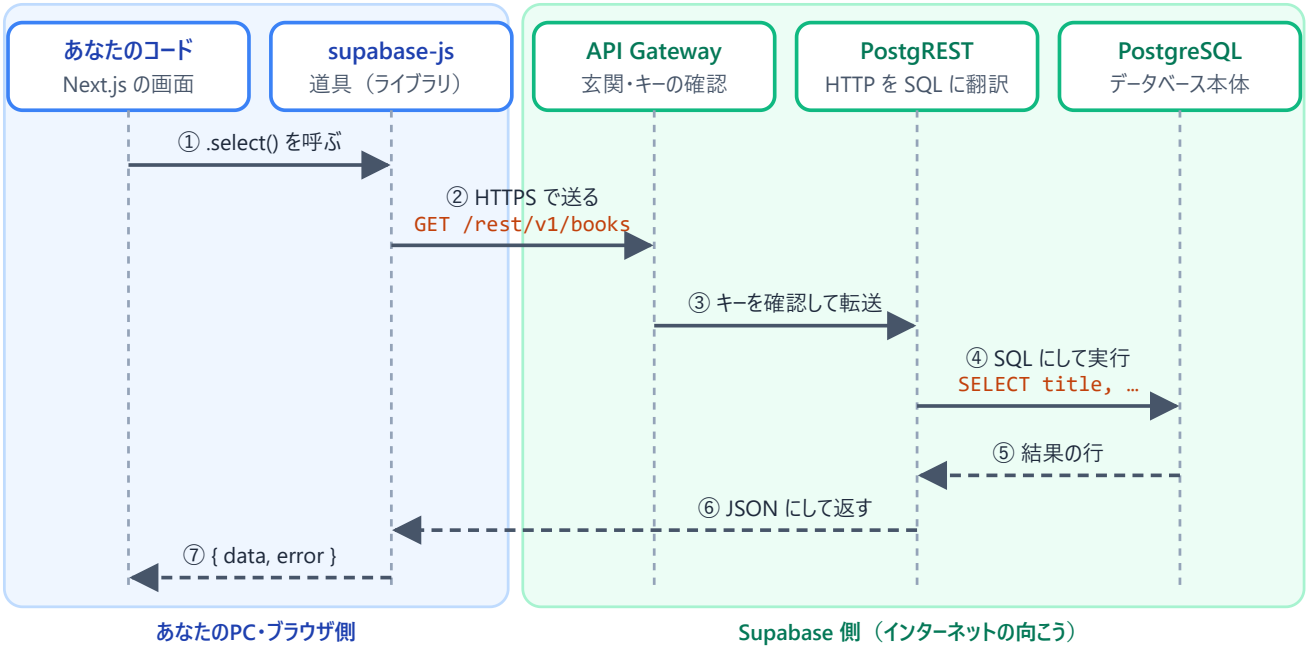


図5-5: supabase-js の1行が、HTTP と SQL に姿を変えて往復する流れ（実線が行き、点線が帰り）

1. ① あなたのコードが `supabase.from('books').select('title, status')` を呼びます。
2. ② supabase-js が、それを `GET https://xxxx.supabase.co/rest/v1/books?select=title%2Cstatus` のような **HTTP リクエスト**（Web の世界の「お願いの手紙」）に組み立てて送ります。ヘッダー（手紙の封筒部分）には `.env.local` に書いた**キー**が入っています。
3. ③ Supabase の玄関（API Gateway）がキーを確認し、正しければ中へ通します。
4. ④ PostgREST（ポストグレスト）が URL を読み、`SELECT title, status FROM books` に相当する SQL に**翻訳**して PostgreSQL に渡します。このとき「anon（ログインしていない人）として実行する」という**身分**（**ロール**）も一緒に伝えます。
5. ⑤ PostgreSQL は、その身分で**テーブルに入ってよいか**（**GRANT**）、**どの行を見せてよいか**（**RLS**）を確認してから結果の行を返します。
6. ⑥⑦ PostgREST が結果を **JSON**（JavaScript のオブジェクトとほぼ同じ形の文字列）に変えて送り返し、supabase-js がそれを `{ data, error }` という形にしてあなたのコードに渡します。

> **用語: HTTP リクエスト** = ブラウザやプログラムがサーバーに送る「お願い」。`GET`（ちょうだい）、`POST`（追加して）、`PATCH`（一部を書き換えて）、`DELETE`（消して）などの種類があり、それぞれ SELECT・INSERT・UPDATE・DELETE に対応します。

> **ヒント:** この流れを覚えておくと、エラーの意味が読みやすくなります。③で止まれば「キーが違う」、⑤の GRANT で止まれば `permission denied for table`、⑤の RLS で行が全部はじかれれば「エラーなしの空配列 `[]`」になります。詳しくは 4.3.1 節・第5節・第9節で実例とともに見ていきます。

1. Supabase とは

1.1 BaaS (Backend as a Service) の概念

Web アプリケーションを作るとき、通常は「フロントエンド（画面）」と「バックエンド（サーバー・データベース）」の両方を構築する必要があります。

従来の開発フロー:

1. データベースサーバーを用意する（PostgreSQL、MySQL など）
2. API サーバーを構築する（Express、Django、Rails など）
3. 認証の仕組みを実装する
4. ファイルストレージを用意する
5. これらすべてをデプロイ・運用する

これは初心者にとって非常に大きなハードルです。学ぶべき技術が多すぎて、アプリの本質的な部分に集中できません。

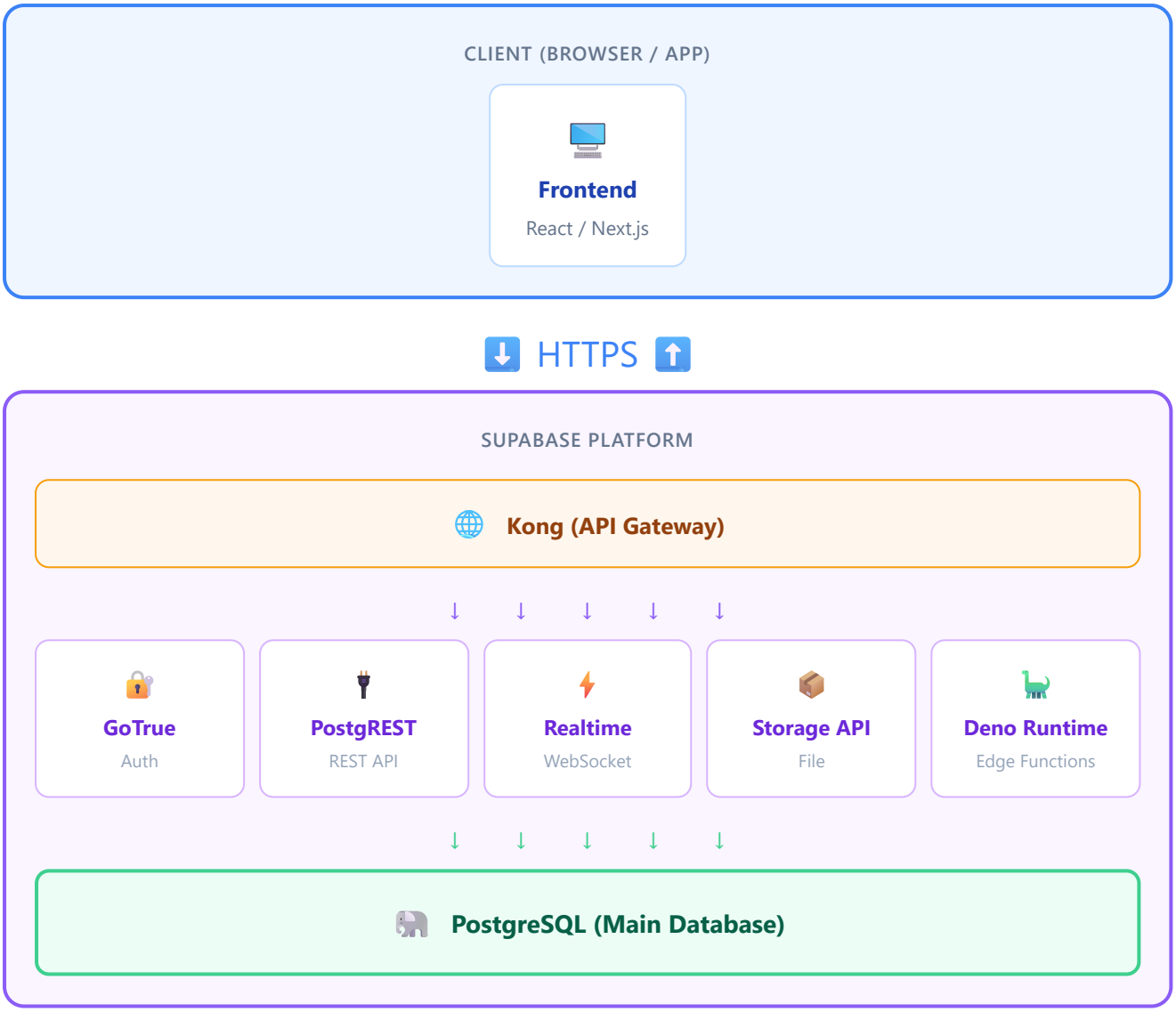
BaaS（Backend as a Service：バックエンドをサービスとして利用する形態。Firebase や Supabase など） は、このバックエンド部分をまるごとクラウドサービスとして提供してくれるものです。つまり、データベース・認証・ストレージ・API といったバックエンドの機能を、自分でサーバーを構築することなく利用できます。

```
従来の開発:
  フロントエンド → 自作 API サーバー → 自作データベース → 自作認証 → 自作ストレージ
  （すべて自分で構築・運用）

BaaS を使った開発:
  フロントエンド → BaaS（Supabase）
  （バックエンドはすべて Supabase が提供）
```

1.2 Supabase の全体アーキテクチャ

Supabase がどのような構成になっているか、全体像を見てみましょう。



ポイント: Supabase はオープンソースのツールを組み合わせて構築されています。PostgreSQL を中核として、PostgREST（自動 API 生成）、GoTrue（認証）、Realtime（リアルタイム通信）などが連携しています。

登場する各サービスの読み方と役割:

- **Kong**（コング：API Gateway。すべてのリクエストを最初に受け取る玄関口）
- **GoTrue**（ゴートゥルー：認証サーバー。ログイン・サインアップ・JWT発行を担当）
- **PostgREST**（ポストグレストレスト：DBのテーブル定義から自動的にRESTful APIを生成するソフトウェア）
- **Realtime**（リアルタイム：DBの変更を WebSocket でブラウザに即時配信するサーバー）
- **Storage**（ストレージ：画像やPDFなどファイルを保管するS3互換のサービス）
- **Edge Functions**（エッジファンクション：世界各地のサーバーで動く小さな関数。Deno で実行される）

1.3 Firebase との比較

BaaS の中で最も有名なのは Google の **Firebase** です。Supabase は「Firebase のオープンソース代替」を目指して作られました。両者を比較してみましょう。

項目	Supabase	Firebase
データベース	PostgreSQL（リレーショナル DB）	Firestore（NoSQL ドキュメント DB）
クエリ言語	SQL（標準的で汎用性が高い）	独自クエリ API
リアルタイム	PostgreSQL の変更検知	ネイティブリアルタイム
認証	GoTrue（メール、OAuth 等）	Firebase Auth（メール、OAuth 等）
ストレージ	S3 互換ストレージ	Cloud Storage
サーバーレス関数	Edge Functions（Deno）	Cloud Functions（Node.js）
料金	無料枠あり、従量課金	無料枠あり、従量課金
オープンソース	はい（セルフホスト可能）	いいえ
ベンダーロックイン	低い（標準 SQL で移行しやすい）	高い（独自仕様が多い）
学習の汎用性	SQL は他の仕事でも使える	Firestore の知識は限定的
複雑なクエリ	JOIN、サブクエリなど強力	限定的（非正規化が必要）
型安全性	CLI で TypeScript 型を自動生成可能	手動で型定義が必要

1.4 PostgreSQL ベースであることの利点

Supabase が PostgreSQL を採用していることには大きなメリットがあります。

1. SQL は業界標準のスキル

SQL（Structured Query Language）は、1970年代から使われ続けている データベース操作の標準言語です。Web開発だけでなく、データ分析、機械学習、業務システムなど、あらゆる分野で使われています。Supabase で SQL を学ぶことは、他のどんなプロジェクトでも役立つスキルを身につけることを意味します。

2. リレーショナルデータベースの信頼性

PostgreSQL は 30年以上の歴史を持つ、世界で最も信頼されているオープンソースデータベースの一つです。データの整合性を厳密に保証する仕組み（トランザクション、制約など）が備わっています。

3. 豊富な機能

- **JSON サポート**: NoSQL のような柔軟なデータも格納可能
- **全文検索**: テキスト検索機能を内蔵
- **地理空間データ**: PostGIS 拡張で位置情報も扱える
- **拡張機能**: 数百の拡張機能でほぼあらゆる用途に対応

4. 移行のしやすさ

標準 SQL を使っているため、将来 Supabase 以外のサービス（AWS RDS、Google Cloud SQL、自前サーバーなど）に移行する場合でも、データベースの知識とコードの大部分をそのまま活かせます。

1.5 Supabase が提供する機能一覧

機能	説明	本チュートリアルでの使用
Database	PostgreSQL データベース	書籍データの保存
Auth	ユーザー認証（メール、OAuth、マジックリンク等）	第7章で使用予定
Storage	ファイルアップロード・管理	書籍カバー画像の保存（応用編）
Realtime	データ変更のリアルタイム配信	応用編で使用予定
Edge Functions	サーバーレス関数（Deno ランタイム）	応用編で使用予定
Auto-generated API	テーブルから REST / GraphQL API を自動生成	本章でメインで使用
Dashboard	Web ベースの管理画面	テーブル作成・データ確認
CLI	コマンドラインツール	型の自動生成

---

2. アカウント作成とプロジェクトセットアップ

2.1 Supabase アカウントの作成

ステップ 1: Supabase 公式サイトにアクセス

ブラウザで以下の URL にアクセスします。

```
https://supabase.com
```

トップページが表示されたら、右上の「Start your project」ボタンをクリックします。

ステップ 2: GitHub アカウントでサインアップ

Supabase には、GitHub アカウントでサインアップする方法と、メールアドレス＋パスワードで登録する方法があります。本書では、入力が少なく、開発者なら持っておきたい GitHub アカウントを使う方法を紹介します（メールで登録した場合も、この先の手順は同じです）。

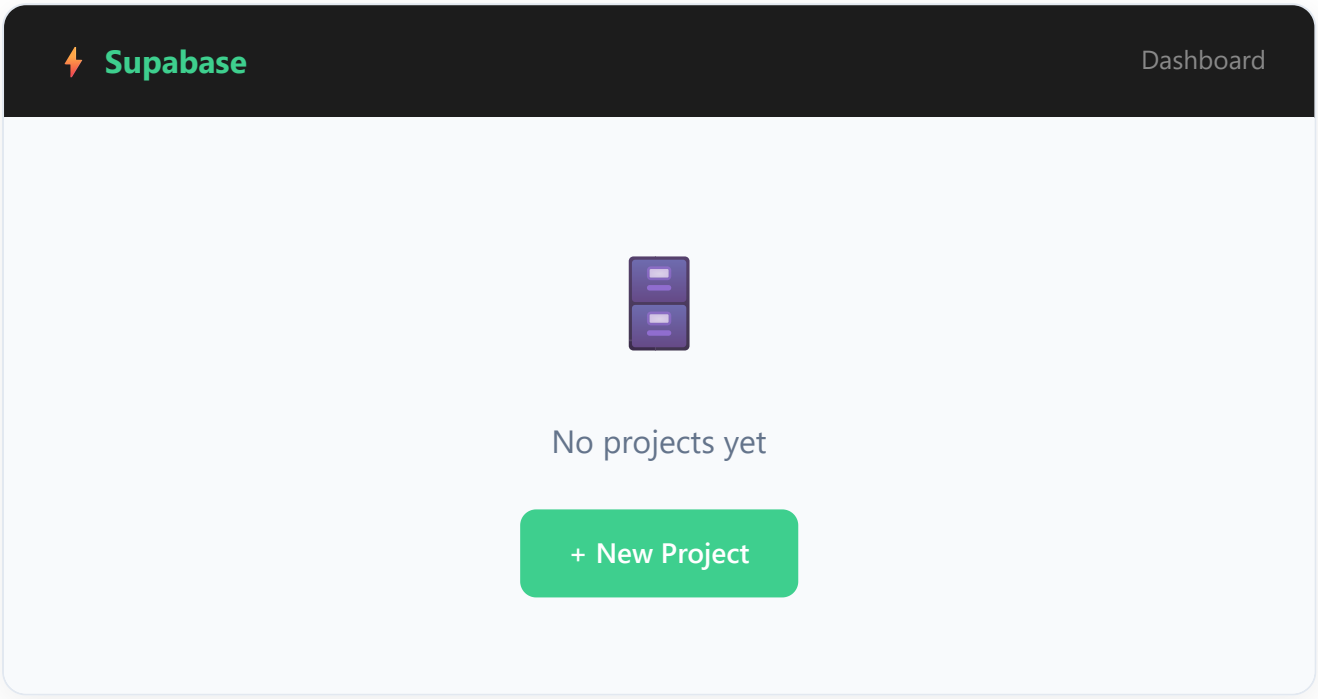
1. 「Continue with GitHub」（または「Sign in with GitHub」）ボタンをクリック
2. GitHub のログイン画面が表示されるので、自分の GitHub アカウントでログイン
3. Supabase にアクセスを許可するかの確認画面が出るので「Authorize supabase」をクリック

4. Supabase のダッシュボードにリダイレクトされれば成功

> **GitHub アカウントを持っていない場合:** まず https://github.com でアカウントを作成してください。GitHub は開発者にとって必須のサービスなので、この機会に作成しておきましょう。

ステップ 3: ダッシュボードの確認

ログイン後、Supabase のダッシュボード画面が表示されます。初回ログイン時はプロジェクトが一つもない状態です。



2.2 新規プロジェクトの作成

ステップ 1:「New Project」ボタンをクリック

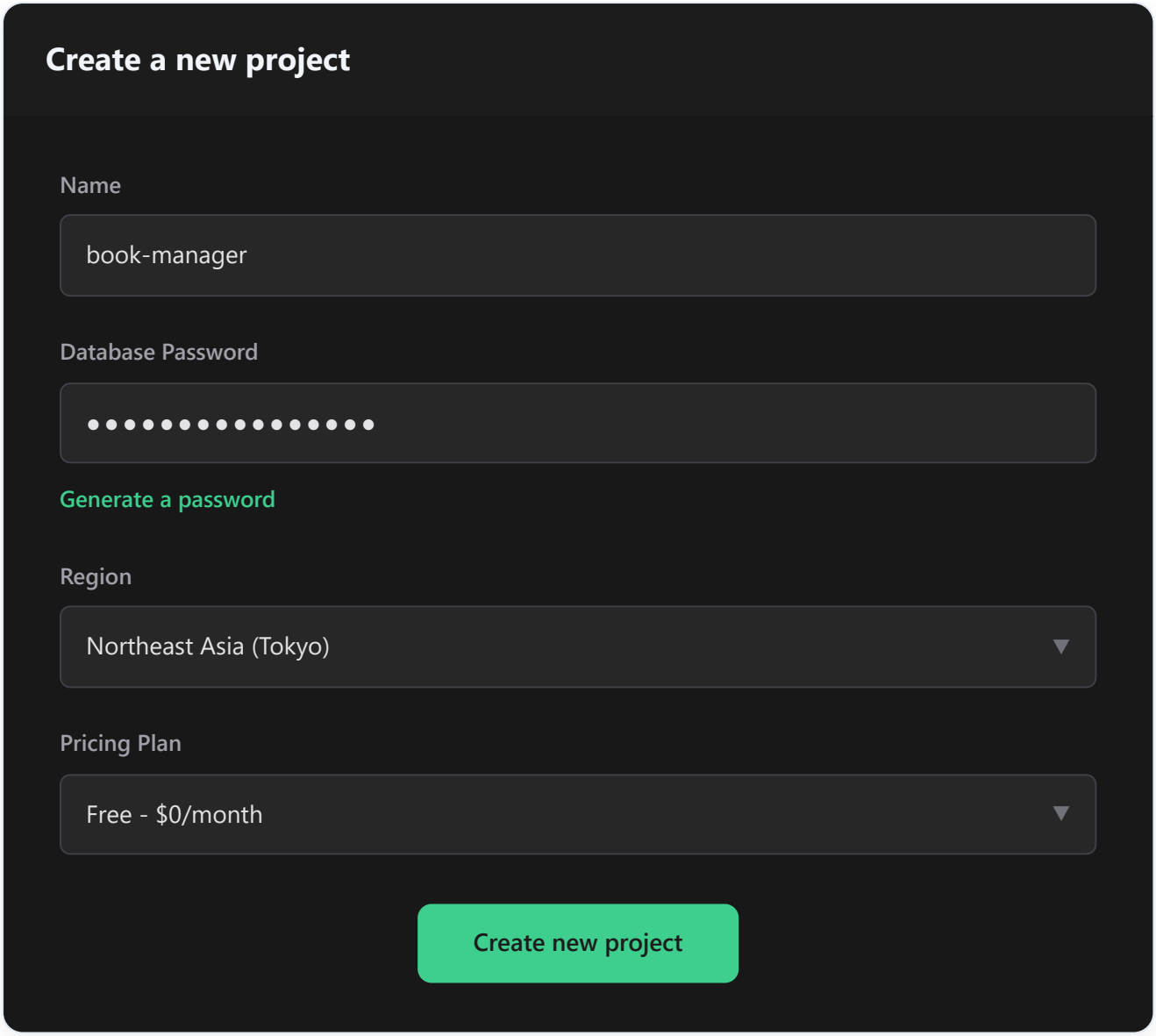
ダッシュボードの「New Project」ボタンをクリックします。

ステップ 2: Organization（組織）の選択

初回の場合は、自動的に個人用の Organization が作られています。そのまま選択してください。

ステップ 3: プロジェクト情報の入力

以下の情報を入力します。



各項目の詳細:

項目	入力値	説明
Name	`book-manager`	プロジェクト名。分かりやすい名前をつけましょう
Database Password	（自動生成推奨）	「Generate a password」をクリックして安全なパスワードを生成。**必ずどこかにメモしておくこと**
Region	`Northeast Asia (Tokyo)`	データベースが配置される地域。日本在住なら Tokyo を選択
Pricing Plan	`Free`	無料プラン。学習には十分な機能が利用可能

ステップ 4: プロジェクトの作成

「Create new project」ボタンをクリックします。プロジェクトのセットアップには 1〜2 分程度かかります。画面に進捗バーが表示されるので、完了するまで待ちます。

ステップ 5: プロジェクトダッシュボードの確認

セットアップが完了すると、プロジェクトのダッシュボードが表示されます。この章でよく使う場所は、左のメニューと右上のボタンの4か所だけです。



図5-6: プロジェクトのダッシュボード（画面イメージ。メニューの並びや表記は Supabase の更新で変わることがあります）

重要: API キーを控えておく

後ほどコードから Supabase に接続する際に、次の2つの値が必要です。いちばん簡単なのは、画面上部の「**Connect**」ボタンを押して表示される画面からコピーする方法です。設定画面から探す場合は「**Project Settings**」の中の「**Data API**」（Project URL）と「**API Keys**」（キー）にあります（以前のダッシュボードでは「Settings」>「API」という1つの画面にまとまっていました）。

- **Project URL**: `https://xxxxxxxx.supabase.co` の形式
- **公開用キー**: 新しい形式の **Publishable key**（`sb_publishable_...` で始まる短めの文字列）、または従来の **anon (public) key**（`eyJhbGciOiJIUzI1NiIs...` で始まる長い文字列。実体は **JWT**：JSON Web Token と呼ばれる署名付きの文字列）。どちらも「ブラウザに置いてよい公開用のキー」で、本書ではどちらを使っても同じように動きます

ANON Key と Service Role Key の違い（重要）

Supabase のプロジェクトには2種類のAPIキーが発行されます。混同すると重大な事故になります。

キー名	用途	RLSの効果	どこに置く
anon key（アノン：anonymous の略。匿名キー）／新形式では **Publishable key**	ブラウザから使う公開キー	RLS が有効。ポリシーに従う	フロントエンド、`.env.local` の `NEXT_PUBLIC_` プレフィックス付きでOK
service_role key（サービスロール：管理者キー）／新形式では **Secret key**	サーバー処理から使う特権キー	**RLSを完全にバイパス（無視）**	**絶対にフロントエンドに含めない。**サーバー側だけ

service_role key（Secret key）が漏えいすると、全データの読み取り・書き換え・削除を誰でもできるようになります。Gitにコミットしたり、ブラウザに渡したりするのは絶対に避けてください。

キーは「家の鍵」にたとえると分かりやすくなります。**公開用キー**は「お店の入口の鍵（営業時間中は誰でも入れる）」で、入った後に何ができるかは店内のルール（RLS）が決めます。**秘密キー**は「店長のマスターキー」で、金庫も倉庫も全部開きます。マスターキーをお客さんに配ってはいけないのと同じで、秘密キーはブラウザに渡してはいけません。

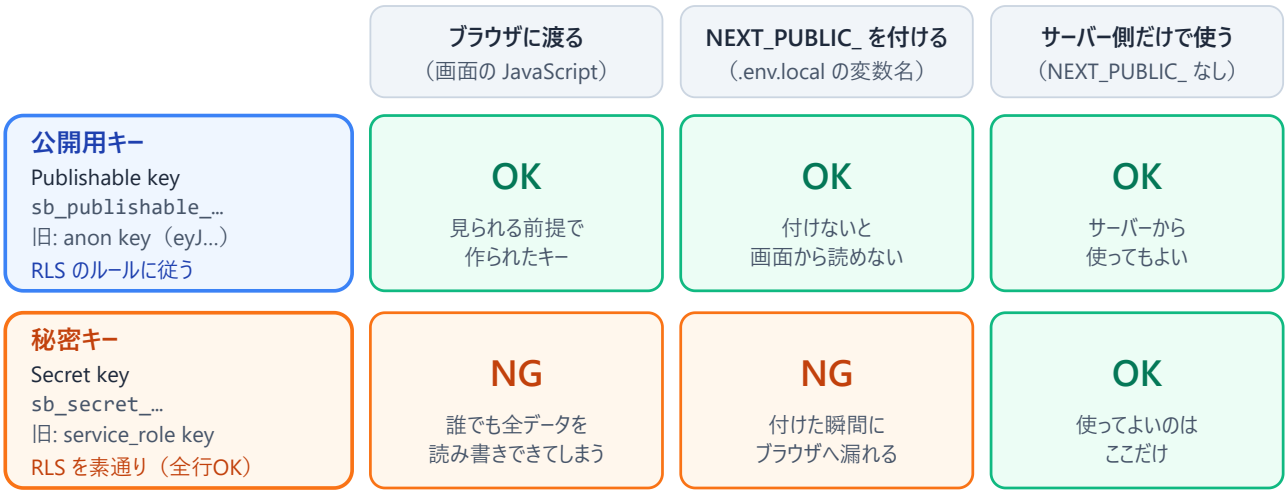


図5-7: 2種類のキーを「どこに置いてよいか」。どちらのキーも、.env.local ごと Git にコミットしないのは共通

注意: キーの形式が2種類ある理由 Supabase は2025年から、新しい形式の **Publishable key**（`sb_publishable_...`）と **Secret key**（`sb_secret_...`）を導入しました。従来の **anon key / service_role key**（どちらも `eyJ...` で始まる長い JWT）は「Legacy（旧形式）」として扱われ、公式ドキュメントでは2026年末までに廃止する予定と案内されています。役割は同じなので、本書では変数名を `NEXT_PUBLIC_SUPABASE_ANON_KEY` のままにし、**値には Publishable key を貼り付ける**ことをおすすめします（従来の anon key が表示されるプロジェクトでは、それを貼っても動きます）。

2.3 リージョンの選び方

リージョンとは、Supabase のサーバー（データベース）が物理的に配置されるデータセンターの場所です。

選び方の基本原則:

> ユーザー（自分やアプリの利用者）に最も近いリージョンを選ぶ

日本在住で、主に日本国内向けのアプリを作る場合は **Northeast Asia (Tokyo)** を選択するのがベストです。

なぜリージョンが重要なのか:

データベースとの通信には物理的な距離に応じた遅延（レイテンシ）が発生します。

リージョン	日本からのおおよその遅延
Tokyo（東京）	5〜15ms
Singapore（シンガポール）	60〜80ms
US West（米国西部）	100〜150ms
US East（米国東部）	150〜200ms
EU West（ヨーロッパ西部）	200〜300ms

> **注意:** リージョンはプロジェクト作成後に変更できません。間違えた場合は新しいプロジェクトを作り直す必要があります。

2.4 無料プランの制限

Supabase の無料プラン（Free Plan）には以下の制限があります。学習目的であればまったく問題ありません。

項目	無料プランの制限
プロジェクト数	同時に動かせるのは2つまで
データベース容量	500 MB（プロジェクトごと）
ストレージ容量	1 GB
転送量（Egress）	5 GB / 月（＋キャッシュ経由の転送 5 GB）
Edge Function 呼び出し	50万回 / 月
認証ユーザー数	月間アクティブユーザー（MAU）50,000人まで
一時停止	1週間アクセスがないと自動停止（再起動可）

> **注意:** 料金プランの数字は変わることがあります。最新の値は公式の料金ページ（supabase.com/pricing）で確認してください（上の表は2026年10月時点の内容です）。

> **自動停止について:** 無料プランでは、1週間以上ダッシュボードやAPIへのアクセスがないとプロジェクトが自動的に一時停止されます。停止されてもデータは消えません。ダッシュボードにアクセスして「Restore project」ボタンを押せば数分で復旧します。学習中は定期的にアクセスするようにしましょう。

---

3. データベース設計の基礎

3.1 リレーショナルデータベースとは

リレーショナルデータベース（RDB） とは、データを「テーブル（表）」の形式で管理するデータベースです。Excel のスプレッドシートをイメージすると分かりやすいでしょう。

例: 書籍情報のテーブル

id	title	author	rating
1	ノルウェイの森	村上春樹	5
2	人間失格	太宰治	4
3	1Q84	村上春樹	5

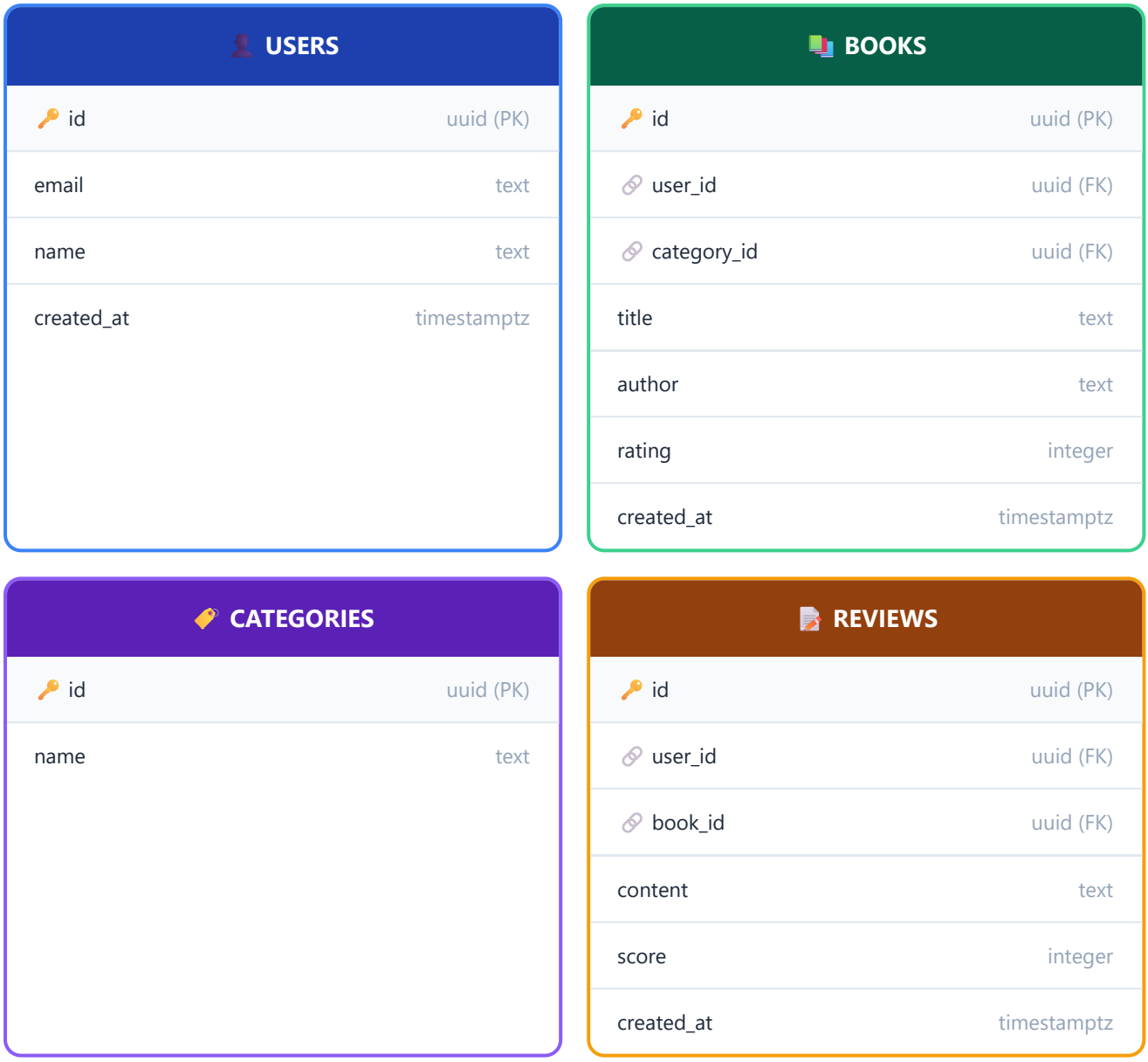
このテーブルの各部分には名前がついています:

- **テーブル（Table）**: データのまとまり全体（上の表そのもの）。「books テーブル」のように名前をつけます
- **行（Row / Record）**: テーブル内の1件のデータ。上の例では3行 = 3冊の書籍データ
- **列（Column / Field）**: データの各項目。上の例では id, title, author, rating の4列

3.2 リレーショナルデータベースの全体像

実際のアプリケーションでは、複数のテーブルが関連し合います。例えば「ユーザー」と「書籍」のように、テーブル間に関係（リレーション）があるのが特徴です。

Relationships: USERS ||--o{ BOOKS (owns) USERS ||--o{ REVIEWS (writes) BOOKS ||--o{ REVIEWS (has) BOOKS }o--|| CATEGORIES (belongs to)



> **注意:** 上の図は一般的な書籍管理システムの例です。本チュートリアルでは、まず **books テーブル1つ** から始め、後の章で認証やリレーションを追加していきます。

3.3 主キー（Primary Key）

主キー（PK: Primary Key） とは、テーブル内の各行を一意に識別するための列です。

なぜ主キーが必要か？

例えば、同じ「村上春樹」の「ノルウェイの森」が2冊登録された場合、どちらを指しているか区別できなくなります。主キーがあれば、`id = 1` と `id = 2` のように一意に特定できます。

主キーのルール:

1. テーブル内でユニーク（重複しない）こと
2. NULL（空）にはできないこと
3. 基本的に変更しないこと

UUID vs 連番（SERIAL）:

方式	例	メリット	デメリット
UUID（ユーユーアイディー：Universally Unique Identifier。世界中で重複しない128ビットの識別子）	`550e8400-e29b-41d4-a716-446655440000`	衝突の可能性がほぼゼロ。分散システムに適する	長い。人間が読みにくい
SERIAL（シリアル：自動採番される連番）	`1`, `2`, `3` ...	短い。人間が読みやすい	連番が推測可能。分散環境で衝突しうる

Supabase では **UUID** がデフォルトで推奨されています。本チュートリアルでも UUID を使用します。

3.4 外部キー（Foreign Key）

外部キー（FK: Foreign Key） とは、別のテーブルの主キーを参照する列です。テーブル間の関連付けに使います。

books テーブル（`user_id` が外部キー＝users を指す）

id (PK)	user_id (FK)
uuid-001	user-AAA
uuid-002	user-AAA
uuid-003	user-BBB

users テーブル（`id` が主キー）

id (PK)	name
user-AAA	田中太郎
user-BBB	鈴木花子

> **関連付け:** books の `user_id` が users の `id` を参照します（`books.user_id ⟶ users.id`）。この例では user-AAA（田中太郎）が2冊（uuid-001・uuid-002）、user-BBB（鈴木花子）が1冊（uuid-003）を登録した、という関係を表します。

上の例では、books テーブルの `user_id` が users テーブルの `id` を参照しています。これにより「どのユーザーがどの本を登録したか」が分かります。

> **本チュートリアルでは:** まだ認証機能を実装していないため、外部キーは後の章で追加します。今回は books テーブル単体で始めます。

3.5 データ型

PostgreSQL で使う主なデータ型を紹介します。

データ型	説明	例
`uuid`	一意識別子（128ビット）	`550e8400-e29b-41d4-a716-446655440000`
`text`	可変長テキスト（長さ制限なし）	`'ノルウェイの森'`
`varchar(n)`	可変長テキスト（最大 n 文字）	`'abc'`（最大n文字）
`integer`	整数（-2147483648 〜 2147483647）	`42`
`bigint`	大きい整数	`9223372036854775807`
`boolean`（bool）	真偽値	`true` / `false`
`date`	日付	`'2024-01-15'`
`timestamptz`	タイムスタンプ（タイムゾーン付き）	`'2024-01-15T10:30:00+09:00'`
`jsonb`	JSON データ（バイナリ形式）	`'{"key": "value"}'`
`numeric`	精密な小数	`3.14`

> **`text` vs `varchar(n)` について:** PostgreSQL では `text` と `varchar` の性能差はほとんどありません。長さの制限が本当に必要な場合以外は `text` を使うのがシンプルです。Supabase の公式ドキュメントでも `text` が推奨されています。

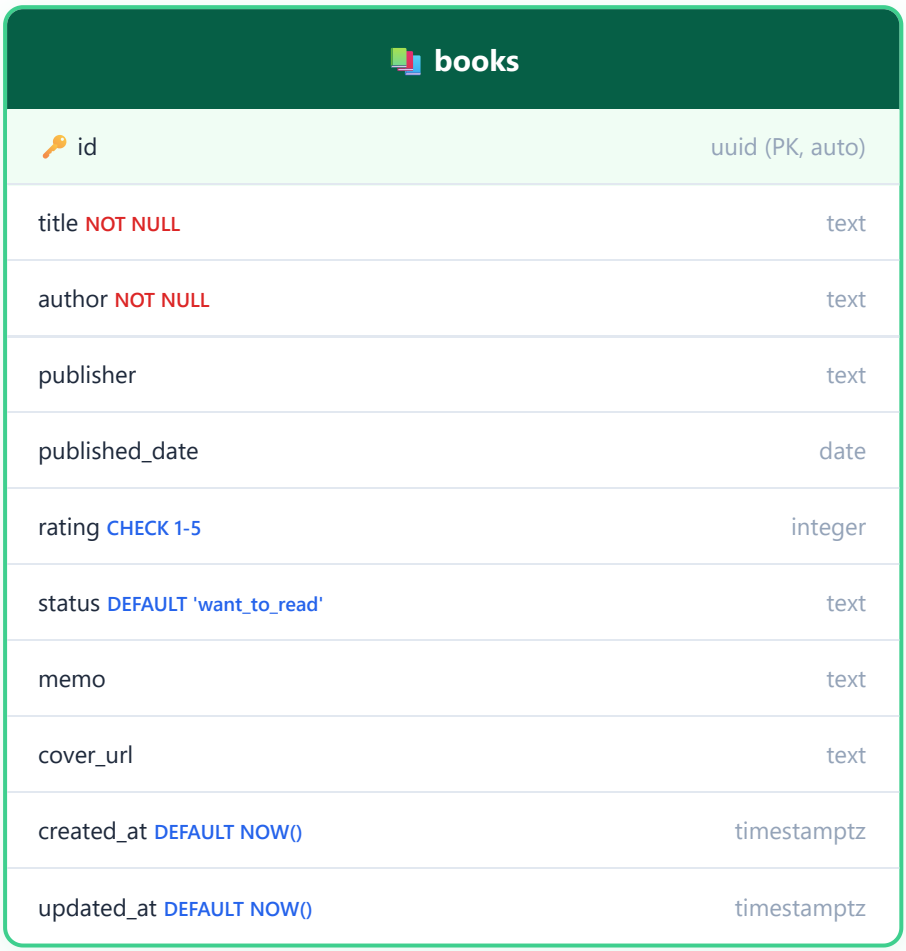
> **`timestamptz` と `timestamp` の違い:** `timestamptz` は "timestamp with time zone" の略で、タイムゾーン情報を保持する型です。世界中どこからINSERTしてもUTCに換算して保存され、SELECT時はクライアントのタイムゾーンに自動変換されます。`timestamp`（タイムゾーンなし）だと「東京の23時」「ロンドンの23時」が同じ値になって混乱するので、原則 `timestamptz` を使います。

---

4. 書籍管理アプリのデータベース設計

4.1 books テーブルの設計

書籍管理アプリで必要なデータを整理し、テーブルを設計しましょう。



4.2 各カラムの設計理由

各カラム（列）がなぜ必要で、なぜそのデータ型を選んだのか、一つずつ解説します。

`id` - uuid (PK, auto-generated)

```
-- id              : カラム名（このテーブル内での識別用、慣習的にidという名前を使う）
-- uuid            : データ型。128ビットのUUID（世界中で重複しないランダム文字列）を格納
-- DEFAULT ...     : INSERTで値が指定されなかったときに自動で入る値
-- gen_random_uuid(): Postgresが標準で持つ「新しいUUIDを1つ作る」関数
-- PRIMARY KEY     : このカラムを主キーにする宣言（NOT NULL かつ UNIQUE が自動付与される）
id uuid DEFAULT gen_random_uuid() PRIMARY KEY
```

- **役割:** 各書籍を一意に識別するための主キー

- **型の理由:** UUID を使うことで、グローバルに一意な ID を自動生成できる。将来的にユーザー認証を追加して複数ユーザーのデータを扱う際にも衝突しない
- `DEFAULT gen_random_uuid()`: INSERT 時に自動的に UUID が生成されるため、フロントエンドから ID を指定する必要がない
- `PRIMARY KEY`: この列が主キーであることを示す

`title` - text (NOT NULL)

```
-- title    : カラム名（書籍のタイトル）
-- text     : 可変長の文字列型。長さ制限なし（日本語もOK）
-- NOT NULL : 値が空（NULL）の行は許可しない、という制約
title text NOT NULL
```

- **役割:** 書籍のタイトル
- **型の理由:** タイトルの長さは予測できないため、可変長の `text` を使用
- `NOT NULL`: 書籍にタイトルがないのはありえないので、必須項目にする。NULL（空）でのINSERTを防ぐ

`author` - text (NOT NULL)

```
-- author   : カラム名（著者名）
-- text     : 文字列型
-- NOT NULL : 必須項目（空欄での登録を防ぐ）
author text NOT NULL
```

- **役割:** 著者名
- **型の理由:** 著者名も長さが予測できないため `text` を使用
- `NOT NULL`: 著者不明の書籍は「著者不明」と入力する想定。NULL は避ける
- **設計メモ:** 本格的なアプリでは著者を別テーブルに分離して多対多のリレーションにしますが、学習用アプリなのでシンプルにテキストで保持します

`publisher` - text

```
-- publisher : カラム名（出版社名）
-- text      : 文字列型
-- NOT NULL を付けないので NULL を許可（任意項目になる）
publisher text
```

- **役割:** 出版社名
- **型の理由:** 出版社名も長さが予測できないため `text`
- **NULL 許可:** 出版社が分からない、または入力しない場合もあるため、NULL を許可（任意項目）

`published_date` - date

```
-- published_date : カラム名（出版日）
-- date           : 「年月日」だけを格納する型。時分秒は持たない
-- NOT NULLなし   : NULL許可（出版日不明でも登録できる）
published_date date
```

- **役割:** 出版日
- **型の理由:** 年月日のみで十分なので `date` 型を使用（時刻は不要）
- **NULL 許可:** 出版日が不明な場合もあるため、NULL を許可
- **注意:** `timestamp` ではなく `date` を使うのは、出版日に時分秒の情報は必要ないため

`rating` - integer (1-5)

```
-- rating  : カラム名（評価値）
-- integer : 整数型（-2147483648〜2147483647までの範囲）
-- CHECK ( ... ) : 値の妥当性を検査する制約。条件を満たさない値はINSERT/UPDATE時に拒否
-- rating >= 1 AND rating <= 5 : 1以上かつ5以下のみ許可、という条件式
rating integer CHECK (rating >= 1 AND rating <= 5)
```

- **役割:** 自分の評価（星1〜5つ）
- **型の理由:** 1〜5 の整数値なので `integer` が最適
- `CHECK` **制約:** データベースレベルで 1〜5 の範囲を強制する。これにより、フロントエンドのバグで不正な値が入ることを防げる
- **NULL 許可:** まだ読んでいない本は評価できないため、NULL を許可

`status` - text ('reading' | 'completed' | 'want_to_read')

```
-- status : カラム名（読書状態）
-- text   : 文字列型
-- DEFAULT 'want_to_read' : INSERT時に値が指定されなければ 'want_to_read' を入れる
-- CHECK (status IN (...)) : 指定された3つの値のいずれかでないと弾く制約
-- IN は「リストの中に含まれるか」を判定するSQL演算子
status text DEFAULT 'want_to_read' CHECK (status IN ('reading', 'completed',
'want_to_read'))
```

- **役割:** 読書状態の管理
- **型の理由:** 決まった値のいずれかなので `text` + `CHECK` 制約で管理
- `DEFAULT 'want_to_read'` : 新しく登録した本はデフォルトで「読みたい」状態にする
- `CHECK` **制約:** 許可された3つの値以外は登録できないようにする
- **各値の意味:**

- `reading`: 現在読んでいる
- `completed`: 読了済み
- `want_to_read`: 読みたい

> **なぜ `enum` 型ではなく `text` + `CHECK` か？** PostgreSQL には `enum` 型がありますが、後から値を削除するのが難しいという問題があります。`text` + `CHECK` であれば、将来的に `CHECK` 制約を変更するだけで値を追加・変更・削除できます。

`memo` - text

```
-- memo : カラム名（読書メモ）
-- text : 文字列型（長文OK）
-- NULL許可（メモは任意）
memo text
```

- **役割:** 読書メモ・感想
- **型の理由:** 長文を格納できる `text` を使用
- **NULL 許可:** メモは任意項目

`cover_url` - text

```
-- cover_url : カラム名（表紙画像のURL）
-- text      : URL文字列を格納
-- NULL許可（画像未設定でもOK）
cover_url text
```

- **役割:** 書籍の表紙画像の URL
- **型の理由:** URL は文字列なので `text`
- **NULL 許可:** 画像がない場合もある
- **設計メモ:** 将来的に Supabase Storage を使って画像をアップロードする機能を追加する際、ここに Storage の URL を格納する

`created_at` - timestamptz

```
-- created_at  : カラム名（作成日時。慣習的にこの名前を使う）
-- timestamptz : タイムゾーン付きの日時型（worldwide で正しい時刻を扱える）
-- DEFAULT NOW() : INSERT時に値がなければ「現在時刻」が自動で入る
-- NOW() : 現在の日時を返すPostgres組み込み関数
created_at timestamptz DEFAULT NOW()
```

- **役割:** データが作成された日時
- **型の理由:** タイムゾーン情報を含む `timestamptz`（timezone 付き timestamp）を使用。世界中どこからアクセスしても正しい時刻が記録される
- `DEFAULT NOW()`: INSERT 時に自動的に現在時刻が設定される

`updated_at` - timestamptz

```
-- updated_at  : カラム名（更新日時）
-- timestamptz : タイムゾーン付きの日時型
-- DEFAULT NOW() : 作成時の値として現在時刻を入れる
-- UPDATE時に自動で書き換えるためには「トリガー」を別途定義する必要あり（後述）
updated_at timestamptz DEFAULT NOW()
```

- **役割:** データが最後に更新された日時
- **型の理由:** `created_at` と同じく `timestamptz`
- `DEFAULT NOW()`: INSERT 時に自動的に現在時刻が設定される
- **注意:** UPDATE 時に自動更新するにはトリガーが必要（後述）

4.3 SQL でのテーブル作成

以下の SQL で books テーブルを作成します。Supabase の SQL Editor で実行してください。

ステップ 1: SQL Editor を開く

Supabase ダッシュボードの左メニューから「SQL Editor」をクリックします。

> **Supabase Studio（管理画面）とSQLの対応:** Supabase の管理画面は内部的には毎回SQLを実行しています。「Table Editor で列を追加する」ボタンを押すと、裏で `ALTER TABLE ... ADD COLUMN ...` が走るだけ。GUIに慣れたら、SQL Editor で直接書いた方が早いと感じるはずです。

ステップ 2: 新しいクエリを作成

「New query」ボタンをクリックし、以下の SQL をコピー＆ペーストします。SQL Editor の画面は、おおよそ次のような4つの部分でできています。

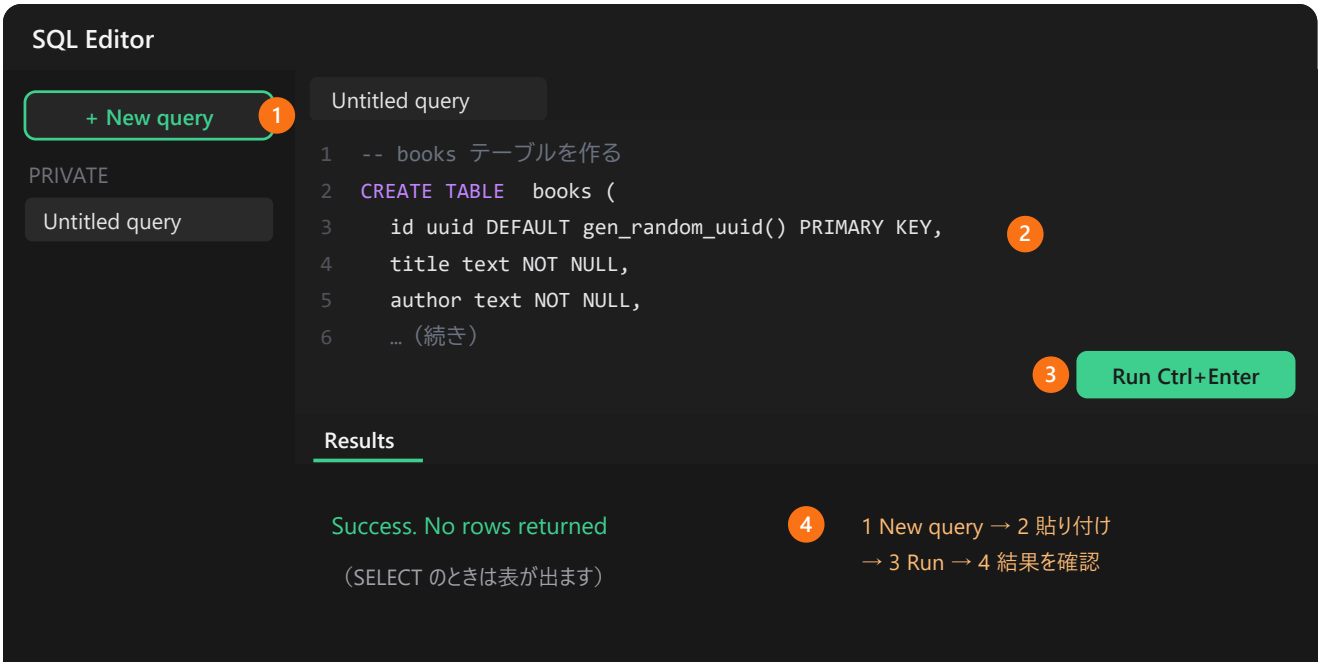


図5-8: SQL Editor の画面イメージ。①新しいクエリ → ②SQL を貼る → ③Run → ④結果欄を見る

ヒント: SQL Editor では、複数の SQL 文をまとめて貼って一度に Run できます。そのとき結果欄に表示されるのは**最後の文の結果だけ**です。途中の文が失敗すると、そこで止まって赤い `ERROR` が表示されます。

▼ このコードがやること（先に日本語で）: このアプリの中心となる **books（書籍）テーブルを新しく作る**SQLです。`CREATE TABLE` で「どんな列（カラム）を、どんなデータ型・制約で持つか」を一気に定義します。あわせて、行が更新されるたびに `updated_at`（更新日時）を自動で書き換える**トリガー**も用意します。長く見えますが、ほとんどが各列の説明コメントなので、まずはコピペで実行してOKです。

```sql
-- ============================================================================
-- books テーブルの作成 – 書籍管理アプリのメインテーブル
-- ----------------------------------------------------------------------------
-- SQL の構文: CREATE TABLE テーブル名 ( カラム定義1, カラム定義2, ... );
-- カラム定義: 「カラム名 データ型 制約」の3つを並べて書く。
-- 行末のセミコロン ; が SQL文の終わり。
-- -- (ハイフン2つ) で始まる行はコメント（実行時に無視される）。
-- ============================================================================

-- CREATE TABLE : 新しいテーブルを作るSQL文の開始キーワード
-- books        : 作成するテーブル名（複数形にするのが慣習）
-- (...)        : この中にカラム定義を並べる
CREATE TABLE books (

  -- (1) 主キー(PRIMARY KEY): レコードを一意に識別する列。
  --     id                   : このテーブル内でのカラム名
  --     uuid                 : 文字列の一種で、世界中で重複しないIDを表す型（128ビット）
  --     gen_random_uuid()    : Postgres組み込み関数。新しいUUIDを1つ生成する
  --     DEFAULT ...          : INSERT時にこの値が省略されたら自動で入る初期値
  --     PRIMARY KEY          : 主キー宣言。NULL不可・重複不可になる
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,

  -- (2) 書籍情報（必須）
  --     title    : タイトル列。カラム名
  --     text     : 可変長の文字列（長さ制限なし）。MySQL の VARCHAR に相当
  --     NOT NULL : 「NULL（値なし）を許可しない」制約。空のINSERTを拒否
  title text NOT NULL,
  -- author : 著者名。同じく text NOT NULL（必須項目）
  author text NOT NULL,

  -- (3) 書籍情報（任意）
  --     publisher : 出版社名のカラム
  --     NOT NULL を付けないので、INSERT時に省略可能 → NULLが入る
  publisher text,
  --     published_date : 出版日のカラム
  --     date : 「年月日」だけを保存する型。時刻は持たない（軽量）
  published_date date,

  -- (4) 読書管理
  --     rating  : 評価値（1〜5の星）のカラム
  --     integer : 整数型（負の値も入るが、CHECK制約で1〜5に絞る）
  --     CHECK ( ... ) : 値が条件を満たさなければINSERT/UPDATEを拒否する制約
  --     ここでは「rating は 1以上かつ5以下」を強制する（5を超える値は弾かれる）
  rating integer CHECK (rating >= 1 AND rating <= 5),

  --     status : 読書状態のカラム
  --     text   : 文字列で 'reading'/'completed'/'want_to_read' のいずれかを格納
  --     DEFAULT 'want_to_read' : INSERT時に省略されたら 'want_to_read' を入れる
  --     CHECK (status IN ('reading', 'completed', 'want_to_read'))
```

```
  --       → これらの値以外は登録不可（typoや不正値を防ぐ）
  --       → IN はリストにマッチするかを判定する演算子
  status text DEFAULT 'want_to_read'
    CHECK (status IN ('reading', 'completed', 'want_to_read')),

  -- memo : 読書メモ。任意（NULL可能）
  memo text,

  -- (5) 画像URL
  -- cover_url : 表紙画像のURLを格納。Supabase Storage 等のURLを想定
  cover_url text,

  -- (6) タイムスタンプ
  --     created_at : 作成日時のカラム
  --     timestamptz : 「タイムゾーン付きの日時」を表す型（推奨）
  --     NOW()       : 現在の日時を返す関数
  --     DEFAULT NOW() で「INSERT時に自動で現在時刻が入る」
  created_at timestamptz DEFAULT NOW(),
  -- updated_at : 更新日時。INSERT時はNOW()が入り、UPDATE時はトリガーで更新（後述）
  updated_at timestamptz DEFAULT NOW()
);
-- 上のセミコロンで CREATE TABLE 文が終わる

-- ▼ 実行結果（成功時、Supabase SQL Editor の出力）
--   Success. No rows returned
--   → CREATE 系の文は「行を返さない」ので「成功・行なし」と表示される。


-- ============================================================================
-- updated_at を自動更新するトリガー
-- ----------------------------------------------------------------------------
-- 「レコードが UPDATE されるたびに updated_at を現在時刻にする」仕組みを
-- DBレベルで実現する。アプリ側でいちいち書かなくていい。
-- ============================================================================

-- (1) まず「呼ばれた時に動く関数」を定義する。
--     CREATE OR REPLACE FUNCTION:
--       同名の関数が既にあれば置き換える、無ければ新規作成。
--       何度実行しても安全（冪等性=べきとうせい）。
--     update_updated_at_column : この関数の名前（自分で命名）
--     () : 引数なし
--     RETURNS TRIGGER:
--       戻り値の型がトリガー専用の特殊な値であることを示す。
--     AS $$ ... $$:
--       関数本体を囲む区切り。シングルクォートを多用するときに便利。
--     LANGUAGE plpgsql:
--       関数本体は PL/pgSQL（Postgres の手続き型言語）で書く宣言。
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
-- BEGIN ... END; : 関数本体の処理ブロック
```

```
BEGIN
  -- NEW は「これから書き込まれる新しい行」の擬似変数
  -- そのレコードの updated_at を現在時刻に書き換える
  -- NOW() で現在のタイムスタンプを取得し代入
  NEW.updated_at = NOW();
  -- 書き換えた NEW を返すことでDBに反映される
  -- RETURN を省略するとUPDATEがキャンセル扱いになるので必須
  RETURN NEW;
END;
-- $$ で関数本体の終わり
$$ LANGUAGE plpgsql;

-- (2) 上の関数を「books テーブルが UPDATE される直前」に呼ぶよう紐付ける
--     CREATE TRIGGER : トリガー（特定イベントで自動実行される処理）を作成
--     update_books_updated_at : トリガー名（自由命名）
--     BEFORE UPDATE: 「UPDATE実行の直前に呼ぶ」タイミング指定
--       (AFTER UPDATE だと「実行後に呼ぶ」)
--     ON books      : booksテーブルを対象にする
--     FOR EACH ROW : 「行ごとに1回呼ぶ」（複数行同時更新時も全行に効く）
--     EXECUTE FUNCTION ...: 実行する関数の指定
CREATE TRIGGER update_books_updated_at
  BEFORE UPDATE ON books
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at_column();

-- ▼ これ以降、UPDATE文を実行するたびに自動で updated_at が NOW() に書き換わる。
--   例: UPDATE books SET title='新タイトル' WHERE id='...';
--   → 自分で updated_at を指定しなくても、自動的に現在時刻が入る。

-- ============================================
-- テーブルとカラムにコメントを追加
-- Dashboard での表示が分かりやすくなる
-- ============================================

-- COMMENT ON TABLE テーブル名 IS '説明文'; でテーブル全体に注釈を付ける
COMMENT ON TABLE books IS '書籍管理テーブル';
-- COMMENT ON COLUMN テーブル.カラム名 IS '説明文'; でカラムに注釈を付ける
COMMENT ON COLUMN books.id IS '書籍ID（自動生成）';
COMMENT ON COLUMN books.title IS '書籍タイトル';
COMMENT ON COLUMN books.author IS '著者名';
COMMENT ON COLUMN books.publisher IS '出版社';
COMMENT ON COLUMN books.published_date IS '出版日';
COMMENT ON COLUMN books.rating IS '評価（1-5）';
COMMENT ON COLUMN books.status IS '読書状態（reading/completed/want_to_read）';
COMMENT ON COLUMN books.memo IS 'メモ・感想';
COMMENT ON COLUMN books.cover_url IS '表紙画像URL';
COMMENT ON COLUMN books.created_at IS '作成日時';
COMMENT ON COLUMN books.updated_at IS '更新日時';
```

▼ コードを1つずつ分解して解説

上の SQL は長く見えますが、「①テーブルを作る」「②自動更新の仕組みを作る」「③注釈を付ける」の3つに分かれています。SQL を初めて見る人向けに、塊ごとに区切って解説します。

解説1: CREATE TABLE で「表のひな型」を作る

```
CREATE TABLE books (
  id uuid DEFAULT gen_random_uuid() PRIMARY KEY,
  title text NOT NULL,
  author text NOT NULL,
```

- `CREATE TABLE books ( ... )` は「`books` という名前の新しいテーブル（表）を作る」という命令です。`( )` の中に、列（カラム）を1つずつカンマ区切りで並べます。
- 1つの列は「**列名** → **データ型** → **制約**」の3つの順で書きます。`title text NOT NULL` なら「`title` という名前」「文字列型」「空を許さない」という意味です。
- `uuid` は「世界中で重複しない長いID」を入れる型、`text` は「長さ制限のない文字列」を入れる型です。
- `DEFAULT gen_random_uuid()` は「INSERT時に値を指定しなければ、自動でUUIDを1つ作って入れる」初期値の指定です。

> **用語: カラム（列）** = 表の縦の項目（id・title など）。**データ型** = その列に入れられる値の種類（数値・文字列・日付など）。**制約（constraint）** = 「こういう値は入れてはダメ」というルール。

解説2: NOT NULL / DEFAULT / CHECK の3つの制約

```
  publisher text,
  published_date date,
  rating integer CHECK (rating >= 1 AND rating <= 5),
  status text DEFAULT 'want_to_read'
    CHECK (status IN ('reading', 'completed', 'want_to_read')),
```

- `NOT NULL` を**付けない**列（`publisher`・`published_date` など）は「空（NULL）でもOK＝任意項目」になります。逆に `NOT NULL` を付けると必須項目です。
- `CHECK ( 条件 )` は「条件を満たさない値は登録させない」制約です。`rating >= 1 AND rating <= 5` で「1以上5以下の整数しか入れられない」を強制します。
- `DEFAULT 'want_to_read'` は「INSERT時に `status` を省略したら自動で `'want_to_read'` を入れる」という初期値です。
- `IN ('reading', 'completed', 'want_to_read')` は「この3つの値のどれかであること」を判定します。これでスペルミスや想定外の値の登録を防げます。

> **用語: NULL（ヌル）** = 「値が入っていない（未入力）」状態のこと。空文字 `''` とは別物。**IN** = 「カッコ内のリストのどれかに一致するか」を調べるSQL演算子。

解説3: トリガー関数で「更新時に時刻を自動で書き換える」

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

- これは「行が更新される直前に呼ばれる小さな処理（関数）」を作る部分です。中身は「これから保存する行の `updated_at` を、今の時刻 `NOW()` に書き換える」だけです。
- `NEW` は「これから書き込まれる新しい行」を表す特別な変数です。`NEW.updated_at = NOW()` でその行の更新日時を現在時刻にしています。
- `RETURN NEW` で書き換えた行を返すことで、DBに反映されます（これを省くと更新がキャンセル扱いになります）。
- `CREATE OR REPLACE` なので、同じ名前の関数が既にあっても上書きされ、何度実行しても安全です。

> **用語: 関数（FUNCTION）** = DB内に保存しておき、必要なときに呼び出せる処理のまとまり。**NOW()** = 「現在の日時」を返すPostgreSQLの組み込み関数。

解説4: トリガーで関数を「UPDATEの直前」に紐付ける

```
CREATE TRIGGER update_books_updated_at
  BEFORE UPDATE ON books
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at_column();
```

- `CREATE TRIGGER` は「あるイベントが起きたら、自動で関数を呼ぶ」仕掛けを作る命令です。
- `BEFORE UPDATE ON books` は「`books` テーブルが UPDATE される**直前**に発動する」というタイミング指定です。
- `FOR EACH ROW` は「更新される行1つごとに1回呼ぶ」という意味です。10行まとめて更新すれば10回呼ばれます。
- `EXECUTE FUNCTION update_updated_at_column()` で、解説3で作った関数を実行します。これ以降、`updated_at` を自分で指定しなくても自動で最新時刻になります。

> **用語: トリガー（trigger）** =「DBで特定の操作（INSERT/UPDATE/DELETE）が起きたら自動で動く仕掛け」。引き金（trigger）のように、イベントをきっかけに処理が走る。

ステップ 3: SQL を実行

SQL Editor の右下にある「Run」ボタン（または `Ctrl + Enter` / `Cmd + Enter`）をクリックして実行します。

```
Success. No rows returned.
```

と表示されれば成功です。

ステップ 4: テーブルの確認

左メニューの「Table Editor」をクリックすると、作成した `books` テーブルが表示されているはずです。テーブル名をクリックすると、カラムの一覧やデータ（まだ空）が確認できます。

制約が本当に効くか試してみる

CREATE TABLE で書いた `NOT NULL` や `CHECK` が、本当に「おかしな値」を断ってくれるかを確かめておきましょう。わざと間違ったデータを入れてみるのがいちばん分かりやすい方法です。

> **▼ このコードがやること（先に日本語で）：** `books` テーブルに、①評価が範囲外（10）、②読書状態が決められた3つ以外（'done'）、③必須の著者が空、という3種類のおかしなデータを入れようとして、データベースに断られることを確認します。最後に正しいデータを1件入れて、省略した列に初期値（DEFAULT）が入ることも確かめ、片付けとして削除します。

```sql
-- ① rating（評価）に範囲外の 10 を入れてみる → CHECK 制約で断られるはず
INSERT INTO books (title, author, rating) VALUES ('テスト', 'テスト', 10);

-- ② status（読書状態）に決められた3つ以外の 'done' を入れてみる → これも CHECK で断られるはず
INSERT INTO books (title, author, status) VALUES ('テスト', 'テスト', 'done');

-- ③ 必須の author（著者）を省略してみる → NOT NULL で断られるはず
INSERT INTO books (title) VALUES ('著者なし');

-- ④ 正しいデータ。title と author だけを指定し、残りは初期値に任せる
INSERT INTO books (title, author) VALUES ('こころ', '夏目漱石');

-- ⑤ 入った行を確かめる
SELECT title, author, status, rating FROM books;

-- ⑥ 片付け: 確認用に入れた行を消しておく（8.1 節で改めてサンプルを入れるため）
DELETE FROM books WHERE title = 'こころ';
```

▼ **実行結果**（PostgreSQL 17 の psql で1文ずつ実行した出力。SQL Editor でまとめて Run すると、最初の ERROR の所で止まるので、**1文ずつ貼り替えて実行**してください）

```
ERROR:  new row for relation "books" violates check constraint "books_rating_check"
DETAIL:  Failing row contains (caf7938a-8cd0-4f79-8cdb-73aa147d02a5, テスト, テスト,
null, null, 10, want_to_read, null, null, 2026-10-17 02:29:33.925622+00, 2026-10-17
02:29:33.925622+00).
ERROR:  new row for relation "books" violates check constraint "books_status_check"
DETAIL:  Failing row contains (1a9b07ea-b3ec-4544-8799-3d585642b520, テスト, テスト,
null, null, null, done, null, null, 2026-10-17 02:29:33.933411+00, 2026-10-17
02:29:33.933411+00).
ERROR:  null value in column "author" of relation "books" violates not-null constraint
DETAIL:  Failing row contains (13b59f42-4ad4-4538-b428-ebb5c2497e31, 著者なし, null,
null, null, null, want_to_read, null, null, 2026-10-17 02:29:33.93456+00, 2026-10-17
02:29:33.93456+00).
INSERT 0 1
 title  |  author  |    status    | rating
--------+----------+--------------+--------
 こころ | 夏目漱石 | want_to_read |
(1 row)

DELETE 1
```

▼ コードを1つずつ分解して解説

解説1: エラー文は「どの表の」「どの制約に」違反したかを教えてくれる

```
ERROR:  new row for relation "books" violates check constraint "books_rating_check"
```

- 直訳すると「`books` 表への新しい行が、チェック制約 `books_rating_check` に違反しています」です。
- `books_rating_check` は、CREATE TABLE で `rating integer CHECK (...)` と書いたときに PostgreSQL が自動で付けた制約の名前です（「表名_列名_check」という決まった付け方）。名前を見れば、どの列のルールに引っかかったかが分かります。
- SQL Editor では、同じ英文の前に `23514` のような**エラーコード**（エラーの種類を表す番号）が添えられて表示されることがあります。`23514` は CHECK 違反、`23502` は NOT NULL 違反です。

解説2: DETAIL には「入れようとした1行」がまるごと出る

```
DETAIL:  Failing row contains (caf7938a-..., テスト, テスト, null, null, 10,
want_to_read, ...).
```

- `Failing row contains (...)` は「失敗した行の中身」です。列は CREATE TABLE で書いた順番（id, title, author, publisher, ...）に並んでいます。
- 自分では指定していない `id`（UUID）や `want_to_read`、日時が入っていることに注目してください。**DEFAULT（初期値）が埋められた後**に制約がチェックされている、という順番がここから読み取れます。

解説3: NOT NULL 違反は「どの列が空だったか」が分かる

```
ERROR:  null value in column "author" of relation "books" violates not-null constraint
```

- 「`books` 表の `author` 列に null（空）が入ろうとして、NOT NULL 制約に違反した」という意味です。
- 第7章で作る登録フォームでは、著者を空のまま送らないよう**画面側でも必須チェック**をします。画面とデータベースの二重の守りがあると、どちらかにバグがあってもおかしなデータが保存されません。

> **用語: 制約（constraint）** = 列に入れてよい値のルール。違反するとデータベースが保存を断り、エラーを返す。

4.3.1 Data API にテーブルを公開する（GRANT）

テーブルを作っただけでは、まだ **supabase-js（あなたの Next.js アプリ）からは使えません。**2026年に Supabase の仕様が変わり、新しく作ったテーブルは、自分で「公開します」と宣言するまでアプリ側の通信（**Data API**：supabase-js が使う、URL でデータベースを操作する窓口）から見えなくなったためです。

プロジェクトを作った時期	新しく作ったテーブル
2026年5月30日より前	これまでは自動で Data API に公開されていた（ただし 2026年10月30日からは、既存プロジェクトでも新しいテーブルは自動で公開されなくなる）
2026年5月30日以降	**自動では公開されない**。GRANT を書いて公開する必要がある

公開の宣言に使うのが **GRANT（グラント：権限を与える SQL の命令）** です。たとえるなら、GRANT は**ビルの入館証**、第5節で設定する RLS は**部屋の中の棚ごとの貸し出しルール**です。入館証がなければ建物に入れず、入館できても棚のルールで許された本しか手に取れません。supabase-js からの通信は、この2つの関門を順番に通ります。

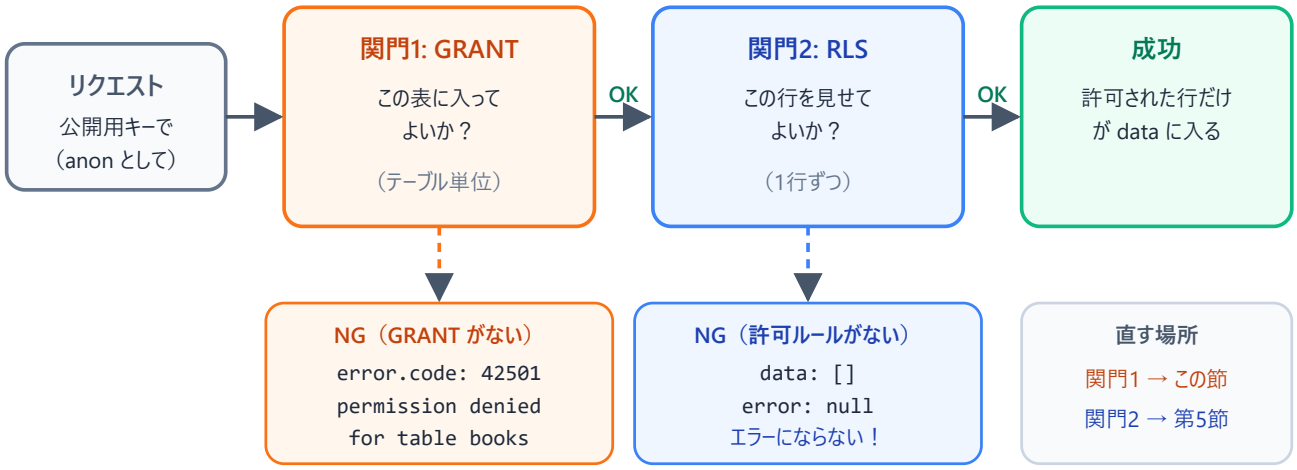


図5-9: supabase-js からの通信は「GRANT（表に入れるか）」と「RLS（どの行か）」の2つの関門を通る

GRANT は「**だれ（ロール）** に、**どの表** の、**どの操作** を許すか」を書きます。Supabase では、通信してきた相手を次の3つの**ロール**（role：役割・身分）に分けて扱います。

ロール	どんな相手か	本書での使い道
`anon`	ログインしていない人（公開用キーだけで来た人）	認証を入れるまでの書籍管理アプリは、すべてこの身分で動く
`authenticated`	ログインした人	発展の「認証」で使う
`service_role`	秘密キー（Secret key）を使うサーバー側の処理	管理用のスクリプトなど。RLS を素通りする

books テーブルを公開する SQL

> **▼ このコードがやること（先に日本語で）**：まず安全のために `books` の RLS をオンにし（許可ルールは第5節で作ります）、続けて3つのロールに `books` の読み取り・追加・更新・削除を許可します。最後に「だれにどの権限があるか」を一覧表示して、正しく公開できたかを確かめます。

```
-- 安全のため、先に RLS をオンにしておく（オンにした直後は「全部拒否」。許可ルールは第5節で作る）
ALTER TABLE books ENABLE ROW LEVEL SECURITY;

-- anon（ログインしていない人）に、books の読み取り・追加・更新・削除を許す
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO anon;

-- authenticated（ログインした人）にも同じ操作を許す
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO authenticated;

-- service_role（秘密キーを使うサーバー側の処理）にも同じ操作を許す
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO service_role;

-- 確認: books に対して、どのロールがどの権限を持っているかを一覧にする
SELECT grantee, string_agg(privilege_type, ', ' ORDER BY privilege_type) AS privileges
FROM information_schema.role_table_grants
WHERE table_schema = 'public' AND table_name = 'books'
  AND grantee IN ('anon', 'authenticated', 'service_role')
GROUP BY grantee
ORDER BY grantee;
```

▼ 実行結果（psql の出力。SQL Editor では最後の SELECT の表が表示されます）

```
ALTER TABLE
GRANT
GRANT
GRANT
    grantee    |           privileges
---------------+--------------------------------
 anon          | DELETE, INSERT, SELECT, UPDATE
 authenticated | DELETE, INSERT, SELECT, UPDATE
 service_role  | DELETE, INSERT, SELECT, UPDATE
(3 rows)
```

▼ コードを1つずつ分解して解説

解説1: GRANT 操作 ON 表 TO ロール

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO anon;
```

- 英語の語順どおり「**SELECT・INSERT・UPDATE・DELETE を、`public.books` について、`anon` に与える**」と読みます。
- `public.books` の `public` は**スキーマ**（テーブルを入れておくフォルダのようなもの）の名前です。何も指定せずに作ったテーブルは `public` に入ります。
- 認証を入れる前の書籍管理アプリは、画面から本を追加・編集・削除するので、`anon` にも4つの操作すべてを許しています。発展の「認証」を入れたら、`anon` からは書き込みの権限を外します（`REVOKE`：GRANT の取り消し）。

> **用語: ロール（role）** = データベースが「相手がだれか」を区別するための身分。**スキーマ（schema）** = テーブルなどをまとめて入れておく名前付きの入れ物。

解説2: 先に RLS をオンにする理由

```
ALTER TABLE books ENABLE ROW LEVEL SECURITY;
```

- GRANT だけを実行して RLS がオフのままだと、公開用キーを知っている**だれでも全行を読み書きできる**状態になります。公開用キーはブラウザで丸見えなので、これはとても危険です。
- そこで「入館証を配る前に、まず全部の棚に鍵をかける」順番にしています。RLS をオンにした直後は許可ルールが1つもないので、データは1行も見えません（次の第5節でルールを作ります）。

解説3: 確認用の SELECT は「権限の一覧表」を読んでいる

```
SELECT grantee, string_agg(privilege_type, ', ' ORDER BY privilege_type) AS privileges
FROM information_schema.role_table_grants
```

- `information_schema.role_table_grants` は、PostgreSQL が自動で用意している「だれに・どの表の・どの権限があるか」の一覧表です。
- `string_agg(...)` は、同じロールの権限を「DELETE, INSERT, …」のように1つの文字列につなげる関数です。3つのロールに4つずつ権限が並んでいれば成功です。
- 2026年5月より前に作ったプロジェクトでは、自動で付いていた `REFERENCES`・`TRIGGER`・`TRUNCATE` なども並ぶことがあります。上の4つが含まれていれば問題ありません。

> **つまずきポイント: GRANT を忘れて `permission denied for table books` になる**
>
> テーブルも RLS のポリシーも正しく作ったのに、supabase-js から読むと必ずエラーになる場合は、関門1（GRANT）で止まっています。新しいプロジェクトで最も起きやすいトラブルです。

```
// books を読む（6.5 節の接続テストと同じ処理）
const { data, error } = await supabase.from('books').select('title, status');
console.log('data :', data);
console.log('error:', error);
```

```
// ❌ NG: GRANT をしていないとき（PostgREST 14 + supabase-js 2.117 の実際の出力）
data : null
error: {
  code: '42501',
  details: null,
  hint: null,
  message: 'permission denied for table books'
}
```

```
// ✅ OK: この節の GRANT と、第5節のポリシーを作ったあと（表に2冊入っている場合）
data : [
  { title: 'こころ', status: 'reading' },
  { title: '人間失格', status: 'completed' }
]
error: null
```

> **ヒント:** エラーコード `42501` は「権限が足りない」という意味の PostgreSQL の番号です。環境によっては `hint` に `GRANT SELECT ON public.books TO anon;` のような直し方の例が表示されることもあります。GRANT の SQL は何度実行しても害はないので、迷ったらこの節の SQL をもう一度実行してください。

> **注意:** プロジェクト作成画面のセキュリティ設定に「新しいテーブルを自動で公開する（Automatically expose new tables）」のような項目がある場合、そこをオンにすると従来どおり自動で公開されます。ただし本書では、**何を公開しているかを自分で把握できる**ように、GRANT を書く方法をおすすめします（画面の表記は変わることがあります）。

4.4 Supabase Dashboard での GUI 操作手順（代替手段）

SQL を使わずに、Dashboard の GUI でテーブルを作成することもできます。ここではその手順を説明します。

> **推奨:** SQL での作成を推奨しますが、GUI での操作も知っておくと便利です。

ステップ 1: Table Editor を開く

左メニューの「Table Editor」をクリックします。

ステップ 2: 新しいテーブルを作成

「Create a new table」ボタンをクリックします。

ステップ 3: テーブル名の入力

- **Name:** `books` と入力
- **Description:** `書籍管理テーブル` と入力（任意）
- **Enable Row Level Security (RLS):** チェックを入れる（後で設定）

ステップ 4: カラムの追加

デフォルトで `id`（Primary Key）と `created_at`（timestamptz）が用意されています。ただし `id` の型は初期状態では `int8`（自動で増える連番）になっていることが多いので、4.3 節の SQL 版とそろえる場合は、`id` の型を `uuid` に変え、Default Value に `gen_random_uuid()` を指定してください。残りのカラムを以下の手順で追加します:

1. 「Add column」ボタンをクリック
2. 以下の情報を入力:

```
┌─────────────────────────────────────────────┐
│ Column Name: title                          │
│ Type: text                                  │
│ Default Value: (空欄)                       │
│ Primary: [ ]                                │
│ Nullable: [ ] ← チェックを外す（NOT NULL にする）  │
│ Unique: [ ]                                 │
│ [Save]                                      │
└─────────────────────────────────────────────┘
```

3. 同様に、残りのカラムも一つずつ追加: - `author` (text, NOT NULL) - `publisher` (text, Nullable) - `published_date` (date, Nullable) - `rating` (int4, Nullable) - `status` (text, Default: 'want_to_read') - `memo` (text, Nullable) - `cover_url` (text, Nullable) - `updated_at` (timestamptz, Default: now())
4. すべてのカラムを追加したら「Save」ボタンをクリック

> **注意:** GUI では CHECK 制約やトリガーを設定できないため、別途 SQL Editor で以下を実行する必要があります:

> **▼ このコードがやること（先に日本語で）:** GUI（画面ボタン操作）でテーブルを作った場合に**足りない設定を後から付け足す**SQLです。`ALTER TABLE ... ADD CONSTRAINT` で「rating は1〜5」「status は3つの値だけ」という**CHECK制約**を追加し、さらに `updated_at` 自動更新の**トリガー**も作ります。GUI操作の仕上げとして実行する、と捉えてください。

```sql
-- ALTER TABLE : 既存テーブルの定義を変更するSQL文
-- books       : 対象テーブル名
-- ADD CONSTRAINT 制約名 ... : 新しい制約を追加する
-- books_rating_check : 制約に付ける名前（テーブル名_カラム名_check が慣習）
-- CHECK (rating >= 1 AND rating <= 5) : rating が 1〜5 の範囲のみ許可
ALTER TABLE books
  ADD CONSTRAINT books_rating_check CHECK (rating >= 1 AND rating <= 5);

-- 同様に status の値を 3つに限定する制約を追加
-- IN ('reading', 'completed', 'want_to_read') : このリストの値以外は弾く
ALTER TABLE books
  ADD CONSTRAINT books_status_check CHECK (status IN ('reading', 'completed',
'want_to_read'));

-- updated_at 自動更新トリガーの追加（仕組みは前述と同じ）
-- CREATE OR REPLACE FUNCTION : 既存なら置換、なければ作成
CREATE OR REPLACE FUNCTION update_updated_at_column()
-- RETURNS TRIGGER : トリガー専用関数として宣言
RETURNS TRIGGER AS $$
BEGIN
  -- NEW.updated_at : これから書き込まれる行の updated_at を…
  -- NOW()          : 現在時刻に書き換える
  NEW.updated_at = NOW();
  -- 書き換えた行を返す（これでDBに反映される）
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- トリガーの作成: UPDATE 直前にこの関数を呼ぶよう紐付け
CREATE TRIGGER update_books_updated_at
  -- UPDATE 文の直前に発火
  BEFORE UPDATE ON books
  -- 行ごとに1回呼ぶ
  FOR EACH ROW
  -- 上で作った関数を実行
  EXECUTE FUNCTION update_updated_at_column();
```

> **注意:** GUI で作ったテーブルでも、コード（supabase-js）から使うには 4.3.1 節の「Data API への公開（GRANT）」が必要になる場合があります。Table Editor にも公開のオン／オフを切り替える表示が加わっていますが、画面の表記は変わりやすいので、迷ったら **4.3.1 節の GRANT の SQL をそのまま実行**してください。何度実行しても害はありません。

5. Row Level Security (RLS)

5.1 RLS とは

Row Level Security（RLS: 行レベルセキュリティ） とは、PostgreSQL の機能で、テーブルの各行に対して「誰がどの行にアクセスできるか」を制御する仕組みです。

なぜ RLS が必要か？

Supabase はフロントエンド（ブラウザ）から直接データベースにアクセスします。つまり、API キー（anon key）はブラウザの JavaScript コードに埋め込まれ、誰でも見ることができます。

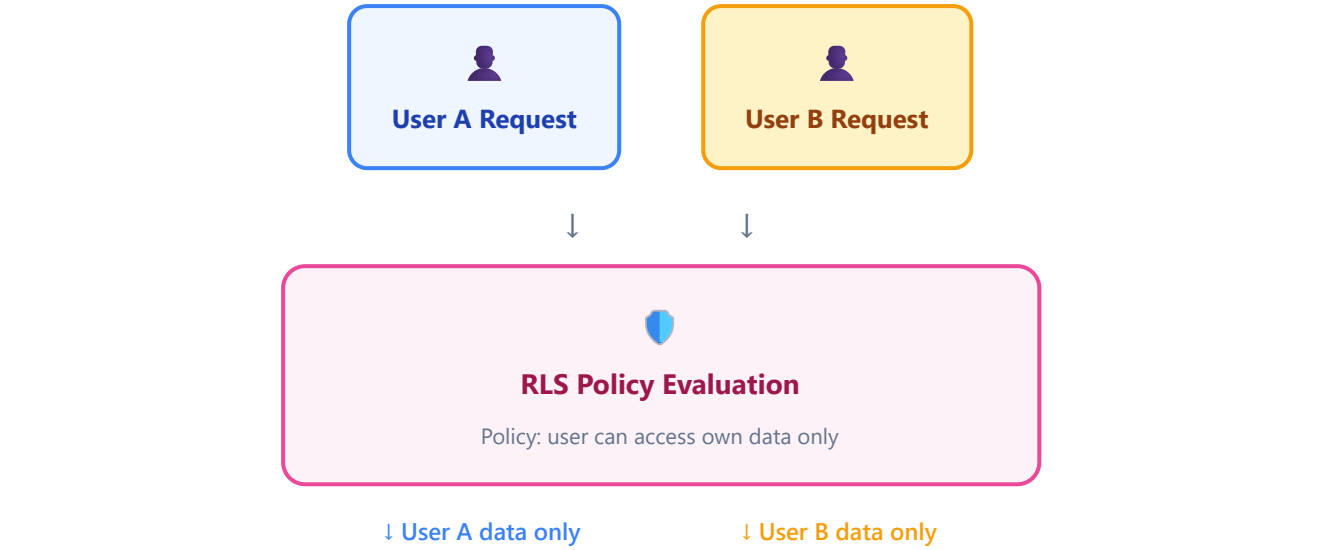
```
通常のサーバー構成：
  ブラウザ → API サーバー（ここでアクセス制御）→ データベース

Supabase の構成：
  ブラウザ → Supabase API（ここに RLS でアクセス制御）→ データベース
```

RLS がなければ、anon key を知っている人は誰でもすべてのデータを読み書きできてしまいます。RLS を有効にすることで、「どのユーザーがどのデータにアクセスできるか」をデータベースレベルで制御できます。

> **RLSを無効化したら何が起きる？**
>
> RLSがOFFの状態でテーブルを世に公開すると、anon key（フロントに埋め込まれている公開キー）を知る誰でもが
>
> 1. 全データを SELECT で読める（個人情報の漏えい）
> 2. 任意のデータを INSERT できる（スパム書き込み）
> 3. 既存データを UPDATE で改ざんできる
> 4. DELETE で全削除もできる（破壊行為）
>
> という状態になります。**本番環境ではRLSは必須**と覚えてください。ダッシュボードにも「Unprotected」と警告が出ます。

5.2 RLS の仕組み（図解）



上の図を「1行ずつ何が起きているか」まで拡大したのが次の図です。RLS は、テーブルの**行を1つずつ取り出して、ポリシーの条件式に当てはめ、`true` になった行だけを通す**検問所のように働きます。

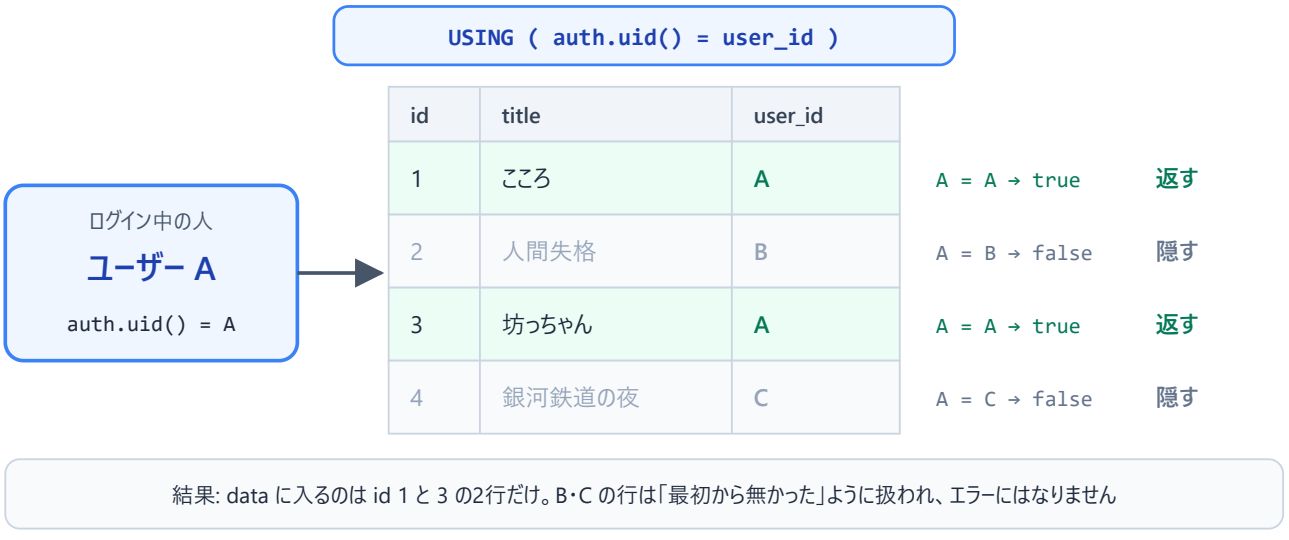


図5-10: RLS は1行ずつ条件式を計算し、true の行だけを返す（ユーザー A がアクセスした場合）

> **ヒント:** 「行が隠される」とき、Supabase は**エラーを返しません。**見えない行は「存在しない行」と同じ扱いになるからです。これが、第9節で扱う「エラーは出ないのに `data` が空配列 `[]` 」という定番トラブルの正体です。

5.3 書籍管理アプリでの RLS 設定（簡易版）

本チュートリアルではまだ認証機能を実装していないため、まずは **誰でも全データにアクセスできる** シンプルなポリシーを設定します。認証機能の実装後に、より厳密なポリシーに変更します。

> **注意:** RLS のポリシーは「入館した人が、どの行を触れるか」を決めるルールです。そもそも入館できなければ（4.3.1 節の **GRANT** をしていなければ）、ポリシーを書いても `permission denied for table books` で止まります。**4.3.1 節の GRANT を先に実行してから**、この節の SQL を実行してください。

SQL Editor で以下を実行してください:

> **▼ このコードがやること（先に日本語で）**：books テーブルに「**誰がどの行を操作できるか**」のルール（**RLS：行レベルセキュリティ**）を設定するSQLです。まず `ENABLE ROW LEVEL SECURITY` で防御を ON にし、次に `CREATE POLICY` で SELECT/INSERT/UPDATE/DELETE それぞれの**許可ルール**を1つずつ追加します。RLS は「何も書かなければ全部拒否、書いたぶんだけ許可」のホワイトリスト方式です。ここではまだ認証がないので、いったん**全員に全操作を許可（開発用）**にしています。

```sql
-- ================================================================================
-- RLS（Row Level Security）の設定
-- --------------------------------------------------------------------------------
-- 「テーブルの行ごとに、誰がアクセスできるかを決める」セキュリティ機能。
-- Supabase ではテーブル作成時の状態によってRLSの初期値が変わる。
-- ALTER TABLE で有効化し、CREATE POLICY で「許可ルール」を1つずつ追加していく。
-- ポリシーは「ホワイトリスト方式」: 何も設定しないと全部拒否、書いたぶんだけ許可される
-- ================================================================================

-- (1) books テーブルの RLS を有効化する
--     ALTER TABLE           : 既存テーブルの設定を変更するSQL文
--     books                 : 対象テーブル
--     ENABLE ROW LEVEL SECURITY : RLSを ON にする
--     ※ RLSを有効にしたあと、ポリシーを1つも作っていないと「全部拒否」になる
--        （SELECTしてもデータが空配列で返るのに気づかずハマる定番ポイント）
ALTER TABLE books ENABLE ROW LEVEL SECURITY;


-- (2) SELECT（読み取り）を全員に許可するポリシー
--     CREATE POLICY "ポリシー名" ON テーブル名
--     "Allow public read access" : ポリシー名（自由命名、空白OK）
--     ON books       : booksテーブルに適用
--     FOR SELECT     : どの操作（SELECT/INSERT/UPDATE/DELETE/ALL）に効くか
--     USING (条件)   : どの「既存の行」がこの操作に使えるかを決めるブール式
--     true は「常に真」 → 全行に対して許可するという意味
CREATE POLICY "Allow public read access"
  ON books
  FOR SELECT
  USING (true);


-- (3) INSERT（新規作成）を全員に許可するポリシー
--     "Allow public insert access" : ポリシー名
--     FOR INSERT     : INSERT操作向けポリシー
--     INSERTでは「これから書き込まれる新行」が対象なので USING ではなく
--     WITH CHECK を使う。
--     WITH CHECK (true) → どんな値の行でも書き込みOK
CREATE POLICY "Allow public insert access"
  ON books
  FOR INSERT
  WITH CHECK (true);


-- (4) UPDATE（更新）を全員に許可するポリシー
--     FOR UPDATE    : UPDATE操作向けポリシー
--     UPDATEは「既存の行を選んで」「新しい値で上書き」の2フェーズなので
--     USING（読み取り対象の判定）と WITH CHECK（書き込み内容の判定）の両方を書く。
CREATE POLICY "Allow public update access"
  ON books
```

```
  FOR UPDATE
  -- 全ての既存行を更新対象にできる
  USING (true)
  -- どんな新しい値でも書き込みOK
  WITH CHECK (true);


-- (5) DELETE（削除）を全員に許可するポリシー
--     FOR DELETE    : DELETE操作向けポリシー
--     DELETEは新規行を作らないので WITH CHECK は不要、USINGだけで判定する。
CREATE POLICY "Allow public delete access"
  ON books
  FOR DELETE
  USING (true);

-- ▼ 実行後の状態
--   このテーブルは「誰でもCRUD可能な開発用設定」になる。
--   本番運用や認証実装後は、この4つを DROP POLICY で削除し、
--   auth.uid() = user_id のような「自分のレコードだけ」ポリシーに差し替える。
```

▼ コードを1つずつ分解して解説

RLS は「**まず防御をON → 許可ルール（ポリシー）を1つずつ足していく**」という流れです。塊ごとに見ていきましょう。

解説1: ENABLE ROW LEVEL SECURITY で防御をONにする

```
ALTER TABLE books ENABLE ROW LEVEL SECURITY;
```

- `ALTER TABLE books` は「既存の `books` テーブルの設定を変更する」命令です（`CREATE` は新規作成、`ALTER` は変更）。
- `ENABLE ROW LEVEL SECURITY` で、そのテーブルの「行レベルセキュリティ（RLS）」をONにします。
- **重要な落とし穴:** RLSをONにした直後は「ポリシーが1つもない＝全部拒否」状態です。SELECTしてもエラーは出ず、ただ空配列 `[]` が返るだけなので、原因に気づきにくいです。

> **用語: RLS（Row Level Security）** ＝「テーブルの行（レコード）1件ごとに、誰がアクセスできるかを決める」しくみ。ホワイトリスト方式（許可したものだけ通す）。

解説2: CREATE POLICY で SELECT（読み取り）を許可する

```
CREATE POLICY "Allow public read access"
  ON books
  FOR SELECT
  USING (true);
```

- `CREATE POLICY "名前"` で許可ルールを1つ作ります。名前は自由で、空白を含めてもOKです。
- `ON books` は「`books` テーブルに対するルール」、`FOR SELECT` は「SELECT（読み取り）操作に効くルール」という指定です。
- `USING ( 条件 )` は「**どの既存の行が、この操作の対象になれるか**」を真偽（true/false）で判定する式です。
- `USING (true)` は「常に真＝すべての行を読み取り可能」という意味です（開発用の全許可）。

> **用語: ポリシー（policy）** = RLSにおける「許可ルール」1つ1つのこと。**USING** = 「すでにテーブルにある行のうち、どれを対象にできるか」を決める条件。

解説3: INSERT は WITH CHECK で「書き込む値」を判定する

```
CREATE POLICY "Allow public insert access"
  ON books
  FOR INSERT
  WITH CHECK (true);
```

- `FOR INSERT` は「新規追加の操作に効くルール」です。
- INSERTは「これから書き込む新しい行」が対象なので、`USING`（既存行の判定）ではなく `WITH CHECK`（これから書く行の判定）を使います。
- `WITH CHECK (true)` は「どんな値の行でも書き込みOK」という意味です。

> **用語: WITH CHECK** = 「これから書き込もうとしている（新しい/更新後の）行が、登録を許される値かどうか」を判定する条件。INSERT・UPDATE で使う。

解説4: UPDATE は USING と WITH CHECK の両方を使う

```
CREATE POLICY "Allow public update access"
  ON books
  FOR UPDATE
  -- 全ての既存行を更新対象にできる
  USING (true)
  -- どんな新しい値でも書き込みOK
  WITH CHECK (true);
```

- UPDATEは「①既存の行を選ぶ → ②新しい値で上書きする」の2段階なので、両方の判定が必要です。
- `USING (true)` で「すべての既存行を更新対象にできる」、`WITH CHECK (true)` で「上書き後の値は何でもOK」を表します。

- DELETE（解説では割愛しますが上のSQLの(5)）は新しい行を作らないため `WITH CHECK` は不要で、`USING` だけで「どの行を消せるか」を判定します。

> **用語: FOR SELECT / INSERT / UPDATE / DELETE** = そのポリシーが「どの操作に効くか」の指定。`FOR ALL` と書くと4つすべてに効く。

各ポリシーの解説:

ポリシー名	対象操作	`USING`	`WITH CHECK`	意味
Allow public read access	SELECT	`true`	-	すべての行を読み取り可能
Allow public insert access	INSERT	-	`true`	すべての行を挿入可能
Allow public update access	UPDATE	`true`	`true`	すべての行を更新可能
Allow public delete access	DELETE	`true`	-	すべての行を削除可能

`USING` と `WITH CHECK` の違い:

- `USING`: 既存の行に対する条件（SELECT, UPDATE, DELETE で使用）。「どの行が見えるか」
- `WITH CHECK`: 新しく書き込まれる行に対する条件（INSERT, UPDATE で使用）。「どの行を書き込めるか」

2つの条件が「いつ」チェックされるのかを、いちばん複雑な UPDATE で図にすると次のようになります。改札にたとえると、`USING` は**入口の改札**（この行に触ってよいか）、`WITH CHECK` は**出口の改札**（書き換えた結果を保存してよいか）です。



図5-11: USING は「変更前の行」、WITH CHECK は「変更後の行」を検査する

つまずきポイント: RLS をオンにしたのに許可ルールを作らず、`data` が空配列 `[]` になる

RLS は「書いたぶんだけ許可」のホワイトリスト方式です。オンにしただけでポリシーが1つもないと、すべての行が隠されます。しかも**エラーにはならない**ので、「データが入っていない」と勘違いしやすいのが厄介な点です。

```
// ❌ NG: GRANT 済み・RLS オン・ポリシーなし で books を読んだときの実際の出力
data : []
error: null
```

```
-- ✅ OK: 読み取り（SELECT）を許可するポリシーを作る
CREATE POLICY "Allow public read access"
  ON books
  FOR SELECT
  USING (true);
```

```
// ✅ OK: ポリシーを作ったあと、同じコードで読んだときの出力（表に2冊入っている場合）
data : [
  { title: 'こころ', status: 'reading' },
  { title: '人間失格', status: 'completed' }
]
error: null
```

- 見分け方のコツは、**SQL Editor で `SELECT * FROM books;` を実行してみる**ことです。SQL Editor はテーブルの持ち主（`postgres`）として実行されるので RLS の影響を受けません。SQL Editor では行が見えるのにアプリでは `[]` なら、原因は RLS のポリシーです。

つまずきポイント: 読み取りのポリシーだけ作って、追加（INSERT）が `new row violates row-level security policy` で失敗する

ポリシーは操作（SELECT / INSERT / UPDATE / DELETE）ごとに別々です。`FOR SELECT` のポリシーしかない状態で追加しようとすると、今度はエラーになります（読み取りと違って、書き込みの拒否はエラーとして返ってきます）。

```
// ❌ NG: SELECT のポリシーしかない状態で insert したときの実際の出力
error: {
  code: '42501',
  details: null,
  hint: null,
  message: 'new row violates row-level security policy for table "books"'
}
```

```
-- ✅ OK: 追加（INSERT）用のポリシーも作る。INSERT は「これから書く行」を調べるので WITH CHECK を使う
CREATE POLICY "Allow public insert access"
  ON books
  FOR INSERT
  WITH CHECK (true);
```

- 同じ `42501` でも、`permission denied for table books`（関門1: GRANT がない）と、`new row violates row-level security policy`（関門2: RLS のポリシーがない）は**直す場所が違います**。メッセージの英文まで読んで区別しましょう。

5.4 認証実装後の RLS ポリシー（参考）

第7章で認証機能を追加した後は、以下のようなポリシーに変更します。参考として掲載しておきます。

> **▼ このコードがやること（先に日本語で）**：認証（ログイン）を入れた後に使う、「**自分のデータだけ操作できる」厳しめのRLSポリシー**の見本です。鍵は `auth.uid() = user_id` という条件で、「いま**ログイン中のユーザーのID**」と「その行の持ち主」が一致する行だけを許可します。今はまだ実行せず、将来こう書き換えるのだという**完成イメージ**として読んでください（全行がコメントアウトされています）。

```
-- 認証済みユーザーが自分のデータのみアクセスできるポリシー
--  （第7章で実装予定。今は実行しないでください）

-- 既存のポリシーを削除（DROP POLICY: ポリシーを消すSQL）
-- DROP POLICY "Allow public read access" ON books;
-- DROP POLICY "Allow public insert access" ON books;
-- DROP POLICY "Allow public update access" ON books;
-- DROP POLICY "Allow public delete access" ON books;

-- ユーザーは自分のデータのみ SELECT 可能
-- auth.uid()  : Supabaseが提供する関数。現在ログインしているユーザーのUUIDを返す
-- user_id     : booksテーブルの「持ち主」を表すFKカラム（あとで追加予定）
-- = で両者が等しい行だけを許可する条件
-- CREATE POLICY "Users can view own books"
--   ON books
--   FOR SELECT
--   USING (auth.uid() = user_id);

-- ユーザーは自分の user_id でのみ INSERT 可能
-- INSERT時の「これから書く行」の user_id がログイン中ユーザーと一致するかを検査
-- CREATE POLICY "Users can insert own books"
--   ON books
--   FOR INSERT
--   WITH CHECK (auth.uid() = user_id);

-- ユーザーは自分のデータのみ UPDATE 可能
-- USING で「自分の行だけ」読み出し、WITH CHECK で「自分のIDのまま」書き込みを保証
-- CREATE POLICY "Users can update own books"
--   ON books
--   FOR UPDATE
--   USING (auth.uid() = user_id)
--   WITH CHECK (auth.uid() = user_id);

-- ユーザーは自分のデータのみ DELETE 可能
-- CREATE POLICY "Users can delete own books"
--   ON books
--   FOR DELETE
--   USING (auth.uid() = user_id);
```

6. Supabase Client のセットアップ

6.1 @supabase/supabase-js のインストール

プロジェクトのルートディレクトリで以下のコマンドを実行します。

```
# npm install : npmパッケージをプロジェクトに追加するコマンド
# @supabase/supabase-js : Supabase公式のJavaScript/TypeScript SDKパッケージ名
#                         "@組織名/パッケージ名" の形式（スコープ付きパッケージ）
npm install @supabase/supabase-js
```

正常にインストールされると、`package.json` の `dependencies` に追加されます。

```
{
  "dependencies": {
    "@supabase/supabase-js": "^2.x.x"
  }
}
```

> **`^2.x.x` の意味:** キャレット `^` は「メジャーバージョン（先頭の数字）は固定、それ以下は最新を使う」の意味。
> `^2.0.0` なら `2.x.x` の範囲で最新版が入る。

6.2 環境変数の設定

Supabase に接続するために必要な情報を環境変数として設定します。環境変数を使うことで、API キーなどの秘密情報をコードにハードコードせずに管理できます。

ステップ 1: Supabase Dashboard から情報を取得

1. Supabase Dashboard にログイン
2. 対象プロジェクトを選択
3. 画面上部の「**Connect**」ボタンをクリックし、表示された画面から Project URL と公開用キーをコピーする（いちばん簡単な方法）
4. 「Connect」が見つからない場合は、左メニュー下の「**Project Settings**」（歯車アイコン）を開き、「**Data API**」で Project URL を、「**API Keys**」で公開用キーを確認する（以前のダッシュボードでは「Settings」>「API」という1つの画面でした）
5. 以下の2つの値をコピー: - **Project URL**: `https://xxxxxxxx.supabase.co` - **公開用キー**: Publishable key（`sb_publishable_...`）、または従来の anon public key（`eyJhbGciOiJIUzI1NiIs...` で始まる長い JWT 形式の文字列）

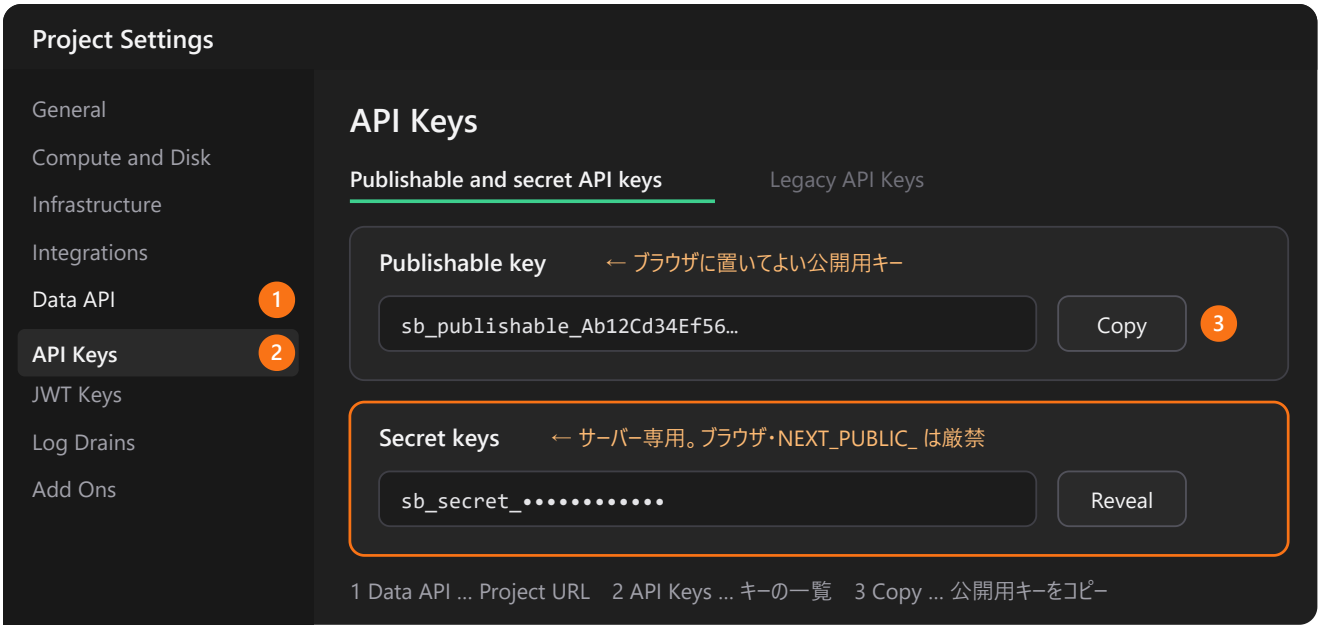


図5-12: Project Settings の API Keys 画面（画面イメージ）。コピーするのは上の Publishable key だけ

> **ヒント:**「Legacy API Keys」のタブには、従来の anon key と service_role key（どちらも `eyJ...` で始まる長い文字列）が表示されます。古い記事や動画の手順で「anon key をコピー」と出てきたら、新しい画面では Publishable key がその代わりだと読み替えてください。

ステップ 2: `.env.local` ファイルの作成

プロジェクトのルートディレクトリに `.env.local` ファイルを作成します。

> **▼ このコードがやること（先に日本語で）:** Supabase への**接続情報（URLとanonキー）を環境変数として書いておく**ファイルの中身です。「変数名=値」を1行ずつ書きます。先頭の `NEXT_PUBLIC_` は「この値はブラウザにも渡してよい」という Next.js の合図です。`xxxxxxxx` の部分は自分のプロジェクトの実際の値に置き換えること、そして**このファイルは絶対にGitに公開しない**ことを必ず守ってください。

```
# .env.local
# Supabase の接続情報
# 「キー=値」の形式で1行1変数を書く（イコールの周りに空白を入れない）

# NEXT_PUBLIC_SUPABASE_URL : Supabase プロジェクトのAPIエンドポイント
#   NEXT_PUBLIC_ プレフィックスはNext.jsの約束で「ブラウザにも値を渡してOK」を意味する
NEXT_PUBLIC_SUPABASE_URL=https://xxxxxxxx.supabase.co
# NEXT_PUBLIC_SUPABASE_ANON_KEY : 公開用キー（Publishable key。sb_publishable_ で始まる）
#   ブラウザに埋め込まれて公開される前提のキー。RLSと組み合わせて安全に使う
#   旧形式の anon key（eyJ... で始まる長い文字列）を貼っても動く（2026年末に廃止予定）
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxxxxxxxxx
```

重要な注意点:

1. `xxxxxxxx` の部分は、あなたのプロジェクトの実際の値に置き換えてください
2. `NEXT_PUBLIC_` プレフィックスは Next.js で環境変数をブラウザ側で使うために必要です（このプレフィックスがない環境変数はサーバー側だけで参照可能）
3. `.env.local` は **絶対に Git にコミットしないでください。** `.gitignore` に含まれていることを確認しましょう
4. **service_role key（管理者キー）を `.env.local` に書く場合は、絶対に `NEXT_PUBLIC_` を付けてはいけません。** 付けるとブラウザに漏れます

`NEXT_PUBLIC_` プレフィックスの仕組み: Next.js は `process.env.XXX` を「ビルド時に値を埋め込む」処理をします。`NEXT_PUBLIC_` で始まる名前のものだけがブラウザ向けバンドルにも埋め込まれます。そうでないものはサーバー側（API Route や Server Component）でしか参照できません。これにより「ブラウザに渡したい変数」と「サーバーに隠したい変数」を分けて管理できます。

つまずきポイント: 秘密キー（Secret key / service_role key）に `NEXT_PUBLIC_` を付けてしまう

「動かないから、とりあえず強いキーにしてみよう」と秘密キーを `NEXT_PUBLIC_` 付きの変数に入れると、ビルドした瞬間に**全世界へ公開**されます。RLS も素通りするので、だれでも全データを消せる状態になります。

```
# ❌ NG: 秘密キーに NEXT_PUBLIC_ を付けている（ブラウザ向けのファイルに埋め込まれてしまう）
NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY=sb_secret_xxxxxxxxxxxxxxxx
```

```
# ✅ OK: 画面から使うのは公開用キーだけ。秘密キーが必要になったら NEXT_PUBLIC_ を付けずにサーバー側だけで使う
NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_xxxxxxxxxxxxxxxx
SUPABASE_SECRET_KEY=sb_secret_xxxxxxxxxxxxxxxx
```

- 本書の書籍管理アプリでは、秘密キーは**一度も使いません。** `.env.local` には URL と公開用キーの2行だけを書けば十分です。
- もし秘密キーを `NEXT_PUBLIC_` 付きで公開してしまった（Git に push した、デプロイした）場合は、Project Settings の API Keys 画面でそのキーを削除・作り直し（ローテーション）してください。変数名を直すだけでは、すでに漏れたキーは無効になりません。

キーの置き方を点検するスクリプト

`.env.local` の書き間違いを、目で見るだけでなくプログラムで点検してみましょう。6.5 節で作る作業用フォルダ（`supabase-playground`）に置いて実行できます。

▼ **このコードがやること（先に日本語で）**：環境変数のうち名前に `SUPABASE` を含み `_KEY` で終わるものを1つずつ調べ、値の書き出し（ `sb_publishable_` / `sb_secret_` / 旧形式の JWT の中身）から「公開用」か「秘密」かを判定します。秘密キーなのに `NEXT_PUBLIC_` が付いていたら NG と表示します。通信はせず、手元の値を調べるだけです。

```ts
// 07-key-check.ts : .env.local のキーの置き方が安全かを点検する
// 実行方法: node --env-file=.env.local 07-key-check.ts
//  （通信はしません。手元の環境変数を調べるだけです）

// キーの文字列を見て「公開用」か「秘密」かを判定する関数
function keyKind(value: string): string {
  if (value.startsWith('sb_publishable_')) {
    return '公開用（Publishable key）';
  }
  if (value.startsWith('sb_secret_')) {
    return '秘密（Secret key）';
  }
  // 旧形式のキーは「.」で3つに区切られた JWT で、真ん中の部分に role が書かれている
  if (value.startsWith('eyJ')) {
    try {
      const middle = value.split('.')[1];
      const payload = JSON.parse(Buffer.from(middle, 'base64url').toString('utf8'));
      if (payload.role === 'anon') {
        return '公開用（旧 anon key）';
      }
      if (payload.role === 'service_role') {
        return '秘密（旧 service_role key）';
      }
    } catch {
      return '不明（JWT として読めません）';
    }
  }
  return '不明';
}

// 見つかった問題の数
let problems = 0;

// すべての環境変数のうち、名前に SUPABASE を含み、_KEY で終わるものを調べる
for (const [name, value] of Object.entries(process.env)) {
  if (!value || !name.includes('SUPABASE') || !name.endsWith('_KEY')) {
    continue;
  }
  const kind = keyKind(value);
  // NEXT_PUBLIC_ 付きの変数は、ブラウザに送られる
  const goesToBrowser = name.startsWith('NEXT_PUBLIC_');
  // 秘密キーがブラウザに送られる設定になっていたら NG
  const ok = !(goesToBrowser && kind.startsWith('秘密'));
  if (!ok) {
    problems++;
  }
  const mark = ok ? 'OK' : 'NG';
  const advice = ok ? '' : '  ← 秘密キーに NEXT_PUBLIC_ を付けてはいけません';
  console.log(`${mark}  ${name} : ${kind}${advice}`);
```

```
}

if (problems === 0) {
  console.log('問題は見つかりませんでした');
} else {
  console.log(`${problems}件の問題があります。キーを作り直す（ローテーション）ことも検討してくださ
い`);
}
```

▼ **実行結果**（Node.js 25.9 で実行。キーはダミーの値です。上が正しい `.env.local`、下が NG 例の `.env.local` で実行したもの）

```
$ node --env-file=.env.local 07-key-check.ts
OK  NEXT_PUBLIC_SUPABASE_ANON_KEY : 公開用（Publishable key）
OK  SUPABASE_SECRET_KEY : 秘密（Secret key）
問題は見つかりませんでした

$ node --env-file=.env.local 07-key-check.ts
OK  NEXT_PUBLIC_SUPABASE_ANON_KEY : 公開用（旧 anon key）
NG  NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY : 秘密（旧 service_role key）  ← 秘密キーに
NEXT_PUBLIC_ を付けてはいけません
1件の問題があります。キーを作り直す（ローテーション）ことも検討してください
```

▼ コードを1つずつ分解して解説

解説1: キーの「書き出し」で種類を見分ける

```
if (value.startsWith('sb_publishable_')) {
  return '公開用（Publishable key）';
}
```

- `startsWith('...')` は「文字列が `...` で始まるか」を調べるメソッドです。新しい形式のキーは、先頭を見るだけで公開用か秘密かが分かるように作られています。
- 旧形式のキー（`eyJ` で始まる JWT）は先頭が同じなので、真ん中の部分を Base64URL という方式で元の文字に戻し、中に書かれた `role`（`anon` か `service_role` か）を読んでいます。

> **用語: JWT（ジョット）** = 「.」で区切られた3つの部分（ヘッダー・中身・署名）からなる文字列。中身は暗号化ではなく**ただの変換**なので、だれでも読めます。だから秘密の情報を入れてはいけません。

解説2: Object.entries で環境変数を全部調べる

```
for (const [name, value] of Object.entries(process.env)) {
```

- `process.env` は「環境変数の名前 → 値」の入ったオブジェクトです。`Object.entries(...)` で `[名前, 値]` の組の配列に変え、`for ... of` で1組ずつ取り出しています。
- `--env-file=.env.local` を付けて実行すると、`.env.local` の中身が `process.env` に読み込まれます。

解説3:「秘密 かつ NEXT_PUBLIC_」だけを NG にする

```
const ok = !(goesToBrowser && kind.startsWith('秘密'));
```

- `&&` は「両方とも true のとき true」、先頭の `!` は「反対にする」です。つまり「ブラウザに送られる **かつ** 秘密キー」のときだけ `ok` が false（NG）になります。
- 秘密キーでも `NEXT_PUBLIC_` が付いていなければ（サーバー専用なら）OK、公開用キーに `NEXT_PUBLIC_` が付いているのも OK、という図5-7 のルールをそのままコードにしたものです。

ステップ 3: `.gitignore` に追加されていることを確認

Next.js のプロジェクトを `create-next-app` で作成した場合、`.env.local` はデフォルトで `.gitignore` に含まれています。念のため確認しましょう。

```
# .gitignore に以下が含まれていることを確認
# .env*.local は「.envで始まり.localで終わる任意のファイル」をGit管理から除外する
.env*.local
```

もし含まれていなければ、`.gitignore` に以下を追加してください:

```
# 環境変数ファイル
.env*.local
```

6.3 Supabase クライアントの初期化

Supabase に接続するためのクライアントを作成します。

ステップ 1: ファイルの作成

`src/lib/supabase.ts` ファイルを作成します。

```
# mkdir -p : 中間ディレクトリも含めて作る（既にあってもエラーにならない）
# src/lib  : srcの下にlibディレクトリを作るパス
mkdir -p src/lib
```

ステップ 2: クライアントコードの記述

▼ **このコードがやること（先に日本語で）**：アプリ全体から使い回す**Supabaseクライアント（DBへの接続窓口）を1個だけ作って共有する**ファイルです。 `.env.local` に書いた**接続URLとanonキー**を読み込み、 `createClient(...)` に渡して接続オブジェクトを生成します。値が未設定なら早めにエラーを出す仕組みも入れてあります。これを `export` しておくと、他のファイルから `import { supabase }` するだけでDB操作できるようになるのがポイントです。

```ts
// ============================================================================
// ファイルパス: src/lib/supabase.ts
// 役割       : アプリ全体で使う「Supabaseクライアント」を1個だけ作って共有する
// ----------------------------------------------------------------------------
// このファイルを作っておくと、他のファイルから
//   import { supabase } from "@/lib/supabase";
// と書くだけでDB操作できるようになる。
// クライアントを1個にまとめる(=シングルトン化)ことで、接続の無駄遣いを防ぐ。
// ============================================================================

// (1) Supabase クライアント作成関数を取り込む
//     import { createClient } : 名前付きインポート（特定の関数だけ取り込む構文）
//     @supabase/supabase-js   : Supabase の公式SDK（npmパッケージ）の名前
import { createClient } from '@supabase/supabase-js';

// (2) DBスキーマから自動生成した型を取り込む
//     `import type` は「型情報だけ取り込み、実行時のJSには残さない」記法
//     Database はテーブル定義から自動生成された型（後述）
//     @ は src/ を表すパスエイリアス（tsconfig.json の paths で設定済み）
//     よって @/types/supabase は src/types/supabase.ts を指す
import type { Database } from '@/types/supabase';

// (3) 環境変数から接続情報を取得
//     process.env はNode.jsで環境変数を読むオブジェクト
//     Next.jsの場合、ビルド時に NEXT_PUBLIC_ で始まる値が埋め込まれる
//     NEXT_PUBLIC_SUPABASE_URL : .env.local で設定したSupabaseのURL
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
// NEXT_PUBLIC_SUPABASE_ANON_KEY : .env.local で設定したanonキー（JWT文字列）
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

// (4) 環境変数が未設定の場合は早期にエラーを出す（Fail-Fast）
//     これを書かないと「なぜか動かない」と原因不明になりやすい。
//     エラー文に「.env.local を確認」と書いて初心者にやさしくする。
//     !（論理否定）で「変数がundefined/空文字なら true」を判定
if (!supabaseUrl) {
  // throw new Error : エラーオブジェクトを投げる構文。実行を即停止する
  throw new Error(
    'NEXT_PUBLIC_SUPABASE_URL が設定されていません。.env.local ファイルを確認してください。'
  );
}

if (!supabaseAnonKey) {
  throw new Error(
    'NEXT_PUBLIC_SUPABASE_ANON_KEY が設定されていません。.env.local ファイルを確認してください。'
  );
}

// (5) Supabase クライアントを作って export する
```

```
//     createClient(url, anonKey) : Supabaseに接続するためのオブジェクトを返す関数
//     <Database> はジェネリクス型引数。これを渡しておくと
//       supabase.from('books').select('*') と書いたときに
//       VS Code が books の存在やカラム名を補完・検証してくれる。
//     export const にすることで、他のファイルから
//       import { supabase } from "@/lib/supabase";
//     で使えるようになる。
export const supabase = createClient<Database>(supabaseUrl, supabaseAnonKey);

// ▼ 使用例（別ファイルから）
//   import { supabase } from "@/lib/supabase";
//   const { data, error } = await supabase.from("books").select("*");
// Book[] 型（自動推論）
//   console.log(data);
```

▼ コードを1つずつ分解して解説

このファイルは「①必要なものを取り込む → ②接続情報を読む → ③値の有無をチェック → ④クライアントを作って共有する」という流れです。塊ごとに見ていきます。

解説1: createClient と型をインポートする

```
import { createClient } from '@supabase/supabase-js';

import type { Database } from '@/types/supabase';
```

- `import { createClient } from '@supabase/supabase-js'` は、Supabaseの公式SDKから「クライアントを作る関数」だけを取り込む書き方です（名前付きインポート）。
- `import type { Database }` は「**型情報だけ**を取り込む」書き方で、実行時のJavaScriptには残りません。`Database` はテーブル定義から自動生成した型です（このあとの 6.4 で作ります）。
- `@/types/supabase` の `@` は「`src/` フォルダ」を指す近道（パスエイリアス）で、`src/types/supabase.ts` を意味します。

> **用語: import** = 別ファイルやパッケージの機能を「持ち込む」命令。**SDK** = ある機能を使うための道具一式（ここではSupabase操作用のライブラリ）。

解説2: 環境変数から接続情報を読み込む

```
const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL;
const supabaseAnonKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;
```

- `process.env.XXX` は「環境変数 XXX の値を読む」書き方です。さきほど `.env.local` に書いた値がここに入ってきます。
- URLとanonキーをコードに直接書かず環境変数から読むことで、秘密情報をソースコードに埋め込まずに済みます。
- `NEXT_PUBLIC_` で始まる名前なので、これらの値はブラウザ側でも参照できます（anonキーは公開前提のキーなのでOK）。

> **用語: 環境変数** = プログラムの外側（OSや設定ファイル）から値を渡すしくみ。接続先やキーなど、環境ごとに変わる値を入れておく。

解説3: 値が無ければ早めにエラーを出す（Fail-Fast）

```
if (!supabaseUrl) {
  throw new Error(
    'NEXT_PUBLIC_SUPABASE_URL が設定されていません。.env.local ファイルを確認してください。'
  );
}
```

- `!supabaseUrl` は「`supabaseUrl` が空（undefinedや空文字）なら true」という判定です。`!` は「〜でない」を表す否定記号です。
- 値が無いまま進むと「なぜか動かない」原因不明の状態になりがちです。そこで `throw new Error(...)` で**すぐに**処理を止め、分かりやすいメッセージを出します。
- このように「問題があれば早く失敗させる」考え方を Fail-Fast（フェイルファスト）と呼びます。

> **用語: throw（スロー）** = エラーを発生させて処理を中断する命令。**Error オブジェクト** = エラーの内容（メッセージなど）を入れる入れ物。

解説4: クライアントを作って export し、共有する

```
export const supabase = createClient<Database>(supabaseUrl, supabaseAnonKey);
```

- `createClient(url, anonKey)` で「Supabaseに接続するためのオブジェクト」を1つ作ります。
- `<Database>` は型情報を渡す部分（ジェネリクス）です。これを渡しておくと、`supabase.from('books').select('*')` と書いたときにエディタがテーブル名やカラム名を補完・検証してくれます。
- `export const supabase = ...` で外部に公開しているので、他のファイルから `import { supabase } from '@/lib/supabase'` と書くだけで同じ接続を使い回せます（シングルトン）。

> **用語: export** = このファイルの値を「他ファイルから使えるように公開する」命令。**シングルトン** = 「インスタンスを1個だけ作って全体で共有する」設計。

コードの解説:

1. `createClient`: Supabase クライアントを作成する関数。`@supabase/supabase-js` からインポート
2. `Database`: TypeScript の型定義（次のセクションで生成）。これにより、テーブル名やカラム名の補完が効くようになる
3. `process.env.NEXT_PUBLIC_SUPABASE_URL`: 環境変数から Supabase の URL を取得
4. `process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY`: 環境変数から API キーを取得
5. エラーチェック: 環境変数が未設定の場合、分かりやすいエラーメッセージを表示

> **ヒント:** 第4章 8.3 節では、`@supabase/ssr` の `createBrowserClient` / `createServerClient` を使ったクライアントの作り方を紹介しました。あちらは**ログイン状態をクッキーで受け渡す**ための作り方で、認証を入れるときに必要になります。この章のようにログインなしでデータを読み書きする段階では、ここで作った `createClient` の1つで十分です。

6.4 TypeScript 型の自動生成（supabase gen types）

Supabase CLI を使うと、データベースのスキーマ（テーブル定義）から TypeScript の型を自動生成できます。これにより、コード内でテーブル名やカラム名の入力補完が効くようになり、タイプミスを防げます。

ステップ 1: Supabase CLI のインストール

```
# -D は --save-dev の短縮形。開発時のみ使うパッケージとして package.json に記録
# supabase : CLIツールのパッケージ名
npm install -D supabase
```

ステップ 2: Supabase CLI にログイン

```
# npx : ローカルにインストールしたCLIツールを実行する命令
# supabase login : Supabaseアカウントにログインする
# 実行するとブラウザが開き、ログイン処理が走る
npx supabase login
```

ブラウザが開き、Supabase へのログインが求められます。認証を完了してください。

ステップ 3: 型定義ファイルの生成

> **▼ このコードがやること（先に日本語で）**：データベースのテーブル定義を読み取り、それに対応するTypeScriptの型定義を自動で作って `src/types/supabase.ts` に書き出すコマンドです。これを実行しておくと、コードで `books` のカラム名を間違えたときにエディタが教えてくれるようになります。`--project-id` の部分は自分のプロジェクトIDに置き換えるのを忘れないでください。

```
# プロジェクトのリファレンス ID を確認
# Supabase Dashboard の URL: https://supabase.com/dashboard/project/[ここがリファレンスID]
# または Settings > General > Reference ID で確認

# npx supabase gen types typescript : DBスキーマからTSの型定義を生成するサブコマンド
# --project-id "..."   : 対象のプロジェクトID（自分のものに置き換え）
# --schema public      : publicスキーマのテーブルを対象にする（標準のスキーマ名）
# > src/types/supabase.ts : 標準出力をファイルにリダイレクト（ファイルが上書きされる）
npx supabase gen types typescript --project-id "あなたのプロジェクトID" --schema public >
src/types/supabase.ts
```

> **注意:** `あなたのプロジェクトID` は Supabase Dashboard の URL に含まれる英数字の文字列です。

ステップ 4: 型定義ディレクトリの作成

事前にディレクトリを作成しておく必要があります:

```
# src/types ディレクトリがなければ作る
mkdir -p src/types
```

ステップ 5: 生成される型定義ファイルの確認

自動生成された `src/types/supabase.ts` は以下のような内容になります（一部抜粋）:

```ts
// src/types/supabase.ts
// このファイルは自動生成されます。手動で編集しないでください。

// Json型 : PostgreSQLのjsonb等で扱える値の型を表現
// | はTypeScriptのユニオン型（このいずれかの型をとる）
export type Json =
  // 文字列
  | string
  // 数値
  | number
  // 真偽値
  | boolean
  // null
  | null
  // オブジェクト（キーは文字列、値は再帰的にJson）
  | { [key: string]: Json | undefined }
  // Jsonの配列
  | Json[];

// Database型 : DB全体のスキーマを表す型
// 「スキーマ → テーブル → Row/Insert/Update」のネスト構造
export type Database = {
  // publicスキーマ
  public: {
    // テーブル一覧
    Tables: {
      // booksテーブル
      books: {
        // SELECTで返ってくる「1行」の型
        Row: {
          // uuid → string
          id: string;
          // text NOT NULL → string
          title: string;
          // text NOT NULL → string
          author: string;
          // text NULLable → string | null
          publisher: string | null;
          // date → ISO形式の文字列 or null
          published_date: string | null;
          // integer → number | null
          rating: number | null;
          // text → string | null
          status: string | null;
          memo: string | null;
          cover_url: string | null;
          // timestamptz → ISO文字列
          created_at: string | null;
          updated_at: string | null;
```

```
      };
      // INSERT時に渡すデータの型
      Insert: {
        // ? は「省略可」。DEFAULTがあるので省略OK
        id?: string;
        // NOT NULL なので必須
        title: string;
        // NOT NULL なので必須
        author: string;
        // NULL許可なので省略可
        publisher?: string | null;
        published_date?: string | null;
        rating?: number | null;
        // DEFAULTがあるので省略可
        status?: string | null;
        memo?: string | null;
        cover_url?: string | null;
        created_at?: string | null;
        updated_at?: string | null;
      };
      // UPDATE時に渡すデータの型
      Update: {
        // 全カラムが省略可（部分更新を想定）
        id?: string;
        title?: string;
        author?: string;
        publisher?: string | null;
        published_date?: string | null;
        rating?: number | null;
        status?: string | null;
        memo?: string | null;
        cover_url?: string | null;
        created_at?: string | null;
        updated_at?: string | null;
      };
    };
  };
  // ビュー（仮想テーブル）一覧。今は空
  Views: {
    [_ in never]: never;
  };
  // ストアド関数一覧。今は空
  Functions: {
    [_ in never]: never;
  };
  // enum型一覧。今は空
  Enums: {
    [_ in never]: never;
  };
```

```
    };
  };
```

型定義の3つの種類:

型	説明	使い方
`Row`	SELECT で取得される行の型。全カラムが含まれる	データ表示時に使用
`Insert`	INSERT で送信するデータの型。デフォルト値があるカラムは optional（`?`）	データ作成時に使用
`Update`	UPDATE で送信するデータの型。全カラムが optional	データ更新時に使用

ステップ 6: 便利な型エイリアスの作成

コードで使いやすいように、型のエイリアス（別名）を作成しておきましょう。

> **▼ このコードがやること（先に日本語で）**：自動生成された長い型（`Database['public']['Tables']['books']['Row']` など）に、`Book` や `BookInsert` といった**短い別名（エイリアス）を付けてまとめる**ファイルです。こうしておくと、各ファイルで毎回長い型名を書かずに済みます。あわせて、読書状態の3つの値だけを許す型と、その**日本語ラベル**も用意しています。

```ts
// src/types/book.ts
// books テーブル専用の型エイリアスをまとめたファイル

// 自動生成された Database 型を取り込む
// import type : 型だけ取り込む（実行時のコードには出力されない）
// './supabase' は同じ src/types フォルダの supabase.ts
import type { Database } from './supabase';

// books テーブルの型エイリアス
// Database['public']['Tables']['books']['Row'] という長いパスを Book で再利用可能にする
// ['public'] = publicスキーマ、 ['Tables'] = テーブル群、 ['books'] = booksテーブル
// ['Row'] = SELECTで返ってくる行の型
export type Book = Database['public']['Tables']['books']['Row'];
// INSERT 用の型（省略可能なフィールドあり）
export type BookInsert = Database['public']['Tables']['books']['Insert'];
// UPDATE 用の型（全フィールド省略可能）
export type BookUpdate = Database['public']['Tables']['books']['Update'];

// 読書状態の型
// リテラルユニオン型 : この3つの文字列以外を許さない型
// statusカラムは text だが、CHECK制約のおかげで実質この3つしか入らない
export type BookStatus = 'reading' | 'completed' | 'want_to_read';

// 読書状態の日本語ラベル
// Record<K, V> : すべてのキーKに対して値がVの型 を作るユーティリティ型
// Record<BookStatus, string> = { reading: string, completed: string, want_to_read:
string }
export const BOOK_STATUS_LABELS: Record<BookStatus, string> = {
  // 'reading' を画面表示するときの日本語
  reading: '読書中',
  // 'completed' の表示
  completed: '読了',
  // 'want_to_read' の表示
  want_to_read: '読みたい',
};
```

ステップ 7: package.json にスクリプトを追加

型生成を簡単に実行できるよう、`package.json` にスクリプトを追加します。

```
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "gen:types": "supabase gen types typescript --project-id \"あなたのプロジェクトID\" --
schema public > src/types/supabase.ts"
  }
}
```

> **`gen:types` スクリプトの中身解説:**
>
> - `supabase gen types typescript` : Supabase CLI のサブコマンド。TypeScriptの型を生成
> - `--project-id "..."` : 対象プロジェクトIDを指定
> - `--schema public` : publicスキーマを対象に
> - `> src/types/supabase.ts` : 出力先ファイル（リダイレクト）
> - JSON内のダブルクォートはエスケープ（ `\"` ）が必要

これにより、テーブル定義を変更した後は以下のコマンドで型を再生成できます:

```
# npm run スクリプト名 で package.json の scripts を実行
npm run gen:types
```

6.5 画面を作る前に、ターミナルから接続を確かめる

Next.js の画面を作り始める前に、「Supabase に本当につながるか」「GRANT や RLS の設定が正しいか」を、小さなスクリプトで確かめておきましょう。画面を作ってから問題が見つかると、原因が「画面のコード」なのか「データベースの設定」なのかを切り分けにくくなるからです。料理で言えば、本番の前に**味見**をしておくイメージです。

Node.js 25 は TypeScript のファイルをそのまま実行できるので（型のチェックはせず、型の部分を取り除いて実行します）、準備は少しで済みます。

ステップ 1: 作業用フォルダを作る

```
# 作業用フォルダを作って、その中に移動する
mkdir supabase-playground
cd supabase-playground

# package.json を作る（-y は質問に全部「はい」で答えるオプション）
npm init -y

# import 文を使えるように、このフォルダを「ES モジュール」の設定にする
npm pkg set type=module

# Supabase の公式ライブラリを入れる
npm install @supabase/supabase-js
```

ステップ 2: `.env.local` を置く

`supabase-playground` フォルダの中にも、6.2 節と同じ2行の `.env.local` を作ります（Next.js のプロジェクトとは別のフォルダなので、コピーして置きます）。

ステップ 3: 接続テストのスクリプトを書く

> **▼ このコードがやること（先に日本語で）：** `.env.local` から URL と公開用キーを読み込んで Supabase に接続し、`books` テーブルから題名と読書状態を取り出して、返ってきた `data` と `error` の両方をそのまま表示します。設定のどこに問題があるかで表示が変わるので、「診断用の聴診器」として何度でも使えます。

```
// 01-check.ts : Supabase につながるか確かめる「接続テスト」スクリプト
// 実行方法: node --env-file=.env.local 01-check.ts

// Supabase の公式ライブラリから「クライアントを作る関数」を取り込む
import { createClient } from '@supabase/supabase-js';

// .env.local に書いた接続情報を読み込む（--env-file オプションが読み込んでくれる）
const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
const key = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY;

// 読み込めていなければ、原因が分かるメッセージを出して終了する
if (!url || !key) {
  console.error('環境変数が読み込めていません。--env-file=.env.local を付けましたか？');
  process.exit(1);
}

// 接続窓口（クライアント）を作る
const supabase = createClient(url, key);

// books テーブルから title と status を取り出す（title の順に並べる）
const { data, error } = await supabase
  .from('books')
  .select('title, status')
  .order('title');

// 結果の「2つの箱」を両方とも表示してみる
console.log('data :', data);
console.log('error:', error);
```

▼ **実行結果**（Node.js 25.9 / supabase-js 2.117.3 / PostgREST 14.18 / PostgreSQL 17 の環境で、設定の段階ごとに実行した実際の出力）

```
$ node --env-file=.env.local 01-check.ts
（段階A: テーブルを作っただけで、4.3.1 節の GRANT をしていない）
data : null
error: {
  code: '42501',
  details: null,
  hint: null,
  message: 'permission denied for table books'
}

$ node --env-file=.env.local 01-check.ts
（段階B: GRANT と RLS のオンまで済んだが、第5節のポリシーをまだ作っていない）
data : []
error: null

$ node --env-file=.env.local 01-check.ts
（段階C: 第5節のポリシーまで作り、表に2冊入っている）
data : [
  { title: 'こころ', status: 'reading' },
  { title: '人間失格', status: 'completed' }
]
error: null
```

段階Cの形が出れば、データベース側の準備は完了です。段階A・Bのどちらかで止まっていたら、図5-9 の「直す場所」に戻ってください。

▼ コードを1つずつ分解して解説

解説1: `--env-file=.env.local` で環境変数を読み込む

```
node --env-file=.env.local 01-check.ts
```

- Next.js は `.env.local` を自動で読み込みますが、`node` コマンドで直接スクリプトを動かすときは自動では読み込まれません。`--env-file=ファイル名` を付けると、そのファイルの「名前=値」が `process.env` に入ります。
- `.env.local` の中身を画面に表示したり、ファイルをどこかに貼り付けたりせずに済むので、安全に試せます。

解説2: トップレベル await

```
const { data, error } = await supabase
  .from('books')
  .select('title, status')
  .order('title');
```

- 関数の外で `await` を書けるのは、ステップ1の `npm pkg set type=module` で ES モジュールとして動かしているからです（トップレベル await）。

- `.order('title')` は SQL の `ORDER BY title` にあたり、結果の並び順を決めます。

解説3: `data` と `error` を「両方」表示する

```
console.log('data :', data);
console.log('error:', error);
```

- 診断用なので、成功・失敗を決めつけずに2つとも表示しています。段階A では `data` が `null` で `error` に中身があり、段階B・C では `error` が `null` です。
- 段階B のように「`error` は `null` なのに `data` が空」という形は、7.1 節の図5-13 で「成功だけど0件」として整理します。

> **つまずきポイント: `--env-file` を付け忘れて、環境変数が読み込まれない**
>
> `node 01-check.ts` とだけ打つと、`.env.local` は読み込まれません。スクリプトの先頭で値の有無を確かめているので、次のメッセージで止まります。

```
$ node 01-check.ts
環境変数が読み込めていません。--env-file=.env.local を付けましたか？
```

```
$ node --env-file=.env.local 01-check.ts
（✅ OK: 正しく読み込まれ、段階A〜C のどれかの結果が表示される）
```

- ファイル名を打ち間違えた場合は、Node.js 自身が `.env.lcoal: not found`（`.env.lcoal` の部分は打ち間違えた名前）というメッセージを出して止まります。Windows では、先頭に `C:\Program Files odejs ode.exe:` のような Node.js の場所が付いて表示されます。
- Next.js の開発サーバー（`npm run dev`）で `.env.local` を書き換えたときは、**サーバーを再起動するまで新しい値が反映されない**ことにも注意してください（9.1 節）。

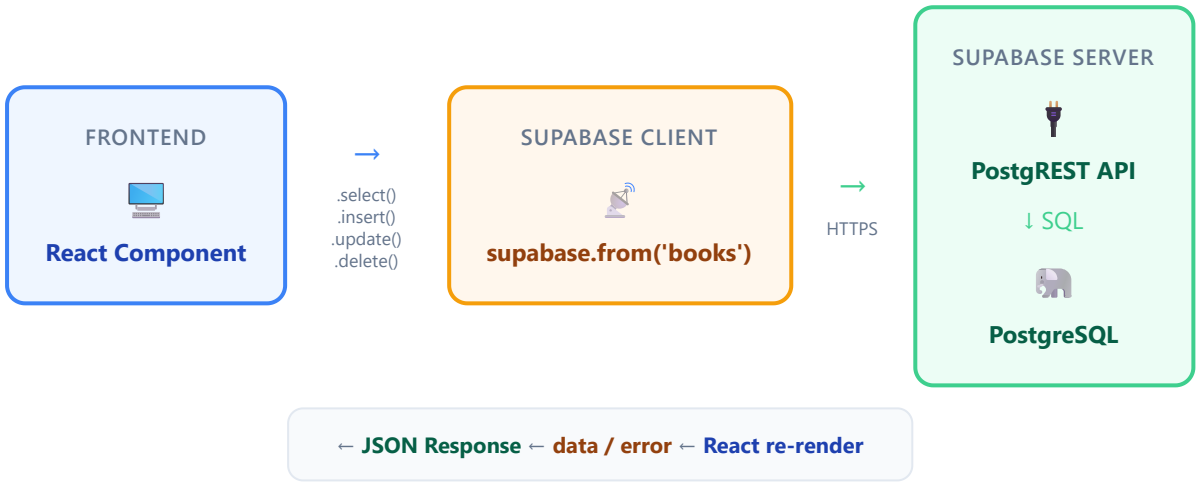
---

7. Supabase の基本操作（CRUD）

CRUD とは、データベースの4つの基本操作の頭文字をとったものです。

操作	意味	SQL	Supabase メソッド
Create	作成	INSERT	`.insert()`
Read	読み取り	SELECT	`.select()`
Update	更新	UPDATE	`.update()`
Delete	削除	DELETE	`.delete()`

以下のデータフローを意識しながら、各操作を学んでいきましょう。



7.1 SELECT（データの読み取り）

全件取得

> **▼ このコードがやること（先に日本語で）**：booksテーブルの**全データを取り出す**、CRUDの「R（読み取り）」の基本形です。`.from('books').select('*')` で「booksテーブルの全カラムをちょうだい」という意味になります。Supabase は失敗しても例外を投げず、戻り値の `error` に情報を入れる設計なので、**`error` を必ず確認してから `data` を使う**のが鉄則です。

```
// すべての書籍を取得するクエリ
// await       : このAPI呼び出しが終わるまで待つ（時間がかかる処理）
// supabase    : 6.3で作ったクライアントオブジェクト
// .from('books') : booksテーブルを操作対象にする（テーブル名を指定）
// .select('*')   : 全カラムを取得する（'*' はワイルドカード）
// 戻り値は { data, error } というオブジェクト。分割代入で取り出す
const { data, error } = await supabase
  .from('books')
  .select('*');

// エラーチェック : Supabaseは例外を投げない設計。errorで判定するのが基本
if (error) {
  // console.error : エラーログをコンソールに赤字で出力
  // error.message にユーザー向けの説明文が入っている
  console.error('エラー:', error.message);
  // 関数を抜ける（以降を実行しない）
  return;
}

// 成功した場合、data に取得結果が配列で入る
console.log('書籍一覧:', data);
// 結果例:
// [
//   {
//     id: '550e8400-e29b-41d4-a716-446655440000',
//     title: 'ノルウェイの森',
//     author: '村上春樹',
//     publisher: '講談社',
//     published_date: '1987-09-04',
//     rating: 5,
//     status: 'completed',
//     memo: '名作。何度も読み返したい。',
//     cover_url: null,
//     created_at: '2024-01-15T10:30:00.000Z',
//     updated_at: '2024-01-15T10:30:00.000Z'
//   },
//   ...
// ]
```

▼ コードを1つずつ分解して解説

この「全件取得」は、これ以降のCRUD操作すべての土台になる形です。塊ごとに分解します。

解説1: メソッドをつないで「どのテーブルから何を取るか」を組み立てる

```
const { data, error } = await supabase
  .from('books')
  .select('*');
```

- `supabase.from('books')` で「`books` テーブルを操作対象にする」と宣言し、続く `.select('*')` で「全カラム（`*`）を取り出す」と指定します。
- このように `.` でメソッドをつないでいく書き方をメソッドチェーンと呼びます。条件や並び替えも、このうしろにつなげて足していけます。
- `await` は「この通信が終わって結果が返るまで待つ」という意味です。DB操作は時間がかかるため必要です。
- 戻り値は `{ data, error }` という形のオブジェクトで、分割代入で `data`（取得結果）と `error`（エラー情報）を同時に取り出しています。

> **用語: メソッドチェーン** = `a.b().c()` のように、メソッドを点でつないで処理を組み立てる書き方。**分割代入** = オブジェクトから必要なプロパティだけを取り出して変数にする書き方。

解説2: error を必ず先にチェックする

```
if (error) {
  console.error('エラー:', error.message);
  return;
}
```

- Supabaseは失敗しても例外を投げず、`error` に情報を入れて返す設計です。そのため `data` を使う前に必ず `error` を確認します。
- `if (error)` は「`error` に中身があれば（＝失敗していれば）」という判定です。失敗時は `error.message`（人間向けの説明）をログに出します。
- `return` でそこより先（`data` を使う処理）に進まないようにします。これでエラー時に壊れた `data` を触らずに済みます。

> **用語: 例外（throw）を投げない設計** = エラーを `try/catch` で捕まえるのではなく、戻り値の `error` で受け取る方式。Supabaseはこちらを採用している。

`{ data, error }` の読み方: 3つのパターン

supabase-js の戻り値は、**2つの箱が並んだ封筒**だと考えると整理しやすくなります。封筒を開けると、必ず `data` の箱と `error` の箱が入っていて、どちらに何が入っているかで結果が分かります。実際の出力（6.5 節・7.5 節）から整理す

ると、パターンは次の3つです。

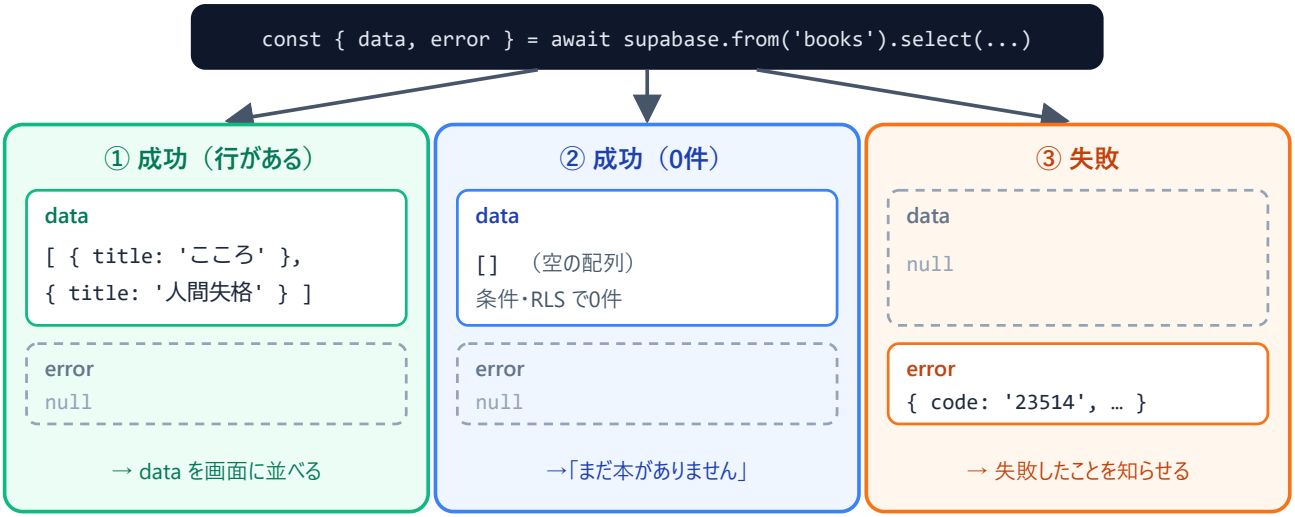


図5-13: 戻り値の3パターン。色の付いた箱に中身があり、点線の箱は null

- 判定の順番は「**まず `error` を見る → 次に `data` が空かどうかを見る**」です。①と②はどちらも `error` が `null`（成功）なので、`error` だけでは区別できません。
- ②の「0件」は失敗ではありません。条件に合う行がない、または RLS で隠された、のどちらかです（9.2 節）。

> **つまずきポイント: `error` を確認せずに `data` を使って、`TypeError` で止まる**
>
> 失敗したときの `data` は `null` です。`error` を見ずに `data.length` のように中身を読もうとすると、supabase-js ではなく**あなたのコード**が例外を投げて止まります。

```
// ❌ NG: error を見ずに data を使っている（rating に範囲外の 10 を入れて、わざと失敗させた例）
const { data } = await supabase
  .from('books')
  .insert({ title: 'テスト', author: 'テスト', rating: 10 })
  .select();
console.log(`${data.length}件追加しました`);
```

```
console.log(`${data.length}件追加しました`);
                    ^

TypeError: Cannot read properties of null (reading 'length')
```

```
// ✅ OK: 先に error を調べ、失敗なら data に触らずに抜ける
const { data, error } = await supabase
  .from('books')
  .insert({ title: 'テスト', author: 'テスト', rating: 10 })
  .select();
if (error) {
  console.error('追加に失敗しました:', error.message);
} else {
  console.log(`${data.length}件追加しました`);
}
```

```
追加に失敗しました: new row for relation "books" violates check constraint
"books_rating_check"
```

- `TypeError: Cannot read properties of null (reading 'length')` は「`null` の `length` を読もうとした」という意味です。この形のエラーを見たら、**直前の `error` チェックが抜けていないか**を疑いましょう（上の NG 出力は、Node.js 25.9 で実行したときのものから、ファイルの場所を示す行を省いています）。

特定のカラムのみ取得

> **▼ このコードがやること（先に日本語で）**：全カラムではなく、**欲しい列だけを指定して取り出す**例です。`.select('title, author, rating')` のようにカンマ区切りで列名を並べます。必要な列だけ取れば通信量が減って表示も速くなる、というのがポイントです。

```
// タイトルと著者のみ取得（データ転送量を減らせる）
// .select('title, author, rating') : カンマ区切りで取得したいカラムを文字列で指定
//   * で全列を取らず、必要な列だけ指定することで通信が軽くなる
const { data, error } = await supabase
  .from('books')
  .select('title, author, rating');

// 結果例: 指定したカラムだけのオブジェクトが配列で返る
// [
//   { title: 'ノルウェイの森', author: '村上春樹', rating: 5 },
//   { title: '人間失格', author: '太宰治', rating: 4 },
//   ...
// ]
```

条件付き取得（フィルタリング）

▼ **このコードがやること（先に日本語で）**：全件ではなく**条件に合う行だけ**を取り出す例です。`.gte('rating', 4)` は「rating が 4 以上（greater than or equal）」という絞り込み条件で、SQL の `WHERE rating >= 4` に相当します。`.select(...)` のうしろにこうした条件メソッドをつなげて使う、という形を覚えましょう。

```
// 評価が4以上の書籍のみ取得
// .gte('rating', 4) : rating >= 4 という条件を追加
//   gte = greater than or equal (以上)
//   メソッドチェーンで好きなだけ条件を追加できる
const { data, error } = await supabase
  .from('books')
  .select('*')
  // gte = greater than or equal (以上)
  .gte('rating', 4);

// 結果例:
// [
//   { title: 'ノルウェイの森', rating: 5, ... },
//   { title: '人間失格', rating: 4, ... },
// ]
```

▼ **このコードがやること（先に日本語で）**：「**ある列の値がぴったり一致する行**」だけを取り出す例です。`.eq('status', 'reading')` は「status が 'reading' と等しい行だけ」を意味し（eq = equal）、SQL の `WHERE status = 'reading'` と同じです。完全一致で絞りたいときの定番メソッドです。

```
// 読書中の書籍のみ取得
// .eq('status', 'reading') : status カラムが 'reading' と等しい行のみ
//   eq = equal (等しい)
//   SQLの WHERE status = 'reading' に相当
const { data, error } = await supabase
  .from('books')
  .select('*')
  // eq = equal (等しい)
  .eq('status', 'reading');
```

▼ **このコードがやること（先に日本語で）**：完全一致ではなく「**一部に含むかどうか（あいまい検索）**」で絞り込む例です。`.ilike('author', '%村上%')` の `%` は「任意の文字列」を表すワイルドカードで、「authorのどこかに『村上』を含む行」を探します。`ilike` は大文字・小文字を区別しない検索です。

```
// タイトルに「村上」を含む書籍を検索
// .ilike('author', '%村上%')
//   ilike = case-insensitive LIKE。大文字小文字を区別しないパターン一致
//   % はワイルドカード（任意の文字列）
//   '%村上%' は「文字列のどこかに『村上』を含む」を意味する
const { data, error } = await supabase
  .from('books')
  .select('*')
  // ilike = 大文字小文字を区別しない部分一致
  .ilike('author', '%村上%');
```

主なフィルタメソッド一覧:

メソッド	SQL 相当	説明	例
`.eq(column, value)`	`= value`	等しい	`.eq('status', 'reading')`
`.neq(column, value)`	`!= value`	等しくない	`.neq('status', 'completed')`
`.gt(column, value)`	`> value`	より大きい	`.gt('rating', 3)`
`.gte(column, value)`	`>= value`	以上	`.gte('rating', 4)`
`.lt(column, value)`	`< value`	より小さい	`.lt('rating', 3)`
`.lte(column, value)`	`<= value`	以下	`.lte('rating', 2)`
`.like(column, pattern)`	`LIKE pattern`	パターン一致	`.like('title', '%森%')`
`.ilike(column, pattern)`	`ILIKE pattern`	パターン一致（大文字小文字無視）	`.ilike('author', '%murakami%')`
`.is(column, value)`	`IS value`	NULL チェック	`.is('memo', null)`
`.in(column, values)`	`IN (values)`	複数値のいずれか	`.in('status', ['reading', 'completed'])`

ソート

▼ **このコードがやること（先に日本語で）**：取り出したデータを**指定した列の順番に並べ替える**例です。`.order('rating', { ascending: false })` は「rating の降順（大きい→小さい）に並べる」という意味で、`ascending: false` が「降順」のスイッチです。`true` にすれば昇順（小さい→大きい）になります。

```
// 評価が高い順にソート
// .order('rating', { ascending: false })
//   第1引数 'rating' : 並び替えのキーとなるカラム
//   第2引数 { ascending: false } : ascending=昇順かどうか。falseで降順（大→小）
//   trueなら昇順（小→大）。デフォルトは true（昇順）
const { data, error } = await supabase
  .from('books')
  .select('*')
  // ascending: false = 降順
  .order('rating', { ascending: false });

// 結果例:
// [
//   { title: 'ノルウェイの森', rating: 5, ... },
//   { title: '人間失格', rating: 4, ... },
//   { title: 'こころ', rating: 3, ... },
// ]
```

```
// 作成日時の新しい順にソート
// created_at は ISO文字列だが、文字列としても日時としても降順で同じ並びになる
const { data, error } = await supabase
  .from('books')
  .select('*')
  .order('created_at', { ascending: false });
```

ページネーション

▼ **このコードがやること（先に日本語で）**：データが多いときに「○件ずつ、○ページ目だけ」**取り出す**ページ送りの例です。`.range(開始, 終了)` で取得する行の範囲を指定し、`{ count: 'exact' }` で全体の総件数も一緒にもらいます。総件数を1ページ件数で割って切り上げれば**総ページ数**が分かる、という流れがポイントです。

```
// 1ページあたり10件、1ページ目を取得
// pageSize : 1ページに表示する件数
const pageSize = 10;
// page : 何ページ目を取りたいか（1始まり）
const page = 1;

// .select('*', { count: 'exact' })
//   第2引数 { count: 'exact' } で「該当行の総件数」も一緒に取得する
//   'exact'  : 正確に数える（重いが正確）
//   'planned': おおよそ（軽い）
//   'estimated': 推定値
const { data, error, count } = await supabase
  .from('books')
  // count: 'exact' で総件数も取得
  .select('*', { count: 'exact' })
  .order('created_at', { ascending: false })
  // .range(開始インデックス, 終了インデックス) : 行範囲を指定（0始まり、両端含む）
  //   1ページ目(page=1)なら 0〜9、 2ページ目(page=2)なら 10〜19 の行を取る
  // range(開始, 終了)
  .range((page - 1) * pageSize, page * pageSize - 1);

// 10件のデータ
console.log('データ:', data);
// テーブル内の全件数
console.log('総件数:', count);
// Math.ceil(x) : 切り上げ。総件数÷1ページ件数 を切り上げると総ページ数になる
// count ?? 0  : count が null/undefined なら 0 を使う（Nullish Coalescing演算子）
console.log('総ページ数:', Math.ceil((count ?? 0) / pageSize));
```

1件だけ取得

▼ **このコードがやること（先に日本語で）**：配列ではなく、**ちょうど1件のデータをオブジェクトとして取り出す**例です。`.eq('id', ...)` で対象を1件に絞り、`.single()` を付けると「配列の中の1個」ではなく「オブジェクトそのもの」が返ります。詳細ページなど「1件だけ表示したい」ときに便利ですが、結果が0件や2件以上だとエラーになる点に注意です。

```
// ID を指定して1件取得
// .eq('id', '...') で対象を1件に絞り込み
// .single() で「配列ではなく1個のオブジェクト」として受け取る
//   結果が0件 or 2件以上だとエラーになる
const { data, error } = await supabase
  .from('books')
  .select('*')
  .eq('id', '550e8400-e29b-41d4-a716-446655440000')
  // single() で1件のみ取得（配列ではなくオブジェクトが返る）
  .single();

// 結果例: 配列ではなくオブジェクトが返る
// {
//   id: '550e8400-e29b-41d4-a716-446655440000',
//   title: 'ノルウェイの森',
//   ...
// }
```

> **`.single()` の注意点:** `.single()` は結果が0件または2件以上の場合にエラーを返します。必ず1件だけ返ることが保証される場面（主キーで検索する場合など）でのみ使用してください。0件の可能性がある場合は `.maybeSingle()` を使用します（0件なら data が null になり、エラーにはなりません）。

試してみる: 0件のときの `.single()` と `.maybeSingle()`

「主キーで検索するなら必ず1件」と思いがちですが、URL の ID が古い（もう削除された本）など、0件になる場面は意外とあります。実際の違いを確かめておきましょう。

> **▼ このコードがやること（先に日本語で）:** 存在しない ID で `books` を1件取り出そうとし、`.single()` を使った場合と `.maybeSingle()` を使った場合で、戻り値がどう違うかを並べて表示します。

```ts
// 04-single.ts : 0件のときの .single() と .maybeSingle() の違い
// 実行方法: node --env-file=.env.local 04-single.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// 存在しない id（形式は正しい UUID）
const missingId = '00000000-0000-0000-0000-000000000000';

// (1) .single() : 「ちょうど1件」でなければエラー
const one = await supabase.from('books').select('title').eq('id', missingId).single();
console.log('(1) single     ->', { data: one.data, error: one.error });

// (2) .maybeSingle() : 0件なら data が null（エラーではない）
const maybe = await supabase.from('books').select('title').eq('id',
missingId).maybeSingle();
console.log('(2) maybeSingle->', { data: maybe.data, error: maybe.error });
```

▼ **実行結果**（6.5 節の作業用フォルダで実行した実際の出力）

```
(1) single     -> {
  data: null,
  error: {
    code: 'PGRST116',
    details: 'The result contains 0 rows',
    hint: null,
    message: 'Cannot coerce the result to a single JSON object'
  }
}
(2) maybeSingle-> { data: null, error: null }
```

▼ コードを1つずつ分解して解説

解説1: `process.env.〇〇!` の `!`

```ts
process.env.NEXT_PUBLIC_SUPABASE_URL!
```

- 末尾の `!` は TypeScript の **non-null アサーション**で、「この値は undefined ではないと約束する」という印です。6.5 節の `01-check.ts` のように値の有無を確かめる処理を省いて、短く書くために使っています。
- Node.js で実行するときは型の部分が取り除かれるだけなので、実際に値が無ければ、その後の処理で失敗します。本番のコード（6.3 節）では、`!` に頼らず値を確かめる書き方をしています。

解説2: PGRST116 は「1件ではなかった」という意味

```
code: 'PGRST116',
details: 'The result contains 0 rows',
message: 'Cannot coerce the result to a single JSON object'
```

- `PGRST` で始まるコードは、PostgreSQL ではなく **PostgREST**（図5-5 の翻訳係）が出すエラーです。
- `Cannot coerce the result to a single JSON object` は「結果を1つのオブジェクトにまとめられない」、`details` の `The result contains 0 rows` は「結果が0行だった」という意味です。2件以上のときは、ここに件数が入ります。

> **つまずきポイント: 0件があり得るのに `.single()` を使い、「見つからない」がエラー扱いになる**
>
> 詳細ページで「その本はもうありません」と案内したいのに、`.single()` だと 0件が `error` 側に入るため、通信エラーなどと区別しにくくなります。

```
// ❌ NG: 0件のときも error になり、「本が無い」のか「通信に失敗した」のかを区別しにくい
const { data, error } = await supabase.from('books').select('*').eq('id', id).single();
```

```
// ✅ OK: 0件は data が null（エラーではない）。本当の失敗だけが error に入る
const { data, error } = await supabase.from('books').select('*').eq('id',
id).maybeSingle();
if (error) {
  console.error('読み込みに失敗しました:', error.message);
} else if (data === null) {
  console.log('その本は見つかりませんでした');
} else {
  console.log('見つかりました:', data.title);
}
```

7.2 INSERT（データの作成）

単一行の挿入

> **▼ このコードがやること（先に日本語で）**: 新しい書籍を**1件追加する**、CRUDの「C（作成）」の基本形です。追加したい値をオブジェクトにまとめ、`.insert(...)` に渡します。`id` や `created_at` のように**DBが自動で埋める列は省略してOK**です。末尾の `.select()` を付けると、追加された行（自動生成されたidを含む）が返ってくるのがポイントです。

```ts
// BookInsert型をインポート（INSERT用のオブジェクト型）
// DEFAULTがあるidやcreated_atは省略可、NOT NULLのtitle/authorは必須
import type { BookInsert } from '@/types/book';

// 新しい書籍を1件追加
// 型注釈 : BookInsert を付けることでミスタイプ・必須項目漏れを検出できる
const newBook: BookInsert = {
  // 必須
  title: 'ノルウェイの森',
  // 必須
  author: '村上春樹',
  // 任意
  publisher: '講談社',
  // 任意。'YYYY-MM-DD' 形式の文字列
  published_date: '1987-09-04',
  // 任意（CHECK制約で 1〜5 のみ）
  rating: 5,
  // 任意。指定しないと 'want_to_read'
  status: 'completed',
  // 任意
  memo: '名作。何度も読み返したい。',
  // id, cover_url, created_at, updated_at は省略 → DBが自動で埋める
};

// .insert(newBook) : booksテーブルに newBook を1件追加するクエリ
// .select()        : 挿入された結果を返してもらう（idやcreated_atも入る）
const { data, error } = await supabase
  .from('books')
  .insert(newBook)
  // .select() を付けると、挿入したデータが返る
  .select();

if (error) {
  console.error('挿入エラー:', error.message);
  return;
}

console.log('挿入されたデータ:', data);
// 結果例:
// [
//   {
//     id: '自動生成されたUUID',
//     title: 'ノルウェイの森',
//     author: '村上春樹',
//     publisher: '講談社',
//     published_date: '1987-09-04',
//     rating: 5,
//     status: 'completed',
//     memo: '名作。何度も読み返したい。',
```

```
//     cover_url: null,
//     created_at: '2024-01-15T10:30:00.000Z',
//     updated_at: '2024-01-15T10:30:00.000Z'
//   }
// ]
```

▼ コードを1つずつ分解して解説

INSERT（作成）は「①追加する値を用意 → ② `.insert()` で送る → ③ `.select()` で結果を受け取る」という流れです。

解説1: 追加するデータをオブジェクトにまとめる

```
const newBook: BookInsert = {
  // 必須
  title: 'ノルウェイの森',
  // 必須
  author: '村上春樹',
  // 任意
  publisher: '講談社',
  // ...
  // id, cover_url, created_at, updated_at は省略 → DBが自動で埋める
};
```

- 追加したい値を1つのオブジェクト（ `{ }` ）にまとめます。キーが列名、値が入れる中身です。
- `: BookInsert` という型注釈を付けると、必須項目（ `title` ・ `author` ）の入れ忘れや、存在しない列名のタイプミスをエディタが教えてくれます。
- `id` ・ `created_at` のように **DEFAULT が設定された列は省略できます。** 省略するとDB側が自動で値を埋めます。

> **用語: 型注釈** = 変数の中身の「型（形）」を明示すること。`BookInsert` は「INSERTで渡してよいデータの形」を表す型。

解説2: .insert() で追加し、.select() で結果を受け取る

```
const { data, error } = await supabase
  .from('books')
  .insert(newBook)
  // .select() を付けると、挿入したデータが返る
  .select();
```

- `.insert(newBook)` で「 `newBook` を1件追加する」クエリになります。

- 末尾の `.select()` がポイントです。これを**付けないと**、追加は成功しても戻り値の `data` は `null` になります（空の配列 `[]` ではなく `null` です。保存した行を送り返さない分、通信が軽くなるため）。
- `.select()` を付けると、自動生成された `id` や `created_at` を含む「実際に保存された行」が `data` に返ってきます。

> **用語: insert** = テーブルに新しい行を追加する操作（SQLの INSERT に相当）。追加後の行を使いたいときは `.select()` をつなげる。

> **`.select()` を付ける理由:** `.insert()` だけだとレスポンスにデータが含まれません（パフォーマンス上の理由）。挿入したデータ（自動生成された id や created_at を含む）を取得したい場合は `.select()` を付けてください。

試してみる: `.select()` の有無で戻り値はどう変わるか

> **▼ このコードがやること（先に日本語で）:** 本を2冊追加します。1冊目は `.select()` を付けずに、2冊目は `.select(...)` を付けて追加し、それぞれの `data`・`error`・`status`（HTTP のステータス番号）を表示して比べます。

```
// 02-insert.ts : .select() を「付けない insert」と「付けた insert」の違いを見る
// 実行方法: node --env-file=.env.local 02-insert.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// (1) .select() を付けずに追加する
const first = await supabase
  .from('books')
  .insert({ title: '銀河鉄道の夜', author: '宮沢賢治' });
console.log('(1) data:', first.data, '/ error:', first.error, '/ status:',
first.status);

// (2) .select() を付けて追加する（保存された行が返ってくる）
const second = await supabase
  .from('books')
  .insert({ title: '坊っちゃん', author: '夏目漱石', rating: 4 })
  .select('title, author, rating, status');
console.log('(2) data:', second.data, '/ error:', second.error, '/ status:',
second.status);
```

▼ **実行結果**（第5節の4つのポリシーを作った状態で実行した実際の出力）

```
(1) data: null / error: null / status: 201
(2) data: [
  { title: '坊っちゃん', author: '夏目漱石', rating: 4, status: 'want_to_read' }
] / error: null / status: 201
```

- (1) も (2) も `status: 201`（HTTP で「作成できました」という意味の番号）で、追加そのものは成功しています。違うのは `data` だけで、(1) は `null`、(2) は保存された行の配列です。
- (2) の `status: 'want_to_read'` は自分では指定していない値です。CREATE TABLE の `DEFAULT 'want_to_read'` が入ったことが、`.select()` で返ってきた行から確かめられます。

> **つまずきポイント: `.select()` を付けずに、追加した本の `id` を使おうとする**
>
> 「登録したら、その本の詳細ページへ移動する」といった処理では、データベースが作った `id` が必要です。`.select()` を付けないと `data` が `null` なので、`id` を取り出せません。

```
// ❌ NG: .select() がないので data は null。data[0].id で TypeError になる
const { data, error } = await supabase.from('books').insert({ title: 'こころ', author:
'夏目漱石' });
```

```
// ✅ OK: .select('id') で、作られた行の id だけを返してもらう
const { data, error } = await supabase
  .from('books')
  .insert({ title: 'こころ', author: '夏目漱石' })
  .select('id')
  .single();
if (error) {
  console.error('登録に失敗しました:', error.message);
} else {
  console.log('作られた本の id:', data.id);
}
```

- 1件だけ追加したときは、`.select('id').single()` と続けると、配列ではなく `{ id: '...' }` という1つのオブジェクトで受け取れます。

複数行の挿入

> ▼ **このコードがやること（先に日本語で）**：1件ずつではなく、**複数の書籍をまとめて一度に追加する**例です。`.insert(...)` に**オブジェクトの配列**を渡すだけで、何件でも同時に追加できます。1件ずつループするより通信回数が減って速い、というのが利点です。

```ts
// 複数の書籍を一度に追加
// BookInsert[] : BookInsert の配列型
// 1回の通信で複数行を入れられる（個別にループするより速い）
const newBooks: BookInsert[] = [
  {
    // 1件目
    title: '人間失格',
    author: '太宰治',
    publisher: '新潮社',
    published_date: '1948-06-01',
    rating: 4,
    status: 'completed',
  },
  {
    // 2件目
    title: 'こころ',
    author: '夏目漱石',
    publisher: '岩波書店',
    published_date: '1914-09-01',
    rating: 5,
    status: 'reading',
  },
  {
    // 3件目（最小限の項目のみ）
    title: '銀河鉄道の夜',
    author: '宮沢賢治',
    // 最低限 title と author があれば OK
    status: 'want_to_read',
  },
];

// .insert に配列を渡すと「複数行同時INSERT」になる
const { data, error } = await supabase
  .from('books')
  .insert(newBooks)
  .select();

if (error) {
  console.error('挿入エラー:', error.message);
  return;
}

// data.length : 返ってきた配列の要素数（=実際に挿入できた件数）
console.log(`${data.length}件の書籍を追加しました`);
```

7.3 UPDATE（データの更新）

▼ **このコードがやること（先に日本語で）**：既存の書籍の情報を**書き換える**、CRUDの「U（更新）」の基本形です。`.update(...)` に**変えたいフィールドだけ**を渡し、`.eq('id', bookId)` で「どの行を更新するか」を指定します。**この `.eq(...)` を付け忘れると全行が書き換わってしまう**ため、対象の絞り込みが最重要ポイントです。

```ts
// BookUpdate型 : UPDATE用の型。全フィールドが optional（一部だけ更新できる）
import type { BookUpdate } from '@/types/book';

// 特定の書籍の情報を更新
// bookId : 更新対象の書籍ID（事前に取得しておく）
const bookId = '550e8400-e29b-41d4-a716-446655440000';

// updates : 書き換えたいフィールドだけを含むオブジェクト
const updates: BookUpdate = {
  // 評価を4に
  rating: 4,
  // 状態を 'completed' に
  status: 'completed',
  // メモを上書き
  memo: '読了。面白かった！',
  // title や author は省略 → 既存値のまま
};

// .update(updates) : updates の内容で行を書き換える
// .eq('id', bookId) : id が bookId の行だけ対象（必須！）
// .select() : 更新後のデータを取得
const { data, error } = await supabase
  .from('books')
  .update(updates)
  // 必ず条件を指定すること！
  .eq('id', bookId)
  .select();

if (error) {
  console.error('更新エラー:', error.message);
  return;
}

console.log('更新されたデータ:', data);
// 結果例:
// [
//   {
//     id: '550e8400-e29b-41d4-a716-446655440000',
//     title: 'ノルウェイの森',
//     rating: 4,           ← 更新された
//     status: 'completed', ← 更新された
//     memo: '読了。面白かった！', ← 更新された
//     updated_at: '2024-01-16T15:00:00.000Z', ← トリガーにより自動更新
//     ...
//   }
// ]
```

▼ コードを1つずつ分解して解説

UPDATE（更新）は「①変えたい値を用意 → ② `.update()` で送り → ③ `.eq()` で対象を絞る」という流れです。**対象の絞り込みが命綱**です。

解説1: 変えたいフィールドだけをオブジェクトにする

```
const updates: BookUpdate = {
  // 評価を4に
  rating: 4,
  // 状態を 'completed' に
  status: 'completed',
  // メモを上書き
  memo: '読了。面白かった！',
  // title や author は省略 → 既存値のまま
};
```

- 更新では「変えたい列だけ」を書きます。書かなかった列（`title` など）は既存の値のまま残ります。
- `: BookUpdate` 型は「すべての列が省略可（optional）」になっており、部分的な更新を安全に書けます。

> **用語: 部分更新** = 行のすべての列ではなく、一部の列だけを書き換えること。Supabaseの `.update()` は渡した列だけを更新する。

解説2: .eq() で「どの行を更新するか」を必ず絞る

```
const { data, error } = await supabase
  .from('books')
  .update(updates)
  // 必ず条件を指定すること！
  .eq('id', bookId)
  .select();
```

- `.update(updates)` で「`updates` の内容で書き換える」、`.eq('id', bookId)` で「`id` が `bookId` の行だけを対象にする」と指定します（eq = equal）。
- **この `.eq(...)` を付け忘れると、テーブルの全行が同じ値に書き換わります。**これは非常に危険なので、更新時は必ず対象を絞ります。
- `.select()` を付けると、更新後の行（トリガーで自動更新された `updated_at` を含む）が返ってきます。

> **用語: .eq(列, 値)** =「その列が指定した値と等しい行だけ」に絞り込むメソッド。SQLの `WHERE 列 = 値` に相当する。

> **重要: `.eq()` 等の条件を必ず指定すること！**
>
> `.update()` に条件を付けないと **テーブルの全行が更新されてしまいます**。これは非常に危険です。必ず `.eq('id', bookId)` のような条件を付けてください。

読書状態だけを更新する例

```
// ステータスだけを変更（オブジェクトには1フィールドだけ書く）
const { data, error } = await supabase
  .from('books')
  // 直接オブジェクトリテラルを渡してもOK
  .update({ status: 'completed' })
  // 対象の絞り込み（忘れずに！）
  .eq('id', bookId)
  .select();
```

試してみる: 更新が「効いたかどうか」を確かめる

UPDATE は、条件に合う行が1つも無くても**エラーになりません**（「0行を更新しました」という成功扱い）。そのため、更新が本当に効いたかは、`.select()` で返ってきた行の数で確かめるのが確実です。

> **▼ このコードがやること（先に日本語で）**: 3通りの条件で「読書状態を completed（読了）にする」更新を試します。(1) id に `'2'` のような番号を渡した場合、(2) 形式は正しいが存在しない id の場合、(3) 実在する本を題名で指定した場合です。最後に、更新できた件数を表示します。

```ts
// 05-update.ts : 更新の結果を「件数」で確かめる
// 実行方法: node --env-file=.env.local 05-update.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// (1) id を「2」のような番号で指定してしまった（id は UUID なので形式が違う）
const wrongType = await supabase
  .from('books')
  .update({ status: 'completed' })
  .eq('id', '2')
  .select();
console.log('(1)', { data: wrongType.data, error: wrongType.error });

// (2) 形式は正しいが、存在しない id を指定した
const notFound = await supabase
  .from('books')
  .update({ status: 'completed' })
  .eq('id', '00000000-0000-0000-0000-000000000000')
  .select();
console.log('(2)', { data: notFound.data, error: notFound.error });

// (3) 実在する本を title で探して更新した
const ok = await supabase
  .from('books')
  .update({ status: 'completed' })
  .eq('title', 'こころ')
  .select('title, status');
console.log('(3)', { data: ok.data, error: ok.error });

// 更新できた件数は data の長さで分かる
console.log('更新された件数:', ok.data?.length ?? 0);
```

▼ **実行結果**（実際の出力）

```
(1) {
  data: null,
  error: {
    code: '22P02',
    details: null,
    hint: null,
    message: 'invalid input syntax for type uuid: "2"'
  }
}
(2) { data: [], error: null }
(3) { data: [ { title: 'こころ', status: 'completed' } ], error: null }
更新された件数: 1
```

▼ コードを1つずつ分解して解説

解説1: 22P02 は「値の形式が列の型に合わない」

```
code: '22P02',
message: 'invalid input syntax for type uuid: "2"'
```

- 「uuid 型として読めない入力です: "2"」という意味です。`books.id` は UUID（`550e8400-...` のような36文字）なので、第0節の練習用の表のような `1`・`2` という番号は受け付けません。
- 画面の URL から id を取り出すときなどに、別の値（番号や空文字）を渡してしまうと、このエラーになります。

解説2: 0行の更新は「成功」として返ってくる

```
(2) { data: [], error: null }
```

- 条件に合う行が無かっただけなので、`error` は `null` です。`if (error)` だけで成功と判断すると、**何も更新されていないのに「更新しました」と表示する**バグになります。
- RLS で UPDATE が許されていない行も、同じく「見えない＝対象外」として0行になります（図5-11 の USING）。

解説3: `?.` と `??` で件数を安全に数える

```
console.log('更新された件数:', ok.data?.length ?? 0);
```

- `ok.data?.length` は「`data` が null でなければ `length` を読む。null なら `undefined` 」という安全な読み方です（オプショナルチェーン）。
- `?? 0` は「左側が null か undefined なら 0 を使う」という意味です（Null 合体演算子）。失敗して `data` が `null` でも、件数を 0 と表示できます。

つまずきポイント: 更新できたかを `error` だけで判断してしまう

上の (2) のように、対象が0行でも `error` は `null` です。更新の成否は「返ってきた行の数」で判断します。

```
// ❌ NG: error が無ければ成功とみなしている（0行でも「更新しました」と出てしまう）
const { error } = await supabase.from('books').update({ status: 'completed' }).eq('id',
bookId);
if (!error) {
  console.log('更新しました');
}
```

```
// ✅ OK: .select() で更新後の行を返してもらい、件数まで確かめる
const { data, error } = await supabase
  .from('books')
  .update({ status: 'completed' })
  .eq('id', bookId)
  .select('id');
if (error) {
  console.error('更新に失敗しました:', error.message);
} else if (data.length === 0) {
  console.log('対象の本が見つかりませんでした（id が違うか、RLS で許可されていません）');
} else {
  console.log('更新しました');
}
```

7.4 DELETE（データの削除）

▼ **このコードがやること（先に日本語で）**：指定した書籍を**削除する**、CRUDの「D（削除）」の基本形です。`.delete()` のあとに `.eq('id', bookId)` を付けて「どの行を消すか」を指定します。**条件を付け忘れると全行が消え、しかも復元できない**ため、削除の絞り込みは何より慎重に。ここでは結果データは使わないので `error` だけを受け取っています。

```
// 特定の書籍を削除
const bookId = '550e8400-e29b-41d4-a716-446655440000';

// .delete() : 削除クエリの宣言。括弧内に引数なし
// .eq('id', bookId) : 対象を1件に絞り込む（必須！）
// 戻り値からは data を分割代入していない → 結果は不要、errorだけ気にする
const { error } = await supabase
  .from('books')
  .delete()
  // 必ず条件を指定すること！
  .eq('id', bookId);

if (error) {
  console.error('削除エラー:', error.message);
  return;
}

console.log('書籍を削除しました');
```

▼ コードを1つずつ分解して解説

DELETE（削除）は最も慎重に扱う操作です。塊ごとに見ていきます。

解説1: .delete() に .eq() で対象を絞る

```
const { error } = await supabase
  .from('books')
  .delete()
  // 必ず条件を指定すること！
  .eq('id', bookId);
```

- `.delete()` で「削除する」クエリを宣言し、`.eq('id', bookId)` で「`id` が `bookId` の行だけ」に対象を絞ります。
- **`.eq(...)` を付け忘れると全行が削除され、しかも復元できません。**削除では対象の絞り込みが何より重要です。
- ここでは戻り値から `data` を取り出していません。削除した中身は不要で、成功・失敗（`error`）だけ確認すればよいからです。

> **用語: delete** = テーブルから行を削除する操作（SQLの DELETE に相当）。条件なしの delete は全件削除になるため、必ず絞り込む。

解説2: 失敗していないかを error で確認する

```
if (error) {
  console.error('削除エラー:', error.message);
  return;
}
```

- 他の操作と同じく、Supabaseは削除の失敗も `error` で返します。`if (error)` で失敗を検知し、メッセージを出して `return` で処理を止めます。
- 削除されたデータを確認したい場合は、このあとの例のように `.eq(...).select()` と `.select()` をつなげると、消した行の内容が `data` に返ってきます。

> **用語: error.message** = エラーの内容を人間向けに説明した文字列。ログに出すと原因を追いやすい。

> **重要: `.eq()` 等の条件を必ず指定すること！**
>
> `.delete()` に条件を付けないと **テーブルの全行が削除されてしまいます**。復元はできません。必ず条件を付けてください。

削除されたデータを確認する

```
// 削除されたデータを返り値で確認したい場合
// .select() を付けると、削除された行の内容が返ってくる
const { data, error } = await supabase
  .from('books')
  .delete()
  .eq('id', bookId)
  // select() を付けると削除されたデータが返る
  .select();

console.log('削除されたデータ:', data);
```

7.5 エラーハンドリングのパターン

Supabase の操作では、常にエラーが発生する可能性があります。以下は推奨されるエラーハンドリングのパターンです。

> **▼ このコードがやること（先に日本語で）**: DB操作で**エラーが起きたときの、おすすめの対処の型（パターン）**を関数にまとめた例です。`error` が入っていたら中身（message・code・details・hint）をログに出し、呼び出し元が気づけるよう改めてエラーを投げ直します。問題が起きたときに「何が原因か」を追いやすくするための、実戦的な書き方だと捉えてください。

```
// 汎用的なエラーハンドリング関数
// async : この関数はPromiseを返す非同期関数
async function fetchBooks() {
  // SELECTクエリの実行
  const { data, error } = await supabase
    .from('books')
    .select('*')
    .order('created_at', { ascending: false });

  // エラーが存在する場合のハンドリング
  if (error) {
    // エラーオブジェクトの中身をオブジェクトで整理して出力
    console.error('Supabase エラー:', {
      // エラーメッセージ（人間向け説明）
      message: error.message,
      // エラーコード（PostgrestErrorのコード文字列）
      code: error.code,
      // 詳細情報（DBが返したdetails）
      details: error.details,
      // ヒント（修正方法の提案。PostgreSQLが付けてくれる）
      hint: error.hint,
    });
    // throw new Error(...) : 上位関数に再スローしてUI側でcatchできるようにする
    // テンプレートリテラル `${...}` で文字列の中に変数を埋め込める
    throw new Error(`書籍の取得に失敗しました: ${error.message}`);
  }

  // 成功時はデータを返す
  return data;
}
```

試してみる: わざと失敗させて `error` の中身を観察する

エラー処理を書くときは、**実際の `error` の中身を一度見ておく**のがいちばんの近道です。4.3 節の「制約が本当に効くか試してみる」と同じ失敗を、今度は supabase-js から起こしてみましょう。

> **▼ このコードがやること（先に日本語で）**：評価（rating）に範囲外の 10 を入れた本を追加しようとして、データベースに断らせます。返ってきた `error` の `code`・`message`・`details` と、そのときの `data` を1行ずつ表示します。

```ts
// 03-error.ts : わざと失敗させて、error の中身を観察する
// 実行方法: node --env-file=.env.local 03-error.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// rating は CHECK 制約で 1〜5 しか入らない。そこへ 10 を入れてみる
const { data, error } = await supabase
  .from('books')
  .insert({ title: 'テスト', author: 'テスト', rating: 10 })
  .select();

// 失敗したら error に「なぜ失敗したか」が入り、data は null になる
if (error) {
  console.log('失敗しました');
  console.log('  code   :', error.code);
  console.log('  message:', error.message);
  console.log('  details:', error.details);
  console.log('  data は:', data);
} else {
  console.log('成功しました:', data);
}
```

▼ 実行結果（実際の出力）

```
失敗しました
  code   : 23514
  message: new row for relation "books" violates check constraint "books_rating_check"
  details: null
  data は: null
```

- `code` の `23514` は、4.3 節で SQL Editor から見たのと同じ「CHECK 制約違反」の番号です。supabase-js 経由でも、PostgreSQL のエラーがほぼそのまま届くことが分かります。
- 画面に表示するときは、英語の `message` をそのまま見せるより、`code` で場合分けして「評価は1〜5で入力してください」のような日本語に置き換えると親切です（第7章のフォームでは、送信前に画面側でもチェックします）。

よく出会うエラーの早見表（この章で実際に出力を確かめたもの）

error.code	message（抜粋）	意味	直し方
42501	permission denied for table books	表への GRANT がない（関門1）	4.3.1 節の GRANT
42501	new row violates row-level security policy	書き込みを許す RLS ポリシーがない（関門2）	5.3 節の INSERT / UPDATE ポリシー
23514	violates check constraint "books_rating_check"	CHECK 制約違反（rating が 1〜5以外 など）	値を直す。画面でも入力チェック
23502	null value in column "author" ... violates not-null constraint	必須の列が空（4.3 節では SQL Editor で確認）	必須項目を入れて送る
22P02	invalid input syntax for type uuid	値の形式が列の型に合わない	正しい UUID を渡す
PGRST116	Cannot coerce the result to a single JSON object	.single() で結果が1件でない	0件があり得るなら .maybeSingle()
PGRST205	Could not find the table 'public.book' in the schema cache	表の名前の打ち間違い（book と books）	hint の候補（'public.books'）を確認

> **ヒント:** PGRST205 のときは、hint に Perhaps you meant the table 'public.books'（もしかして public.books ですか？）のように正しい名前の候補が出ます。英語のエラーでも hint まで読むと、答えがそのまま書いてあることがよくあります。

---

8. テストデータの投入

8.1 SQL でサンプルデータを投入

Supabase の SQL Editor で以下の SQL を実行して、テスト用のサンプルデータを5件投入します。

> **▼ このコードがやること（先に日本語で）**：動作確認用に、**書籍データを5件まとめてテーブルに追加する**SQLです。
> INSERT INTO books (...) VALUES (...), (...), ... のように、VALUES のあとに括弧をカンマで並べると一度に複数行を入れられます。各行の値は、先頭で並べたカラム名と**同じ順番**で書くのがルールです。

```sql
-- ============================================
-- テストデータの投入
-- 書籍管理アプリのサンプルデータ
-- ============================================

-- INSERT INTO books : books テーブルに行を追加
-- (title, author, publisher, published_date, rating, status, memo) : 値を入れるカラム名
-- VALUES (...), (...), ... : 複数行を一度に追加する書き方（カンマで区切る）
INSERT INTO books (title, author, publisher, published_date, rating, status, memo)
VALUES
-- 1件目: ノルウェイの森
(
  -- title
  'ノルウェイの森',
  -- author
  '村上春樹',
  -- publisher
  '講談社',
  -- published_date（YYYY-MM-DD形式の日付リテラル）
  '1987-09-04',
  -- rating（1〜5の整数）
  5,
  -- status（CHECK制約で許可された値）
  'completed',
  -- memo
  '名作。静かで美しい文体に引き込まれた。何度も読み返したくなる作品。'
),
-- 2件目: 人間失格
(
  '人間失格',
  '太宰治',
  '新潮社',
  '1948-06-25',
  4,
  'completed',
  '太宰治の代表作。人間の弱さと苦悩が痛いほど伝わってくる。'
),
-- 3件目: こころ（読書中）
(
  'こころ',
  '夏目漱石',
  '岩波書店',
  '1914-09-20',
  5,
  'reading',
  '先生の手紙の部分を読んでいる。明治時代の人間関係の複雑さが興味深い。'
),
-- 4件目: 銀河鉄道の夜（未読、評価NULL）
(
```

```
  '銀河鉄道の夜',
  '宮沢賢治',
  '岩波書店',
  '1934-01-01',
  -- rating は NULL（未評価）
  NULL,
  -- まだ読みたいリストの段階
  'want_to_read',
  '友人に勧められた。幻想的な世界観が気になる。'
),
-- 5件目: コンビニ人間
(
  'コンビニ人間',
  '村田沙耶香',
  '文藝春秋',
  '2016-07-27',
  4,
  'completed',
  '芥川賞受賞作。「普通」とは何かを考えさせられた。読みやすく一気に読了。'
);
-- 末尾のセミコロンで1つのINSERT文の終わり
```

実行後、`Success. 5 rows affected.` と表示されれば成功です。

8.2 Supabase JavaScript クライアントからデータを投入する方法

SQL の代わりに、JavaScript/TypeScript コードからもテストデータを投入できます。

> **▼ このコードがやること（先に日本語で）**：先ほどのSQLと同じ「サンプルデータ5件の投入」を、**TypeScriptのコードで行う**スクリプトです。`seedBooks` 関数の中で、まず既存データを全部消してから（テスト環境を毎回まっさらにするため）、配列にまとめたデータを `.insert(...)` で一括追加しています。`async` / `await` で「DB処理の完了を待ってから次へ進む」流れになっている点に注目してください。

```ts
// テストデータの投入スクリプト（開発時のみ使用）

// 6.3で作ったSupabaseクライアントを取り込む
import { supabase } from '@/lib/supabase';
// INSERT用の型を取り込む
import type { BookInsert } from '@/types/book';

// 投入したいデータの配列
const sampleBooks: BookInsert[] = [
  {
    title: 'ノルウェイの森',
    author: '村上春樹',
    publisher: '講談社',
    published_date: '1987-09-04',
    rating: 5,
    status: 'completed',
    memo: '名作。静かで美しい文体に引き込まれた。何度も読み返したくなる作品。',
  },
  {
    title: '人間失格',
    author: '太宰治',
    publisher: '新潮社',
    published_date: '1948-06-25',
    rating: 4,
    status: 'completed',
    memo: '太宰治の代表作。人間の弱さと苦悩が痛いほど伝わってくる。',
  },
  {
    title: 'こころ',
    author: '夏目漱石',
    publisher: '岩波書店',
    published_date: '1914-09-20',
    rating: 5,
    status: 'reading',
    memo: '先生の手紙の部分を読んでいる。明治時代の人間関係の複雑さが興味深い。',
  },
  {
    title: '銀河鉄道の夜',
    author: '宮沢賢治',
    publisher: '岩波書店',
    published_date: '1934-01-01',
    // rating は省略（未評価なので NULL になる）
    status: 'want_to_read',
    memo: '友人に勧められた。幻想的な世界観が気になる。',
  },
  {
    title: 'コンビニ人間',
    author: '村田沙耶香',
    publisher: '文藝春秋',
```

```
    published_date: '2016-07-27',
    rating: 4,
    status: 'completed',
    memo: '芥川賞受賞作。「普通」とは何かを考えさせられた。読みやすく一気に読了。',
  },
];

// async関数 : 非同期処理をまとめて書ける関数
async function seedBooks() {
  // 既存データを削除（テスト環境のみ）
  // .delete() に .neq() を付けて「全件削除」を間接的に表現
  //   Supabaseは「条件なしのdelete()」を安全のため拒否することがあるので
  //   .neq('id', 存在しないID) で実質全件選択する裏ワザ
  // 戻り値オブジェクトから error だけを取り出し、別名 deleteError に
  const { error: deleteError } = await supabase
    .from('books')
    .delete()
    // 全行削除の安全策
    .neq('id', '00000000-0000-0000-0000-000000000000');

  if (deleteError) {
    console.error('削除エラー:', deleteError.message);
    return;
  }

  // サンプルデータを挿入
  // .insert(sampleBooks) で配列を一括INSERT
  // .select() で挿入されたデータ（idなど）を取得
  const { data, error } = await supabase
    .from('books')
    .insert(sampleBooks)
    .select();

  if (error) {
    console.error('挿入エラー:', error.message);
    return;
  }

  // テンプレートリテラルで件数を表示
  console.log(`${data.length}件のサンプルデータを投入しました`);
  // forEach : 配列の各要素について順番にコールバックを実行
  // (book) => { ... } : アロー関数。各要素を book として受け取る
  data.forEach((book) => {
    console.log(`  - ${book.title} (${book.author})`);
  });
}

// 実行 : 上で定義した関数を呼び出す
```

```
// async関数を呼ぶとPromiseが返るが、ここでは await せずfire-and-forget
seedBooks();
```

8.3 Supabase Dashboard での確認方法

テストデータが正しく投入されたかを Supabase Dashboard で確認しましょう。

ステップ 1: Table Editor を開く

左メニューの「Table Editor」をクリックします。

ステップ 2: books テーブルを選択

テーブル一覧から `books` をクリックします。

ステップ 3: データの確認

以下のような画面が表示されるはずです:

Table Editor > books `[+ Insert row]` `[Filter]` `[Sort]`

id	title	author	rating	status	memo
3f..	ノルウェイの森	村上春樹	5	completed	名作...
7a..	人間失格	太宰治	4	completed	太宰...
b2..	こころ	夏目漱石	5	reading	先生...
e5..	銀河鉄道の夜	宮沢賢治	NULL	want_to_read	友人...
1c..	コンビニ人間	村田沙耶香	4	completed	芥川...

> *Showing 5 rows* — Table Editor 上では、登録した行（レコード）がこのように一覧表示されます。

確認ポイント:

1. 5件のデータがすべて表示されていること
2. `id` が UUID 形式で自動生成されていること
3. `rating` の値が 1〜5 の範囲内であること（銀河鉄道の夜は NULL）
4. `status` の値が `reading` / `completed` / `want_to_read` のいずれかであること
5. `created_at` と `updated_at` が自動的に設定されていること

ステップ 4: SQL Editor でデータを確認（オプション）

SQL Editor で以下のクエリを実行して、データを確認することもできます:

▼ **このコードがやること（先に日本語で）**：投入したデータを**いろいろな角度から確認する**SELECT文の例です。`ORDER BY` で並べ替え、`WHERE` で絞り込み、`COUNT(*)` で件数を数え、`GROUP BY` で「statusごとの件数」のように**グループに分けて集計**します。集計（COUNTやGROUP BY）はデータの全体像をつかむのに便利な道具だと覚えておきましょう。

```sql
-- 全件取得
-- SELECT * : 全カラム取得
-- FROM books : booksテーブルから
-- ORDER BY created_at DESC : created_at の降順（新しい順）に並べる
SELECT * FROM books ORDER BY created_at DESC;

-- 読了済みの書籍のみ取得
-- SELECT title, author, rating : 必要なカラムだけ取得
-- WHERE status = 'completed' : status が 'completed' の行のみ
-- ORDER BY rating DESC : ratingの降順（高い順）
SELECT title, author, rating FROM books WHERE status = 'completed' ORDER BY rating
DESC;

-- 件数の確認
-- COUNT(*) : 行数を数える集約関数。テーブル全体の行数を返す
SELECT COUNT(*) FROM books;

-- ステータスごとの件数
-- GROUP BY status : 同じstatusの行をグループ化
-- COUNT(*) as count : 各グループの件数を取得し、列名を count にする
SELECT status, COUNT(*) as count FROM books GROUP BY status;
```

▼ **実行結果**（8.1 節の5件だけが入った状態で、最後の2つの SELECT を psql で実行した出力）

```
 count
-------
     5
(1 row)

    status    | count
--------------+-------
 completed    |     3
 reading      |     1
 want_to_read |     1
(3 rows)
```

- `GROUP BY status` は「status が同じ行を1つのグループにまとめる」、`COUNT(*)` は「グループの中の行数を数える」という意味です。読了3冊・読書中1冊・読みたい1冊、と集計できています。

- `GROUP BY` の結果の並び順も約束されていないので、順番をそろえたいときは `ORDER BY status` を付け足します。

発展: アプリでは使っていない重要なSupabase機能

ここまでで、books テーブルを使った基本的なデータの読み書き（CRUD）と、RLS の基礎は身につきました。この章の本編で作るアプリでは使いませんが、実際のサービスを作るときにほぼ必ず必要になる、Supabase の重要な機能を「最小サンプル」で紹介します。

> **この発展セクションの読み方**: どれも独立した小さな例なので、興味のあるものから読んで構いません。各トピックの最初に「いつ使うか・なぜ必要か」を必ず書いているので、まずはそこだけ読んで「自分のアプリに必要そうか」を判断するのがおすすめです。コードは「今すぐ書く」ものではなく「将来必要になったときに戻ってくる辞書」として使ってください。

認証（Auth）: サインアップ・ログイン・ログアウト・現在のユーザー取得

> **▼ このコードがやること（先に日本語で）**: ユーザーが**メールアドレスとパスワードで会員登録（サインアップ）し、ログイン・ログアウトする**ための一式です。「誰がアクセスしているか」をアプリが知るための仕組みで、「自分の投稿だけ表示する」「ログインしていない人には見せない」といった機能の土台になります。Supabase がパスワードの暗号化やログイン状態の管理を全部やってくれるので、私たちは用意された関数を呼ぶだけです。

```
// ① メール+パスワードで新規登録（サインアップ）
// supabase.auth       : 認証まわりの機能がまとまったオブジェクト
// .signUp(...)        : 新しいユーザーを作成する関数
// email / password    : 登録するメールアドレスとパスワード
const { data: signUpData, error: signUpError } = await supabase.auth.signUp({
  email: 'taro@example.com',
  password: 'super-secret-password',
});
if (signUpError) {
  console.error('登録に失敗:', signUpError.message);
}

// ② 登録済みのユーザーがログインする
// .signInWithPassword(...) : メールとパスワードでログインする関数
const { data: loginData, error: loginError } = await supabase.auth.signInWithPassword({
  email: 'taro@example.com',
  password: 'super-secret-password',
});
if (loginError) {
  console.error('ログインに失敗:', loginError.message);
}

// ③ 今ログインしているユーザーの情報を取得する
// .getUser() : 現在のログインユーザーを取得する関数（未ログインなら user は null）
const { data: { user } } = await supabase.auth.getUser();
// 例: taro@example.com
console.log('今ログイン中のユーザー:', user?.email);

// ④ ログアウトする
// .signOut() : ログイン状態を解除する関数
const { error: signOutError } = await supabase.auth.signOut();
if (signOutError) {
  console.error('ログアウトに失敗:', signOutError.message);
}
```

▼ コードを1つずつ分解して解説

解説1: サインアップ（新規会員登録）

```
const { data: signUpData, error: signUpError } = await supabase.auth.signUp({
  email: 'taro@example.com',
  password: 'super-secret-password',
});
```

- `supabase.auth` は、ログインまわりの機能がまとまった入れ口です。`.from('books')` がテーブル操作の入り口だったのに対し、`.auth` は「人（ユーザー）」を扱う入り口だと考えてください。

- `.signUp({ email, password })` を呼ぶと、新しいユーザーがデータベースに登録されます。パスワードはそのまま保存されるのではなく、Supabase が安全な形（暗号化したもの）に変換して保存してくれます。
- これまでのCRUDと同じで、戻り値は `{ data, error }` の形です。`data: signUpData` のように書いているのは、後のログインの `data` と名前がぶつからないように**別名をつけている**だけです。

> **用語: サインアップ** = アプリに初めて登録すること（会員登録）。**ログイン** = すでに登録済みの人が「これは私です」と認証して入ること。

解説2: ログイン（既存ユーザーの認証）

```
const { data: loginData, error: loginError } = await supabase.auth.signInWithPassword({
  email: 'taro@example.com',
  password: 'super-secret-password',
});
```

- `.signInWithPassword(...)` は「メールとパスワードが正しいか確認して、正しければログインさせる」関数です。
- ログインに成功すると、Supabase はブラウザの中に「ログイン証明書」のようなもの（トークン）を保存します。これ以降の `supabase.from('books')...` などの操作は、自動的に「このユーザーとして」実行されるようになります。
- 以降に出てくる RLS ポリシーの `auth.uid()`（自分のID）は、このログイン情報をもとに決まります。

> **用語: トークン** = 「ログイン済みであること」を証明する文字列。毎回パスワードを送らなくても、このトークンを見せれば本人だと分かる仕組み。

解説3: 今ログイン中のユーザーを取得する

```
const { data: { user } } = await supabase.auth.getUser();
console.log('今ログイン中のユーザー:', user?.email);
```

- `.getUser()` で「今このブラウザで誰がログインしているか」を取り出せます。ログインしていなければ `user` は `null`（誰もいない）になります。
- `user?.email` の `?.`（オプショナルチェーン）は、「`user` が `null` でなければ `.email` を取り出す。`null` なら何もせず `undefined` を返す」という安全な書き方です。`user` が `null` のときに `user.email` と書くとエラーになるのを防ぎます。
- 「ログインしていなければログイン画面に飛ばす」といった分岐は、この `user` が `null` かどうかで判断します。

解説4: ログアウト

```
const { error: signOutError } = await supabase.auth.signOut();
```

- `.signOut()` を呼ぶと、ブラウザに保存されていたログイン証明書（トークン）が消され、未ログイン状態に戻ります。
- ログアウト後は `getUser()` の `user` が `null` になり、自分専用のデータも見えなくなります（次のRLSの話につながります）。

認証後の厳密なRLSポリシー:「自分のデータだけ」を守る

> **▼ このコードがやること（先に日本語で）**：ログイン機能とセットで使う、**「自分が作ったデータだけ読める・書ける」**という厳しいルール（ポリシー）を SQL で定義します。本編のRLS（5.3 節）は「ログインしていなくても誰でも全部OK」という開発用の設定でしたが、実際のサービスでは「他人の家計簿や日記が見えてしまう」と大問題です。ここでは、テーブルに `user_id`（持ち主のID）列を足し、その列が**今ログインしている人のIDと一致する行だけ**操作を許可するようにします。

```sql
-- ============================================================================
-- 前提: books テーブルに「持ち主」を表す user_id 列を追加する
-- ----------------------------------------------------------------------------
-- ALTER TABLE     : 既存テーブルの構造を変更するSQL
-- ADD COLUMN      : 新しい列を追加する
-- user_id uuid    : 持ち主のユーザーIDを入れる列（型はuuid）
-- REFERENCES auth.users(id) : 「この値は auth.users テーブルの id を指す」という外部キー
--   auth.users は Supabase が認証ユーザーを管理する標準テーブル
-- DEFAULT auth.uid() : INSERT時に省略されたら「今ログイン中の人のID」が自動で入る
-- ============================================================================
ALTER TABLE books
  ADD COLUMN user_id uuid REFERENCES auth.users(id) DEFAULT auth.uid();

-- RLSを有効化（まだなら）。これが無いとポリシーは効かない
ALTER TABLE books ENABLE ROW LEVEL SECURITY;

-- (0) 本編（5.3 節）で作った開発用の「誰でもOK」ポリシーを先に消す
--     ポリシーは「どれか1つでも true なら許可」（OR）で組み合わさるため、
--     残したままだと USING (true) のせいで、全員が全部の行を見られてしまう
DROP POLICY IF EXISTS "Allow public read access" ON books;
DROP POLICY IF EXISTS "Allow public insert access" ON books;
DROP POLICY IF EXISTS "Allow public update access" ON books;
DROP POLICY IF EXISTS "Allow public delete access" ON books;

-- (0') ログインしていない人（anon）からは、書き込みの権限（4.3.1 節の GRANT）を取り上げる
REVOKE INSERT, UPDATE, DELETE ON public.books FROM anon;

-- (1) 読み取り(SELECT)ポリシー: 自分のbooksだけ見える
-- CREATE POLICY "ポリシー名" : 新しいルールを作る
-- ON books                : booksテーブルに対するルール
-- FOR SELECT              : SELECT（読み取り）のときに適用
-- USING ( 条件 )          : この条件が真の行だけ「見える」
-- auth.uid() = user_id    : 今ログイン中の人のID と 行の持ち主ID が一致する行だけ
CREATE POLICY "自分の本だけ読める"
  ON books FOR SELECT
  USING (auth.uid() = user_id);

-- (2) 追加(INSERT)ポリシー: 自分名義でしか追加できない
-- FOR INSERT           : INSERT（追加）のときに適用
-- WITH CHECK ( 条件 ) : 追加しようとする行がこの条件を満たすときだけ許可
--   USING ではなく WITH CHECK を使うのがINSERTの作法
CREATE POLICY "自分の本として追加できる"
  ON books FOR INSERT
  WITH CHECK (auth.uid() = user_id);

-- (3) 更新(UPDATE)ポリシー: 自分の本だけ更新できる
-- USING      :「どの行を更新対象にできるか」（更新前のチェック）
-- WITH CHECK :「更新後の行が満たすべき条件」（持ち主を他人に書き換えるのを防ぐ）
CREATE POLICY "自分の本だけ更新できる"
```

```
  ON books FOR UPDATE
  USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);

-- (4) 削除(DELETE)ポリシー: 自分の本だけ削除できる
CREATE POLICY "自分の本だけ削除できる"
  ON books FOR DELETE
  USING (auth.uid() = user_id);
```

▼ コードを1つずつ分解して解説

解説1: そもそも `auth.uid()` とは何か

```
auth.uid() = user_id
```

- `auth.uid()` は「**今このリクエストを送ってきたログインユーザーのID**」を返す、Supabaseが用意した関数です。先ほどの `signInWithPassword` でログインした人のIDが、ここに自動的に入ります。
- `user_id` はテーブルの各行に持たせた「その本の持ち主のID」です。
- つまり `auth.uid() = user_id` は「**この行の持ち主は、今アクセスしている本人ですか？**」という質問です。本人の行だけ真（true）になります。
- ログインしていない場合、`auth.uid()` は `null` になり、どの行とも一致しないため、結果として何も見えません（安全側に倒れる）。

> **用語: ポリシー** = 「どの行に・誰が・どの操作をしてよいか」を1つにまとめたルール。テーブルに何枚でも貼れる。

解説2: 操作ごとにポリシーを分ける（SELECT / INSERT / UPDATE / DELETE）

```
CREATE POLICY "自分の本だけ読める"  ON books FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "自分の本として追加できる" ON books FOR INSERT WITH CHECK (auth.uid() =
user_id);
```

- RLS のポリシーは「読む」「追加する」「更新する」「削除する」の**操作ごとに別々に**作るのが基本です。`FOR SELECT` / `FOR INSERT` のように、どの操作向けかを指定します。
- ポリシーを1枚も貼っていない操作は、RLS有効時には**全面的に禁止**されます（「許可リスト方式」）。だから4種類すべてに対してポリシーを用意しています。

解説3: `USING` と `WITH CHECK` の違い

```
CREATE POLICY "自分の本だけ更新できる"
  ON books FOR UPDATE
  -- どの行を触ってよいか（変更前）
  USING (auth.uid() = user_id)
  -- 変更後の行が満たすべき条件
  WITH CHECK (auth.uid() = user_id);
```

- `USING` は「**今ある行のうち、どれを対象にできるか**」の条件です。読み取りや、更新・削除の「対象選び」に使われます。
- `WITH CHECK` は「**新しく書き込もうとしている値が、ルールを満たしているか**」の条件です。追加（INSERT）や更新後（UPDATE）の値をチェックします。
- UPDATE で両方を書いているのは、「他人の本を勝手に編集できない（USING）」だけでなく「更新のついでに `user_id` を他人のものに書き換えて乗っ取る、を防ぐ（WITH CHECK）」ためです。

> **用語: 許可リスト方式** =「明示的に許可したものだけOK、それ以外は全部禁止」という考え方。セキュリティの基本姿勢。

SQL Editor で RLS を体験する（ログイン中の人になりきる）

ポリシーが本当に「自分の本だけ」を返すかは、アプリを作る前に SQL Editor で確かめられます。PostgREST が裏でやっていること（4.3.1 節の図5-9）を、手で再現するイメージです。

> **▼ このコードがやること（先に日本語で）**：取引（トランザクション）の中だけ、身分を「ログインした人（authenticated）」に切り替え、「今ログインしているのはユーザー A です」という情報をセットしてから `books` を読みます。最後に `ROLLBACK` で取引ごと取り消すので、設定はその場限りで元に戻ります。

```
-- 取引を始める（ここから ROLLBACK までの設定は、この取引の中だけで有効）
BEGIN;

-- 身分を「ログインした人」に切り替える（アプリからの通信と同じ条件にする）
SET LOCAL ROLE authenticated;

-- 「今ログインしているのはユーザー A」という情報をセットする（auth.uid() がこの値を返す）
SET LOCAL request.jwt.claim.sub = 'aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa';

-- 見える本と、それが自分の本かどうか（is_mine）を表示する
SELECT title, auth.uid() = user_id AS is_mine FROM books ORDER BY title;

-- 取引を取り消して、身分などを元に戻す
ROLLBACK;
```

▼ **実行結果**（PostgreSQL 17 の psql の出力。ユーザー A が「こころ」「坊っちゃん」、ユーザー B が「人間失格」「銀河鉄道の夜」の持ち主になっている表で実行）

```
BEGIN
SET
SET
   title    | is_mine
------------+---------
 こころ     | t
 坊っちゃん | t
(2 rows)

ROLLBACK
```

- 自分で試すときは、ダッシュボードの **Authentication** → **Users** で作ったユーザーの ID（UUID）を、`'aaaaaaaa-...'` の所に貼り付けてください。持ち主の列 `user_id` にも、そのユーザーの ID が入っている必要があります。
- `SET LOCAL ROLE anon;`（ログインしていない人）に変えて同じ SELECT をすると、`auth.uid()` が null になるので `(0 rows)` になります。
- SQL Editor の Run ボタンの近くにある、**実行するロールを選ぶメニュー**（初期表示は `postgres`）でも、同じ「なりきり」ができます（画面の表記は変わることがあります）。

> **用語: トランザクション（取引）** = 複数の SQL をひとまとまりとして扱う仕組み。`BEGIN` で始め、`COMMIT` で確定、`ROLLBACK` で全部取り消し。**SET LOCAL** = その取引の中だけ設定を変える命令。

> **つまずきポイント: 開発用の `USING (true)` ポリシーを消し忘れて、他人の本まで見える**
>
> RLS のポリシーは、同じ操作に何枚あっても「**どれか1枚でも true なら許可**」（OR）で判定されます。5.3 節の `"Allow public read access"`（USING (true)）を残したまま「自分の本だけ読める」を足しても、`true` の方が勝ってしまい、意味がありません。

```
-- ❌ NG: 開発用ポリシーを残したまま、ユーザー A になりきって SELECT した結果
    title     | is_mine
--------------+---------
 こころ       | t
 人間失格     | f
 坊っちゃん   | t
 銀河鉄道の夜 | f
(4 rows)
```

```sql
-- ✅ OK: 開発用の「誰でもOK」ポリシーを消してから、もう一度確かめる
DROP POLICY IF EXISTS "Allow public read access" ON books;
```

```
-- ✅ OK: 同じ SELECT の結果（is_mine がすべて t = 自分の本だけ）
   title    | is_mine
------------+---------
 こころ     | t
 坊っちゃん | t
(2 rows)
```

- 今あるポリシーの一覧は `SELECT policyname, cmd, qual FROM pg_policies WHERE tablename = 'books';` で確認できます。`qual` 列が `true` のポリシーが残っていないかを見るのがコツです。

Storage: 画像やファイルのアップロードと公開URLの取得

> **▼ このコードがやること（先に日本語で）**：写真やPDFのような**ファイルを Supabase に保存（アップロード）し、そのファイルを表示するためのURLを取得する**例です。テキストデータはテーブル（DB）に保存しますが、画像のような大きなファイルはDBではなく「Storage（ファイル置き場）」に入れるのが定石です。書籍の表紙画像をアップロードして `cover_url` に保存する、といった用途で使います。

```
// 前提: Supabaseダッシュボードの Storage で「book-covers」という
//       バケット（ファイルを入れる箱）を作成し、Public（公開）に設定しておく

// file は <input type="file"> でユーザーが選んだファイル（Fileオブジェクト）
async function uploadCover(file: File) {
  // 保存するときのファイル名（パス）。重複しないよう現在時刻を頭につける
  const filePath = `${Date.now()}-${file.name}`;

  // ① ファイルをアップロードする
  // supabase.storage          : ファイル置き場（Storage）を操作する入り口
  // .from('book-covers')      : 'book-covers' という箱（バケット）を選ぶ
  // .upload(保存先パス, ファイル) : 指定パスにファイルを保存する
  const { data, error } = await supabase.storage
    .from('book-covers')
    .upload(filePath, file);

  if (error) {
    console.error('アップロード失敗:', error.message);
    return null;
  }

  // ② 保存したファイルの「公開URL」を取得する
  // .getPublicUrl(パス) : 公開バケット内のファイルにアクセスできるURLを返す
  const { data: urlData } = supabase.storage
    .from('book-covers')
    .getPublicUrl(filePath);

  console.log('画像のURL:', urlData.publicUrl);
  // 例: https://xxxx.supabase.co/storage/v1/object/public/book-covers/171234-cover.png
  // このURLを books.cover_url に保存すれば表示できる
  return urlData.publicUrl;
}
```

▼ コードを1つずつ分解して解説

解説1: バケットという「ファイルの箱」

```
supabase.storage.from('book-covers')
```

- `supabase.storage` は、テーブル（DB）とは別の「ファイル置き場」を操作する入り口です。
- `.from('book-covers')` の `'book-covers'` は**バケット**の名前です。バケットは「フォルダのような大きな箱」で、用途ごと（表紙画像用、アバター用…）に分けて作ります。バケットはあらかじめダッシュボードで作っておく必要があります。
- バケットには「Public（誰でもURLで見られる）」と「Private（許可された人だけ）」の設定があります。表紙画像のように隠す必要のないものは Public が手軽です。

> **用語: バケット** = Storage の中でファイルをまとめて入れておく箱（フォルダのようなもの）。

解説2: アップロードと、保存先パスの決め方

```
const filePath = `${Date.now()}-${file.name}`;
const { data, error } = await supabase.storage
  .from('book-covers')
  .upload(filePath, file);
```

- `.upload(保存先パス, ファイル)` で、選んだファイルをバケットに保存します。
- `filePath` は「箱の中でのファイル名」です。同じ名前のファイルがあると上書きエラーになるため、`Date.now()`（今の時刻を数値にしたもの）を頭につけて**名前がかぶらないように**しています。
- `file` は、HTML の `<input type="file">` でユーザーが選んだファイルそのものです。

解説3: 公開URLを取り出してDBに保存する

```
const { data: urlData } = supabase.storage
  .from('book-covers')
  .getPublicUrl(filePath);
// urlData.publicUrl を books.cover_url に保存する
```

- `.getPublicUrl(パス)` は、保存したファイルを**ブラウザで表示するためのURL**を返します。このメソッドは通信を伴わないので `await` は不要です。
- 返ってきた `urlData.publicUrl` を、`books` テーブルの `cover_url` 列に保存しておけば、画面では `<img src={cover_url}>` のように表示できます。
- 「ファイルそのものは Storage に、その**場所を指すURLだけ**を DB に持つ」という分担がポイントです。

テーブル結合: 関連する別テーブルのデータも一緒に取得する

> **▼ このコードがやること（先に日本語で）**: ある本のデータを取るとき、**その本の「持ち主ユーザーの情報」も一回でまとめて取得する**例です。データは複数のテーブルに分けて保存しますが（本は books、ユーザー情報は profiles...）、画面に出すときは「本のタイトルと、登録した人の名前」を一緒に見せたいことがよくあります。毎回2回問い合わせる代わりに、Supabase の `select` の中で関連先を指定すると1回でまとめて取れます。

```
// 前提: books テーブルに user_id（profiles.id を指す外部キー）があり、
//       profiles テーブルに id, username, avatar_url がある状態

// .select('*, profiles(*)') :
//   '*'          → books自身の全カラム
//   profiles(*)  → 関連する profiles テーブルの全カラムも一緒に取得
const { data, error } = await supabase
  .from('books')
  .select('*, profiles(*)');

if (error) {
  console.error('取得失敗:', error.message);
} else {
  console.log(data);
  // 結果のイメージ（各bookの中に profiles が入れ子で入る）:
  // [
  //   {
  //     id: 'book-1',
  //     title: 'ノルウェイの森',
  //     user_id: 'user-AAA',
  //     profiles: { id: 'user-AAA', username: 'taro', avatar_url: null }
  //   },
  //   ...
  // ]
}
```

▼ コードを1つずつ分解して解説

解説1: `select` の中に別テーブル名を書く

```
.select('*, profiles(*)')
```

- これまで `select('*')` は「このテーブルの全カラム」でした。ここに `, profiles(*)` を足すと、「**外部キーでつながっている profiles テーブルの全カラムも一緒にちょうだい**」という意味になります。
- `profiles(*)` の `(*)` は「profiles の全カラム」。`profiles(username, avatar_url)` のように、欲しい列だけ指定することもできます。
- これが成立するのは、`books.user_id` が `profiles.id` を指す**外部キー**として設定されているからです。Supabase はその関係を見て「どの行とどの行がつながっているか」を自動で判断します。

> **用語: 結合（JOIN）** = 関連する複数のテーブルを、共通のID（外部キー）でつなげて1つの結果としてまとめること。

解説2: 結果は「入れ子（ネスト）」になって返る

```
// book.profiles.username のように、本の中にユーザー情報が入っている
```

- 結合した結果は、SQLの表のように横に広がるのではなく、**JavaScriptのオブジェクトの入れ子**として返ってきます。各 `book` の中に `profiles` というプロパティができ、その中に持ち主の情報が入ります。
- 画面では `book.profiles.username`（登録者名）のように、ドットでたどって取り出せます。
- 「1回の通信で必要な情報をまとめて取る」ことで、表示が速くなり、コードもシンプルになります。

高度なフィルタ: or・複数条件・in・range

> **▼ このコードがやること（先に日本語で）:「もっと細かい条件でデータを絞り込む」**ための道具を整理します。本編では `.eq()`（〜と等しい）だけ使いましたが、実際には「AまたはB」「リストのどれかに当てはまる」「上位20件だけ」のように、いろいろな絞り込みが必要になります。検索機能やページめくり（ページネーション）を作るときに必須の道具です。

```
// ① 複数条件（AND）：メソッドを並べると「すべて満たす」になる
// .eq('status', 'reading') : status が 'reading'
// .gte('rating', 4)        : rating が 4 以上（gte = greater than or equal）
const { data: reading } = await supabase
  .from('books')
  .select('*')
  .eq('status', 'reading')
  .gte('rating', 4);

// ② OR条件：「どちらか一方でも満たせばOK」
// .or('文字列') ： カンマ区切りで複数条件を並べ、いずれか真ならマッチ
const { data: orResult } = await supabase
  .from('books')
  .select('*')
  // 読了済み または 評価5
  .or('status.eq.completed,rating.eq.5');

// ③ in：「このリストのどれかに当てはまる」
// .in('カラム', [値1, 値2, ...]) ： 値のどれかと一致する行
const { data: inResult } = await supabase
  .from('books')
  .select('*')
  // reading か completed の本
  .in('status', ['reading', 'completed']);

// ④ range：「何件目から何件目まで」を取り出す（ページめくり用）
// .range(開始index, 終了index) ： 0始まりの番号で範囲指定（両端を含む）
const { data: page1 } = await supabase
  .from('books')
  .select('*')
  .order('created_at', { ascending: false })
  // 0〜19番 = 最初の20件
  .range(0, 19);
```

▼ コードを1つずつ分解して解説

解説1: メソッドを並べると AND（かつ）になる

```
.eq('status', 'reading')
.gte('rating', 4)
```

- 絞り込みメソッドを `.` で続けて並べると、「**そのすべてを満たす行**」だけが残ります（AND条件）。上の例は「読書中 **かつ** 評価4以上」です。
- `.gte` は "greater than or equal"（以上）の略です。仲間に `.gt`（より大きい）、`.lte`（以下）、`.lt`（未満）、`.neq`（等しくない）があります。

> **用語: AND条件** = 「AもBも両方とも満たす」絞り込み。**OR条件** = 「AかBのどちらか一方でも満たせばよい」絞り込み。

解説2: `.or()` で「どちらか一方」を表す

```
.or('status.eq.completed,rating.eq.5')
```

- `.or()` は AND とは逆で、「**並べた条件のどれか1つでも満たせばマッチ**」させます。
- 条件の書き方は少し独特で、`カラム名.演算子.値` を**カンマ区切り**でつなぎます。`status.eq.completed` は「status が completed と等しい」、`rating.eq.5` は「rating が 5」を意味します。
- 上の例は「読了済み、**または** 評価が5の本」を取り出します。

解説3: `.in()` で「リストのどれか」を表す

```
.in('status', ['reading', 'completed'])
```

- `.in('カラム', [値の配列])` は、「そのカラムが、配列の中のどれかと一致する行」を取り出します。
- 上の例は「status が reading **または** completed の本」で、`.or()` でも書けますが、同じカラムで候補が多いときは `.in()` の方がずっと簡潔です。

解説4: `.range()` でページめくり（ページネーション）

```
.order('created_at', { ascending: false })
.range(0, 19);
```

- `.range(開始, 終了)` は「**何番目から何番目までの行**」を取り出します。番号は0から始まり、両端を含みます。`range(0, 19)` は最初の20件です。
- 次のページは `range(20, 39)`、その次は `range(40, 59)` …と進めれば、「1ページ20件ずつ表示する」機能が作れます。これを**ページネーション**と呼びます。
- 範囲を指定する前に `.order(...)` で**並び順を固定**しておくのが重要です。順番が決まっていないと「何番目」が毎回ばらついてしまうためです。

> **用語: ページネーション** = 大量のデータを「1ページ◯件ずつ」に分けて表示し、ページを切り替えられるようにする仕組み。

upsert: あれば更新・無ければ挿入

> **▼ このコードがやること（先に日本語で）:「そのデータがすでにあれば上書き更新し、無ければ新しく追加する」**という、insert と update を1回でこなす便利な操作です。たとえば「ユーザー設定」や「お気に入り」のように、「初回は新規作成、2回目以降は更新」となるケースで、毎回「存在するか確認してから insert か update を選ぶ」という面倒な分岐を書かずに済みます。

```
// upsert = update + insert
// .upsert(データ, { onConflict: 'カラム名' }) :
//   指定カラムの値が既存の行と「かぶったら」→ 更新
//   かぶらなければ                          → 新規挿入
const { data, error } = await supabase
  .from('books')
  .upsert(
    { id: 'book-123', title: 'リーダブルコード', status: 'completed' },
    // id がかぶったら更新、なければ挿入
    { onConflict: 'id' }
  )
  // 挿入/更新後の行を返してもらう
  .select();

if (error) {
  console.error('upsert失敗:', error.message);
} else {
  console.log('保存結果:', data);
}
```

▼ コードを1つずつ分解して解説

解説1: insert と update を1つにまとめる

```
.upsert(
  { id: 'book-123', title: 'リーダブルコード', status: 'completed' },
  { onConflict: 'id' }
)
```

- `.upsert(データ, オプション)` は、渡したデータが既存の行と「ぶつかる」かどうかで、自動的に挿入か更新かを切り替えます。
- `{ onConflict: 'id' }` は「**`id` 列の値がすでにある行とかぶったら、新規追加ではなく更新する**」という指定です。`id` が `'book-123'` の行が既にあれば中身を上書きし、無ければ新しく1行追加します。
- これがないと、同じ `id` を insert しようとしたときに「重複エラー」で失敗します。upsert はそのエラーを「更新」に振り替えてくれる、と考えると分かりやすいです。

> **用語: upsert** = update（更新）と insert（挿入）を合わせた造語。「あれば更新・無ければ挿入」を一発でやる操作。**onConflict** = 「どの列の重複を『同じデータ』とみなすか」の指定。

解説2: 最後の `.select()` で結果を受け取る

```
.upsert(...).select();
```

- `.upsert(...)` だけでは、保存はされても結果のデータは返ってきません。うしろに `.select()` をつけると、**挿入または更新された後の行**を `data` で受け取れます。
- 「保存したらすぐ画面に最新の内容を反映したい」ときに便利です。

Realtime購読: テーブルの変更をリアルタイムに受け取る

> **▼ このコードがやること（先に日本語で）: データベースが変わった瞬間に、自分の画面へ自動で通知を受け取る**仕組みです。普通は「再読み込みボタンを押す」「ページを開き直す」をしないと新しいデータは見えませんが、Realtime を使うと、誰かが本を追加した瞬間に自分の一覧へ即座に反映できます。チャットアプリの新着メッセージや、複数人で同時に使う管理画面などで活躍します。

```
// supabase.channel('名前') : リアルタイム通知の受信チャンネルを作る
// .on('postgres_changes', { ... }, コールバック) :
//   DBの変更が起きたら、第3引数の関数を呼んでもらう設定
const channel = supabase
  .channel('books-changes')
  .on(
    'postgres_changes',
    {
      // INSERT/UPDATE/DELETE すべての変更を対象にする
      event: '*',
      // 対象スキーマ（通常はpublic）
      schema: 'public',
      // 監視するテーブル名
      table: 'books',
    },
    (payload) => {
      // payload に「何が起きたか」の情報が入って渡される
      // 'INSERT' など
      console.log('変更を検知:', payload.eventType);
      // 追加/更新後のデータ
      console.log('新しい行:', payload.new);
    }
  )
  // ① ここで実際に購読（受信）を開始する
  .subscribe();

// 画面を閉じるときなどは購読を解除して後始末する
// supabase.removeChannel(channel) : このチャンネルの受信を停止する
//  （Reactなら useEffect の後始末関数の中で呼ぶ）
// supabase.removeChannel(channel);
```

▼ コードを1つずつ分解して解説

解説1: チャンネルを作り、「何を監視するか」を指定する

```
supabase
  .channel('books-changes')
  .on('postgres_changes', { event: '*', schema: 'public', table: 'books' }, (payload)
=> { ... })
```

- `.channel('books-changes')` は「リアルタイム通知を受け取る専用の窓口」を1つ作ります。名前（`'books-changes'`）は自分で分かりやすいものを付けるだけです。
- `.on('postgres_changes', {条件}, 関数)` で「**books テーブルに変更が起きたら、この関数を呼んで**」と登録します。
- `event: '*'` は「追加・更新・削除すべて」。特定の操作だけ見たいなら `'INSERT'` などにします。

- `table: 'books'` で監視対象のテーブルを指定します。
- 変更が起きると、登録した関数に `payload`（変更内容の詳細）が渡されます。`payload.new` に新しいデータ、`payload.eventType` に「INSERT/UPDATE/DELETE のどれか」が入っています。

> **用語: 購読（subscribe）** =「変化があったら教えてね」と登録して、通知を受け取り続けること。新聞の定期購読のイメージ。**コールバック** =「何かが起きたときに呼んでもらう関数」。

解説2: subscribe で開始し、removeChannel で後始末する

```
// 受信を開始
.subscribe();
// 受信を停止
// supabase.removeChannel(channel);
```

- `.subscribe()` を呼んで初めて、実際に通知の受信が始まります。これを忘れると、設定はしたのに何も届きません。
- 受信は「つなぎっぱなし」になるため、画面を閉じるときなどには `supabase.removeChannel(channel)` で**必ず後始末**します。後始末しないと、見えない場所で通信が残り続け、動作が重くなる原因になります。
- React で使う場合は、`useEffect` の中で `subscribe()` し、その**後始末関数（return する関数）**の中で `removeChannel` を呼ぶ、という形が定番です。

> **補足: Realtime は最初はオフ:** Realtime はテーブルごとに有効化が必要です。Supabaseダッシュボードの該当テーブルで「Realtime」をオンにする（または Replication 設定に追加する）と、上のコードで変更が届くようになります。

---

9. トラブルシューティング

9.1 接続エラー

エラー: `FetchError: request to https://xxxxx.supabase.co/rest/v1/books failed`

原因: Supabase に接続できない。

対処法:

① 環境変数を確認する

```
# .env.local の内容を確認（ターミナルで実行）
# cat : ファイルの中身を表示するUnixコマンド（Windows の PowerShell でも使えます）
cat .env.local
```

- `NEXT_PUBLIC_SUPABASE_URL` が正しいか確認
- `NEXT_PUBLIC_SUPABASE_ANON_KEY` が正しいか確認
- URL の末尾にスラッシュ `/` がないか確認（不要です）
- 余分な空白や改行がないか確認

② プロジェクトが起動しているか確認

- Supabase Dashboard にアクセスして、プロジェクトが「Active」状態か確認
- 一時停止されている場合は「Restore project」をクリック

③ 開発サーバーを再起動

```
# 環境変数の変更はサーバー再起動が必要
# Ctrl + C で停止してから
# npm run dev : Next.jsの開発サーバーを起動するコマンド
npm run dev
```

エラー: `TypeError: fetch failed` または `ECONNREFUSED`

原因: ネットワーク接続の問題。

対処法:

1. インターネットに接続されているか確認
2. VPN を使用している場合は一時的に無効化してみる
3. ファイアウォールが Supabase への接続をブロックしていないか確認

9.2 RLS でデータが取得できない

「データが返ってこない」ときは、やみくもに設定を変える前に、**どこで止まっているか**を順番に切り分けるのが近道です。次の図の順に確かめてください。



図5-14: データが返らないときの切り分け。まず error、次に data が空かどうかを見る

症状: `data` が空配列 `[]` で返ってくる（エラーは出ない）

これは Supabase で最もよくあるハマりポイントです。

原因: RLS（Row Level Security）が有効だが、アクセスを許可するポリシーが設定されていない。

確認手順:

1. ダッシュボードの **Policies** 画面（左メニューの「Authentication」または「Database」の中にあります。画面の構成は更新で変わることがあります）で books テーブルのポリシーを確認するか、下の `pg_policies` を使った SQL で確認
2. ポリシーが存在するか、`USING (true)` になっているか確認

対処法:

```
-- 現在のポリシーを確認
-- pg_policies : PostgreSQLが自動で持つ「ポリシー一覧」を見られるシステムビュー
-- WHERE tablename = 'books' : booksテーブルのポリシーだけを表示
SELECT * FROM pg_policies WHERE tablename = 'books';
```

ポリシーが存在しない場合は、セクション5の SQL を再実行してください:

```sql
-- RLS が有効になっているか確認
-- pg_class : テーブル等のメタデータを持つシステムカタログ
-- relname  : 名前（リレーション名）
-- relrowsecurity : RLSが有効か（true/false）
SELECT relname, relrowsecurity
FROM pg_class
WHERE relname = 'books';
-- relrowsecurity が true なら RLS が有効

-- ポリシーがない場合は追加
-- SELECT用ポリシー
CREATE POLICY "Allow public read access"
  ON books
  FOR SELECT
  USING (true);

-- INSERT用ポリシー
CREATE POLICY "Allow public insert access"
  ON books
  FOR INSERT
  WITH CHECK (true);

-- UPDATE用ポリシー
CREATE POLICY "Allow public update access"
  ON books
  FOR UPDATE
  USING (true)
  WITH CHECK (true);

-- DELETE用ポリシー
CREATE POLICY "Allow public delete access"
  ON books
  FOR DELETE
  USING (true);
```

もう一つの対処法（開発時のみ）：

一時的に RLS を無効化して、RLS が原因かどうかを切り分けることもできます。

```sql
-- ⚠️ 開発環境でのみ使用すること！
-- DISABLE ROW LEVEL SECURITY : RLSを OFF にする
-- 本番でこれをやると全データが世界に公開される
ALTER TABLE books DISABLE ROW LEVEL SECURITY;
```

これでデータが取得できるようになったら、RLS のポリシー設定に問題があることが確定します。問題を解決したら必ず RLS を再度有効化してください:

```
-- 再度RLSを有効化（本番運用前に必須）
ALTER TABLE books ENABLE ROW LEVEL SECURITY;
```

エラー: `permission denied for table books`（コード `42501`）

原因: テーブルが Data API に公開されていない（GRANT がない）。2026年5月30日以降に作ったプロジェクトでは、新しいテーブルは自動では公開されません。

対処法: 4.3.1 節の GRANT の SQL を実行してください。RLS のポリシーをいくら直しても、このエラーは消えません（図5-9の関門1で止まっているため）。

症状: INSERT はできるが SELECT でデータが返らない

原因: SELECT のポリシーが設定されていない、または条件が間違っている。

対処法: 上記の SELECT ポリシーを確認してください。

9.3 型エラー

エラー: `Property 'books' does not exist on type 'Database'`

原因: TypeScript の型定義ファイルが古いか、生成されていない。

対処法:

```
# 型定義を再生成
# package.json の gen:types スクリプトを実行
npm run gen:types
```

エラー: `Type 'string' is not assignable to type 'number'`

原因: Supabase から返されるデータの型と、コードで期待している型が一致していない。

対処法:

```
// ❌ 悪い例: 手動で型を定義している（抜粋）
// UUID は string なのに number にしている → 型不一致
interface Book {
  id: number;
  title: string;
}
```

```
// ✅ 良い例: 自動生成された型（6.4 で作った src/types/book.ts）を取り込んで使う
// Book の中身は Database['public']['Tables']['books']['Row'] と同じで、id は string になっている
import type { Book } from '@/types/book';
```

- 同じファイルの中で `Book` という名前を `import` と `interface` の両方で作ると、名前がぶつかって別のエラーになります。手で書いた型は消して、自動生成された型に一本化しましょう。

エラー: `Could not find a declaration file for module '@supabase/supabase-js'`

原因: `@supabase/supabase-js` がインストールされていないか、TypeScript の設定に問題がある。

対処法:

```
# パッケージを再インストール
npm install @supabase/supabase-js

# node_modules を削除して再インストール
# rm -rf : ディレクトリを再帰的・強制的に削除（Unix）
# Windowsの場合は: rmdir /s /q node_modules && del package-lock.json
rm -rf node_modules package-lock.json
npm install
```

9.4 SQL エラー

エラー: `relation "books" already exists`

原因: テーブルがすでに存在する状態で `CREATE TABLE` を実行した。

対処法:

```
-- テーブルを削除してから再作成（データも消える）
-- DROP TABLE : テーブルを削除するSQL
-- IF EXISTS  : 存在する場合のみ実行（存在しなくてもエラーにならない）
DROP TABLE IF EXISTS books;

-- または、テーブルが存在しない場合のみ作成
-- CREATE TABLE IF NOT EXISTS : 存在しない場合のみ作成
CREATE TABLE IF NOT EXISTS books (
  -- ...
);
```

エラー: `new row for relation "books" violates check constraint "books_rating_check"`

原因: `rating` カラムに 1〜5 の範囲外の値を挿入しようとした。

対処法:

```
// 悪い例: 範囲外の値
// CHECK違反
const book = { title: 'テスト', author: 'テスト', rating: 10 };

// 良い例: 1〜5 の範囲内の値
const book = { title: 'テスト', author: 'テスト', rating: 5 };
```

エラー: `new row for relation "books" violates check constraint "books_status_check"`

原因: `status` カラムに許可されていない値を挿入しようとした。

対処法:

```
// 悪い例: 許可されていない値
// CHECK違反
const book = { title: 'テスト', author: 'テスト', status: 'done' };

// 良い例: 許可された値のいずれか
const book = { title: 'テスト', author: 'テスト', status: 'completed' };
// 許可された値: 'reading' | 'completed' | 'want_to_read'
```

9.5 よくある質問

Q: anon key（Publishable key）が漏れたらどうなりますか？

A: anon key は「公開キー」なので、ブラウザの JavaScript から見える前提で設計されています。RLS を正しく設定していれば、anon key だけではデータに不正アクセスできません。ただし、RLS を無効にしている場合は危険です。本番環境では必ず RLS を有効にしてください。

Q: service_role key（Secret key）はいつ使いますか？

A: `service_role` key は RLS をバイパスする管理者用のキーです。**絶対にフロントエンドに含めないでください。** サーバーサイドのバッチ処理やマイグレーションスクリプトでのみ使用します。漏えいすると全データの読み書きが攻撃者に許されるため、Gitリポジトリ・公開URL・ブラウザJSに含めるのは厳禁です。

Q: テーブルの定義を変更したい場合は？

A: SQL Editor で `ALTER TABLE` コマンドを使用します。変更後は `npm run gen:types` で型定義を再生成してください。

> **▼ このコードがやること（先に日本語で）**：作成済みのテーブルに対して、**列を後から追加・削除したり、列の型を変えたりする**SQLの例です。共通して `ALTER TABLE`（テーブル定義を変更する命令）で始まり、`ADD COLUMN`（追加）・`DROP COLUMN`（削除）・`ALTER COLUMN ... TYPE`（型変更）と続けます。`DROP COLUMN` はその列のデータも消える点に注意してください。

```
-- カラムの追加
-- ALTER TABLE : テーブル定義の変更
-- ADD COLUMN  : 新しいカラムを追加
-- page_count integer : カラム名と型
ALTER TABLE books ADD COLUMN page_count integer;

-- カラムの削除
-- DROP COLUMN : 既存カラムを削除（データも消える）
ALTER TABLE books DROP COLUMN page_count;

-- カラムの型変更
-- ALTER COLUMN ... TYPE 新しい型 : カラムの型を変える
-- smallint : 小さい整数型（-32768〜32767）
ALTER TABLE books ALTER COLUMN rating TYPE smallint;
```

練習問題（解答・解説つき）

この章の内容が身に付いたかを確かめましょう。まず自分で考えてから、後ろの「解答と解説」を読んでください。SQL の問題は SQL Editor で、コードの問題は 6.5 節の作業用フォルダ（`supabase-playground`）で実際に試すと、理解がぐっと深まります。問題1・6は、8.1 節のサンプル5冊が入った状態を前提にしています。

問題1: 条件と並び替え（SQL）

`books` テーブルから、評価（rating）が4以上の本の「題名（title）と評価」を、**評価の高い順**に表示する SQL を書いてください。評価が同じ本どうしは、**出版日（published_date）の古い順**に並べます。

問題2: 危ない DELETE（SQL）

「銀河鉄道の夜」だけを削除するつもりで、次の SQL を実行しようとしています。何が起きるかと、安全な直し方を答えてください。

```
DELETE FROM books;
```

問題3: エラーメッセージを読む（GRANT）

2026年10月に新しく作ったプロジェクトで、`books` テーブルを SQL で作り、5.3 節のポリシーもすべて作りました。ところが supabase-js で読むと、次のエラーが返ってきます。原因と直し方を答えてください。

```
error: { code: '42501', details: null, hint: null, message: 'permission denied for table books' }
```

問題4: エラーが無いのに空っぽ（RLS）

supabase-js で `books` を読むと、`error` は `null` なのに `data` が `[]`（空の配列）です。考えられる原因を2つ挙げ、それぞれをどう確かめるかを答えてください。

問題5: キーの置き方（セキュリティ）

次の (a)〜(d) のうち、**やってはいけない**ものをすべて選んでください。

- (a) `.env.local` に `NEXT_PUBLIC_SUPABASE_ANON_KEY=sb_publishable_...` と書く
- (b) `.env.local` に `NEXT_PUBLIC_SUPABASE_SECRET_KEY=sb_secret_...` と書く
- (c) サーバー側の処理だけで使うため、`.env.local` に `SUPABASE_SECRET_KEY=sb_secret_...` と書く
- (d) チームで共有するために、`.env.local` を GitHub の公開リポジトリにコミットする

問題6: 3つのパターンを書き分ける（supabase-js）

読書状態が `reading`（読書中）の本の題名を、題名の順に1行ずつ `- こころ` のような形で表示するスクリプト `q6-reading.ts` を書いてください。読み込みに失敗したときはエラーメッセージを、1冊も無いときは「読書中の本はありません」と表示します。

問題7: INSERT のポリシー（USING と WITH CHECK）

次の SQL を実行すると、エラーになります。なぜエラーになるのかと、正しい書き方を答えてください。

```
CREATE POLICY "bad insert" ON books FOR INSERT USING (true);
```

解答と解説

問題1 の解答

```sql
-- rating が 4 以上の本だけを取り出し、評価の高い順、同じ評価なら出版日の古い順に並べる
SELECT title, rating
FROM books
WHERE rating >= 4
ORDER BY rating DESC, published_date;
```

```
     title      | rating
----------------+--------
 こころ         |      5
 ノルウェイの森 |      5
 人間失格       |      4
 コンビニ人間   |      4
(4 rows)
```

- `WHERE rating >= 4` で「4以上」に絞り、`ORDER BY rating DESC` で「大きい順」にします（`DESC` を付けないと小さい順）。
- `ORDER BY` はカンマで区切ると「1つ目が同じなら2つ目で比べる」という意味になります。評価5の「こころ」（1914年）と「ノルウェイの森」（1987年）は、出版日の古い順に並びました。
- 評価が NULL の「銀河鉄道の夜」は、`NULL >= 4` が true にならないので出てきません。

問題2 の解答

`WHERE` が無いので、**全部の本（5冊）が削除されます**。しかも元に戻す命令はありません（図5-3・図5-4）。次のように、先に SELECT で対象を確かめてから、同じ条件で DELETE します。

```sql
-- まず、消す予定の行が1行だけかを確かめる
SELECT id, title FROM books WHERE title = '銀河鉄道の夜';

-- 確かめた条件をそのまま付けて削除する
DELETE FROM books WHERE title = '銀河鉄道の夜';
```

- 実行後のメッセージが `DELETE 1` であることも確認しましょう。同じ題名の本が複数あるかもしれないときは、SELECT で確かめた `id` を使って `WHERE id = '...'` と指定するのが確実です。

問題3 の解答

原因: テーブルが Data API に公開されていません（GRANT がない）。2026年5月30日以降に作ったプロジェクトでは、新しく作ったテーブルは自動では公開されないためです。RLS のポリシー（図5-9 の関門2）より手前の、関門1で止まっています。

直し方: 4.3.1 節の GRANT を実行します。

```
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO anon;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO authenticated;
GRANT SELECT, INSERT, UPDATE, DELETE ON public.books TO service_role;
```

- 同じ `42501` でも、メッセージが `new row violates row-level security policy` なら原因は RLS のポリシー側です。コードの番号だけでなく、英文のメッセージまで読んで区別しましょう。

問題4 の解答

1. **RLS の SELECT ポリシーが無い（または条件が合わない）**: RLS はポリシーが無いと全行を隠し、エラーは出しません。SQL Editor で `SELECT * FROM books;` を実行して行が見えるなら、こちらが原因です。`SELECT policyname, cmd, qual FROM pg_policies WHERE tablename = 'books';` で、`cmd` が `SELECT` のポリシーがあるかを確かめます。
2. **そもそも条件に合う行が無い**: テーブルが空（8.1 節のデータを入れていない）か、`.eq('status', 'readng')` のように条件の値を打ち間違えている場合です。SQL Editor で同じ条件の SELECT を実行して確かめます。

- どちらも「`error` が `null` の0件」なので、`error` だけを見ていても気づけません（図5-13 の②、図5-14）。

問題5 の解答

(b) と (d) です。

- (b): 秘密キーに `NEXT_PUBLIC_` を付けると、ブラウザ向けのファイルに埋め込まれて全世界に公開されます。RLS も素通りするので、だれでも全データを消せてしまいます（図5-7）。
- (d): `.env.local` は、公開用キーしか書いていなくても Git にコミットしません。`.gitignore` の `.env*.local` で除外されているのはそのためです。チームで共有したいときは、変数名だけを書いた見本ファイル（例: `.env.example`）を用意し、値は個別に渡します。
- (a) は本書の標準の書き方、(c) は「サーバー側だけで使う」なら正しい置き方です（本書のアプリでは秘密キーは使いません）。

問題6 の解答

```ts
// q6-reading.ts : 読書中の本の題名を表示する
// 実行方法: node --env-file=.env.local q6-reading.ts
import { createClient } from '@supabase/supabase-js';

const supabase = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
);

// status が reading の本の title だけを、title の順に取り出す
const { data, error } = await supabase
  .from('books')
  .select('title')
  .eq('status', 'reading')
  .order('title');

// ① 失敗 → ② 0件 → ③ 1件以上 の順に調べる
if (error) {
  console.error('読み込みに失敗しました:', error.message);
} else if (data.length === 0) {
  console.log('読書中の本はありません');
} else {
  for (const book of data) {
    console.log(`- ${book.title}`);
  }
}
```

▼ 実行結果（上は 8.1 節のサンプル5冊が入った状態、下は「こころ」を読了に変えて読書中の本を無くした状態で実行した実際の出力）

```
$ node --env-file=.env.local q6-reading.ts
- こころ

$ node --env-file=.env.local q6-reading.ts
読書中の本はありません
```

- 図5-13 の3パターン（失敗・0件・行がある）を、`if` → `else if` → `else` の順で書き分けるのがポイントです。`error` を最初に調べるので、`data` が `null` のときに `data.length` を読んでしまう心配がありません。
- `for (const book of data)` は、配列の要素を1つずつ `book` に入れて繰り返す書き方です。

問題7 の解答

実際に実行すると、次のエラーになります。

```
ERROR:  only WITH CHECK expression allowed for INSERT
```

INSERT は「まだ存在しない、これから書き込む行」を調べる操作なので、「既存の行のうちどれを対象にするか」を決める `USING` は使えません。`WITH CHECK` で書きます（図5-11）。

```
-- INSERT のポリシーは WITH CHECK で書く
CREATE POLICY "Allow public insert access"
  ON books
  FOR INSERT
  WITH CHECK (true);
```

- 覚え方: **SELECT・DELETE は USING だけ、INSERT は WITH CHECK だけ、UPDATE は両方。**

まとめ

この章で学んだことを振り返りましょう。



この章で達成したこと:

1. Supabase アカウントを作成し、プロジェクトをセットアップした
2. リレーショナルデータベースの基礎を学んだ
3. 書籍管理アプリの `books` テーブルを設計・作成した
4. GRANT でテーブルを Data API に公開し、RLS で「どの行を触れるか」を設定した
5. Supabase クライアントをセットアップした
6. CRUD の全操作（SELECT / INSERT / UPDATE / DELETE）を学んだ
7. テストデータを投入してデータベースの動作を確認した

次の章では:

第6章では、ここで作成した Supabase データベースと React のフロントエンドを接続し、実際に動く書籍管理アプリの画面を構築していきます。